GEMPRO Notebook - Extended

GEMPRO

Índice

1 Contest **4**
- 1.1 template.cpp . . . 4
- 1.2 .vimrc . . . 4
- 1.3 .bashrc . . . 4
- 1.4 Template de stress testing . . . 4
  - 1.4.1 Stress testing reproduzível . . . 4

2 Matemática **5**
- 2.1 Algoritmo euclideano . . . 5
- 2.2 Potência módulo `mod` . . . 5
- 2.3 CRT . . . 5
- 2.4 Crivo . . . 5
- 2.5 Teste de primalidade . . . 5
- 2.6 Fatoração de inteiros . . . 6
- 2.7 Raiz quadrada módulo `mod` . . . 6
- 2.8 Logaritmo discreto . . . 6
- 2.9 Soma de floor . . . 7
- 2.10 Mínimo de progressão modular . . . 7
- 2.11 Somas de totiente e Möbius . . . 7
- 2.12 Permutação para inteiro . . . 8
- 2.13 Matrizes sobre semianéis . . . 8
- 2.14 Eliminação gaussiana . . . 9
- 2.15 Base de XOR . . . 9
- 2.16 Interpolação de Lagrange . . . 10
- 2.17 Polinômio de Lagrange . . . 10
- 2.18 Séries formais: inversa, logaritmo e exponencial . . . 10
- 2.19 Berlekamp-Massey . . . 11
- 2.20 Recorrência linear . . . 11
- 2.21 Bostan–Mori . . . 11
- 2.22 Integração adaptativa (Simpson) . . . 12

3 Estruturas de Dados **12**
- 3.1 Busca binária . . . 12
- 3.2 Busca ternária . . . 12
- 3.3 Maior subsequência crescente . . . 12
- 3.4 Subset sum . . . 13
- 3.5 Disjoint set union com rollback . . . 13
- 3.6 Union-find de intervalos paralelos . . . 13
- 3.7 Sparse table . . . 14
- 3.8 Árvore de Fenwick . . . 14
- 3.9 Deque de monoide . . . 14
- 3.10 Segtree simples . . . 15
- 3.11 Segtree . . . 15
- 3.12 Lazy segtree . . . 15
- 3.13 Max add segtree . . . 16
- 3.14 Segtree beats dinâmica . . . 16
- 3.15 Segtree com rollback: chmax e mínimo . . . 17
- 3.16 Deletar offline . . . 18
- 3.17 Consultas de Mo . . . 18
- 3.18 Wavelet matrix . . . 19
- 3.19 Wavelet matrix: posições e retângulos . . . 19
- 3.20 Trie de XOR . . . 19
- 3.21 Árvore cartesiana . . . 20
- 3.22 Treap implícita . . . 20
- 3.23 Árvore implícita persistente (RBST) . . . 21
- 3.24 Extensões GNU . . . 22
- 3.25 Máximo de sufixo . . . 23
- 3.26 CHT estático e pontos de troca . . . 23
- 3.27 Árvore de Li Chao . . . 23
- 3.28 Simplex . . . 24
- 3.29 Convolução . . . 25

3.29.1 FFT . . . 25
3.29.2 NTT . . . 25
3.30 Convolução de XOR . . . 26
3.31 Zeta e Mobius . . . 26
3.32 Convolução de subconjuntos . . . 26

4 Grafos **27**
4.1 0-1 BFS . . . 27
4.2 Dijkstra . . . 27
4.3 Ciclo de peso médio mínimo . . . 27
4.4 Caminho Euleriano . . . 28
4.5 Componentes do grafo complementar . . . 28
4.6 Árvore de pontes . . . 28
4.7 Árvore de bloco e corte . . . 29
4.8 Numeração st . . . 29
4.9 Componentes fortemente conexas e grafo de condensação . . . 29
4.10 2-SAT . . . 30
4.11 Floresta de grafo funcional . . . 30
4.12 Dominator tree . . . 30
4.13 Árvore geradora mínima direcionada . . . 31
4.14 Matching bipartido máximo . . . 32
4.15 Matching bipartido com menor custo . . . 32
4.16 Encontrar matching geral . . . 33
4.17 Matching geral . . . 33
4.18 Fluxo máximo (Dinic) . . . 33
4.19 Árvore de Gomory–Hu . . . 34
4.20 Fluxo de custo mínimo . . . 34
4.21 Grafo com sequência de graus . . . 35
4.22 Enumeração de triângulos . . . 36
4.23 Coloração de arestas . . . 36

5 Árvores **36**
5.1 Diâmetro de uma árvore . . . 36
5.2 Matching em árvores . . . 36
5.3 Isomorfismo de subárvores . . . 37
5.4 LCA . . . 37
5.5 HLD . . . 37
5.6 Árvore virtual . . . 38
5.7 Árvore de reconstrução de Kruskal . . . 38
5.8 Decomposição em árvore de centróides . . . 38
5.9 Histograma de distâncias em árvore . . . 39
5.10 Link-cut tree . . . 39

6 Strings **40**
6.1 Hashing . . . 40
6.2 KMP . . . 40
6.3 Z-function . . . 41
6.4 Aho-Corasick . . . 41
6.5 Array de sufixos e LCP . . . 41
6.6 Árvore de sufixos . . . 42
6.7 Autômato de sufixos . . . 42
6.8 Maior substring comum . . . 43
6.9 Manacher . . . 43
6.10 Árvore de palíndromos (eertree) . . . 43
6.11 Decomposição de Lyndon . . . 44
6.12 Encontrar runs em uma string . . . 44

7 Geometria **45**
7.1 Base . . . 45
7.2 Interseção de retas . . . 45
7.3 Interseção de segmentos . . . 45
7.4 Área orientada . . . 45
7.5 Convex hull . . . 45
7.6 Ponto em convexo . . . 46
7.7 Corte de polígono . . . 46
7.8 Interseção de semiplanos . . . 46
7.9 Soma de Minkowski . . . 46
7.10 Círculo–reta . . . 47
7.11 Círculo–círculo . . . 47
7.12 Tangentes comuns . . . 47
7.13 Círculo mínimo . . . 47
7.14 Área círculo–polígono . . . 48
7.15 Par mais próximo . . . 48
7.16 MST Manhattan . . . 48

7.17 Faces de grafo planar . . . 49
7.18 Triangulação de Delaunay . . . 49
7.19 Convex hull 3D . . . 50

8 Subproblemas **51**
8.1 Atribuição em intervalos constantes . . . 51
8.2 Somas dinâmicas em retângulos . . . 51
8.3 Área da união de retângulos . . . 52
8.4 Mochila com quantidades limitadas . . . 52
8.5 DP para todas as raízes . . . 53
8.6 Restrições de diferenças e paridade . . . 53
8.7 Circulação com limites inferiores . . . 54

A Fórmulas, Teoremas e Fatos **54**
A.1 Teoria dos números . . . 54
A.2 Contagem . . . 56
A.3 Contagem: simetrias e restrições . . . 57
A.4 Somas e sequências . . . 58
A.5 Tabela de funções geradoras . . . 59
A.6 Identidades para convolução . . . 60
A.7 Atualização de matriz por posto 1 . . . 60
A.8 Combinatória e otimização . . . 60
A.9 DP . . . 61
A.10 Grafos e jogos . . . 61
A.11 Strings . . . 63
A.12 Geometria . . . 64

B Debugging **64**

Contest

Os resultados e intermediários devem caber nos tipos usados.

template.cpp

```cpp
#include <bits/stdc++.h>

using namespace std;

using ll = long long;
using ld = long double;
using pii = pair<int, int>;
using pll = pair<ll, ll>;
using vi = vector<int>;

#define pb push_back
#define eb emplace_back
#define fi first
#define se second
#define all(x) begin(x), end(x)
#define sz(x) (int)(x).size()
#define rep(i,a,b) for (int i = (a); i < (b); i++)

mt19937 rng(random_device{}());

int main() {
    cin.tie(0)->sync_with_stdio(0);
}
```

.vimrc. `Hash`: hash do texto selecionado.

```
set cin ts=4 sw=4 udf cul nu is

ca Hash w !cpp -dD -P -fpreprocessed \| tr -d '[:space:]' \| md5sum \| cut -c -6
```

.bashrc. `c`: compila; `cs`: com sanitizers; `gen`: gera testes; `rr`: executa; `chk`: compara saídas.

```
c() {
    g++ $1.cpp -o $1 -std=c++20 \
        -Wall -Wextra -Wshadow -Wconversion -Wpedantic
}
cs() {
    g++ $1.cpp -o $1 -std=c++20 \
        -Wall -Wextra -Wshadow -Wconversion -Wpedantic \
        -fsanitize=address,undefined \
        -fsanitize=signed-integer-overflow -ggdb
}
gen() {
    local N=${1:-10}
    for i in $(seq 1 $N); do
        python3 gen.py $i > $2$i.in
    done
}
rr() {
    local ext=${2:-out}
    for i in $(ls -v *.in); do
        local f=$(basename $i .in)
        ./$1 < $f.in > $f.$ext
    done
}
chk() {
    local a=${1:-ans}
    local b=${2:-out}
    for i in $(ls -v *.$a); do
        local f=$(basename $i .$a)
        echo -n "$f.$a - $f.$b > "
        if cmp -s $f.$a $f.$b; then
            echo OK
        else
            echo X
        fi
    done
}
```

Template de stress testing. `rr(l,r)` sorteia em $[l, r)$.

```python
from random import randrange as rr, shuffle as sh, sample as sm, choice as ch, seed
from string import ascii_lowercase as alpha
from itertools import combinations as combs, permutations as perms, product as prod
from sys import argv

if len(argv) > 1: seed(argv[1])
```

Stress testing reproduzível

Prepare `sol.cpp`, uma solução lenta `brute.cpp` e um gerador `gen.py` que use a seed recebida. Carregue os helpers com `source .bashrc`; `cs sol && c brute` compila

com sanitizers na candidata. Rode o loop abaixo no Bash; ajuste os 3 segundos ao tamanho dos casos e ao custo dos sanitizers.

```
seed=1
while true; do
    echo "$seed" > fail.seed
    python3 gen.py "$seed" > fail.in || break
    timeout 3s ./sol < fail.in > sol.out || break
    timeout 3s ./brute < fail.in > brute.out || break
    cmp -s sol.out brute.out || break
    seed=$((seed + 1))
done
echo "Reproduzir: python3 gen.py $(cat fail.seed)"
```

Compare saídas com `diff -u brute.out sol.out`. `cmp` exige igualdade byte a byte: para respostas não únicas, valide propriedades; para reais, use a tolerância do problema.

Matemática

Algoritmo euclideano. `g=egcd(a,b,x,y)`, $ax + by = g = \gcd(a, b)$. $O(\log \min(a, b))$.

b4f64d - math/egcd.cpp - 9 lines

```
ll egcd(ll a, ll b, ll &x, ll &y) {
    if (b == 0) {
        x = 1, y = 0;
        return a;
    }
    ll g = egcd(b, a % b, y, x);
    y -= (a / b) * x;
    return g;
} // b4f64d
```

Potência módulo `mod`. `powm(x,e)`: x^e, $e \geq 0$. $O(\log e)$.

44bf2f - math/powm.cpp - 10 lines

```
const ll mod = 998244353; // Change this
ll powm(ll x, ll e) {
    ll r = 1;
    while (e) {
        if (e & 1) (r *= x) %= mod;
        (x *= x) %= mod;
        e >>= 1;
    }
    return r;
} // e2de55
```

CRT. `crt(r1,m1,r2,m2)`: `{r,m}`, $0 \leq r < m$, ou `{0,-1}` se incompatível. Módulos positivos, resíduos normalizados. $O(\log \min(m_1, m_2))$.

6e0e1c - math/crt.cpp - 9 lines

```
pair<ll, ll> crt(ll r1, ll m1, ll r2, ll m2) {
    ll x, y, g = egcd(m1, m2, x, y);
    if ((r2 - r1) % g) return {0, -1};
    ll k = m2 / g, m = m1 / g * m2;
    ll q = ((r2 - r1) / g % k + k) % k;
    x = ((x % k) + k) % k;
    ll r = r1 + (__int128)m1 * (q * x % k);
    return {(r % m + m) % m, m};
} // 6e0e1c
```

Crivo. `Sieve(n)`: `primes` contém primos menores que n; `spf[x]` é o menor fator primo de x. $O(n)$.

eb3eff - math/sieve.cpp - 16 lines

```
struct Sieve {
    vi primes, spf;
    Sieve(int n): spf(n) {
        rep (i, 2, n) {
            if (!spf[i]) {
                spf[i] = i;
                primes.push_back(i);
            }
            for (int j: primes) {
                if (j * i >= n) break;
                spf[i * j] = j;
                if (j == spf[i]) break;
            }
        }
    }
};
```

Teste de primalidade. `isPrime(n)`: inteiros de 64 bits; $O(\log n)$.

cabafb - math/primality.cpp - 25 lines

```
using i128 = __int128;

bool checkPrime(ll p, ll a) {
```

```
    ll k = p - 1, d = 1;
    while (~k & 1) k >>= 1;
    for (ll e = k; e; e >>= 1) {
        if (e & 1) d = i128(d) * a % p;
        a = i128(a) * a % p;
    }
    if (d == 1 || d == p - 1) return true;
    while ((k <<= 1) < p - 1) {
        d = i128(d) * d % p;
        if (d == p - 1) return true;
    }
    return false;
} // d5cf5e

bool isPrime(ll p) {
    if (p < 2) return false;
    for (int i: {2, 3, 5, 7, 11, 13, 17, 19, 23, 29, 31, 37}) {
        if (p == i) return true;
        if (!checkPrime(p, i)) return false;
    }
    return true;
} // 2c12c6
```

Fatoração de inteiros. `factor(n,fs)` acrescenta fatores primos com multiplicidade, em ordem arbitrária. Esperado $O(\sqrt[4]{n})$.

7b1297 - math/factor.cpp - 28 lines

```
mt19937_64 rng64(random_device{}());

using ull = unsigned long long;
using u128 = __uint128_t;

ull modMul(ull a, ull b, ull mod) { return (u128)a * b % mod; }

ull rho(ull n) {
    if (n % 2 == 0) return 2;
    if (n % 3 == 0) return 3;
    ull c = uniform_int_distribution<ull>(1, n - 1)(rng64);
    ull x = uniform_int_distribution<ull>(0, n - 1)(rng64), y = x, d = 1;
    auto f = [&](ull x) { return (modMul(x, x, n) + c) % n; };
    while (d == 1) {
        x = f(x);
        y = f(f(y));
        d = gcd(x > y ? x - y : y - x, n);
    }
    return d == n ? rho(n) : d;
} // 8e1254

void factor(ull n, vector<ull> &fs) {
    if (n == 1) return;
    if (isPrime(n)) { fs.pb(n); return; }
    ull d = rho(n);
    factor(d, fs);
    factor(n / d, fs);
} // 6d58e9
```

Raiz quadrada módulo `mod`. `sqrtMod(a)`: menor raiz não negativa, ou -1. Requer `mod` primo e $0 \le a < \texttt{mod}$. $O(\log^2 \texttt{mod})$.

b04743 - math/mod-sqrt.cpp - 19 lines

```
int sqrtMod(int a) { // mod prime
    if (a == 0 || mod == 2) return a;
    if (powm(a, (mod - 1) / 2) != 1) return -1;
    if (mod % 4 == 3) return powm(a, (mod + 1) / 4);
    int s = __builtin_ctz(mod - 1), q = (mod - 1) >> s, z = 2;
    while (powm(z, (mod - 1) / 2) != mod - 1) z++;
    ll c = powm(z, q), x = powm(a, (q + 1) / 2), t = powm(a, q);
    for (int m = s; t != 1; ) {
        ll tt = t;
        int i = 0;
        while (tt != 1) tt = tt * tt % mod, i++;
        ll b = powm(c, 1 << (m - i - 1));
        x = x * b % mod;
        c = b * b % mod;
        t = t * c % mod;
        m = i;
    }
    return min<ll>(x, mod - x);
} // b04743
```

Logaritmo discreto. `discreteLog(a,b,m)`: menor $x \ge 0$ com $a^x \equiv b \pmod m$, ou -1. Aceita $a, b \ge 0$, $m \ge 1$, inclusive $\gcd(a, m) > 1$; considera $a^0 = 1$. Tempo esperado e memória $O(\sqrt{m})$.

c7ee5d - math/discrete-log.cpp - 24 lines

```
ll discreteLog(ll a, ll b, ll m) {
    a %= m; b %= m;
    auto mul = [](ll x,ll y,ll m) { return (ll)((__int128)x*y % m); };
```

```
    ll k = 1 % m, off = 0;
    for (ll g; (g = gcd(a,m)) > 1; ++off) {
        if (b == k) return off;
        if (b % g) return -1;
        b /= g; m /= g;
        k = mul(k,a/g,m);
    }
    if (b == k) return off;
    ll n = sqrtl(m) + 1, an = 1, cur = b;
    unordered_map<ll,ll> baby;
    for (ll j = 0; j < n; ++j) {
        baby[cur] = j;
        cur = mul(cur,a,m), an = mul(an,a,m);
    }
    cur = k;
    for (ll i = 1; i <= n; ++i) {
        cur = mul(cur,an,m);
        if (baby.count(cur)) return (ll)((__int128)i*n - baby[cur] + off);
    }
    return -1;
} // c7ee5d
```

Soma de floor. `floorSums(n,m,a,b)` retorna $\{\sum f_i, \sum i f_i, \sum f_i^2\}$ em `i128`, $f_i = \lfloor (ai+b)/m \rfloor$, $0 \le i < n$. `floorSum` calcula só a primeira. Requer $n, a, b \ge 0$, $m > 0$. Custo logarítmico.

9744fc - math/floor-sum.cpp - 24 lines

```
using i128 = __int128_t;
array<i128,3> floorSums(ll n, ll m, ll a, ll b) {
    if (!n) return {0,0,0};
    i128 s1 = (i128)n*(n-1)/2, s2 = (i128)n*(n-1)*(2*(i128)n-1)/6;
    ll qa = a/m, qb = b/m;
    if (qa || qb) {
        auto [x,y,z] = floorSums(n,m,a%m,b%m);
        return {x + qa*s1 + (i128)qb*n, y + qa*s2 + qb*s1,
            z + 2*qa*y + 2*qb*x + (i128)qa*qa*s2
            + 2*(i128)qa*qb*s1 + (i128)qb*qb*n};
    }
    ll k = ((i128)a*(n-1)+b)/m;
    if (!k) return {0,0,0};
    auto [x,y,z] = floorSums(k,a,m,m-b-1);
    return {(i128)k*(n-1)-x, k*s1-(z+x)/2, (i128)k*k*(n-1)-2*y-x};
} // 164b59
i128 floorSum(ll n, ll m, ll a, ll b) {
    if (!n) return 0;
    if (a >= m || b >= m)
        return (i128)(a/m)*n*(n-1)/2 + (i128)(b/m)*n
            + floorSum(n,m,a%m,b%m);
    ll k = ((i128)a*(n-1)+b)/m;
    return k ? (i128)k*(n-1)-floorSum(k,a,m,m-b-1) : 0;
} // 3d0695
```

Mínimo de progressão modular. `minMod(n,m,a,b)`: `{valor,i}`, menor resíduo $(ai+b) \bmod m$ em $[0,n)$ e primeiro índice que o atinge. Requer $n, m > 0$, $a, b \ge 0$. $O(\log(m+1))$.

7beb41 - math/min-mod.cpp - 27 lines

```
pair<ll,ll> minMod(ll n, ll m, ll a, ll b) {
    a %= m; b %= m;
    ll A = a, M = m, B = b;
    ll ans = m;
    while (true) {
        if (a > m/2) {
            b = ((__int128)a*(n-1)+b)%m;
            a = m-a;
        }
        ans = min(ans,b);
        ll k = ((__int128)a*(n-1)+b)/m;
        if (!k) break;
        n = k; b = (b-m)%a;
        if (b < 0) b += a;
        ll r = m%a; m = a; a = (m-r)%m;
    }
    ll r = M, s = A, x = 0, y = 1;
    while (s) {
        ll q = r/s, t = r%s;
        r = s; s = t;
        t = x - (__int128)q*y; x = y; y = t;
    }
    ll period = M/r;
    ll i = (__int128)x*((ans-B)/r)%period;
    if (i < 0) i += period;
    return {ans,i};
} // 7beb41
```

Somas de totiente e Möbius. `Summatory(B)`, $B \ge 1$; `sum(n)` retorna $\{\sum_{i=1}^{n} \varphi(i), \sum_{i=1}^{n} \mu(i)\}$ em `i128`, $n \ge 0$. Para uma consulta com $\sqrt{n} \le B \le n$:

tempo esperado $O(B + n/\sqrt{B})$, memória $O(B + n/B)$; $B \approx n^{2/3}$ equilibra o custo.

621960 - math/summatory-functions.cpp - 32 lines

```cpp
struct Summatory {
    vector<ll> phi, mu;
    unordered_map<ll,array<i128,2>> memo;
    Summatory(int b) : phi(b+1), mu(b+1) {
        vector<int> primes, spf(b+1);
        phi[1] = mu[1] = 1;
        for (int i = 2; i <= b; ++i) {
            if (!spf[i]) spf[i] = i, primes.push_back(i),
                phi[i] = i-1, mu[i] = -1;
            for (int p : primes) {
                if (p > b/i) break;
                spf[i*p] = p;
                phi[i*p] = phi[i]*(p-(i%p != 0));
                if (i%p == 0) break;
                mu[i*p] = -mu[i];
            }
        }
        for (int i = 1; i <= b; ++i)
            phi[i] += phi[i-1], mu[i] += mu[i-1];
    }
    array<i128,2> sum(ll n) {
        if (n < (ll)phi.size()) return {phi[n],mu[n]};
        if (memo.count(n)) return memo[n];
        array<i128,2> ans = {i128(n)*(i128(n)+1)/2,1};
        for (ll l = 2, r; l <= n; l = r+1) {
            r = n/(n/l); auto s = sum(n/l);
            for (int j = 0; j < 2; ++j) ans[j] -= (r-l+1)*s[j];
            if (r == n) break;
        }
        return memo[n] = ans;
    } // 14930d
};
```

Permutação para inteiro. `perm2int(p)` / `int2perm(n,k)`: rank lexicográfico 0-based de permutações de $[0, n)$. Use $n \leq 20$, $0 \leq k < n!$. $O(n^2)$.

df96dc - math/perm-int.cpp - 20 lines

```cpp
ll perm2int(vi p) {
    vi a(sz(p)); iota(all(a), 0); ll r = 0;
    for (int x: p) {
        int i = find(all(a), x) - a.begin();
        r = r * sz(a) + i;
        a.erase(a.begin() + i);
    }
    return r;
} // 01ca9e

vi int2perm(int n, ll k) {
    vi a(n), p; iota(all(a), 0); ll f = 1;
    rep(i,1,n) f *= i;
    rep(i,0,n) {
        int j = k / f; k %= f;
        p.pb(a[j]); a.erase(a.begin() + j);
        if (n - i - 1) f /= n - i - 1;
    }
    return p;
} // edd4ac
```

Matrizes sobre semianéis. `matMul`: matrizes retangulares compatíveis não vazias; $O(nkm)$ operações escalares. `matPow`: matriz quadrada não vazia, $e \geq 0$; $O(n^3 \log(e+1))$. `add,mul,zero,one` formam um semianel, como booleanos ou min-plus.

66c23b - math/matrix.cpp - 23 lines

```cpp
template<class T> using Matrix = vector<vector<T>>;
template<class T, class Add, class Mul>
Matrix<T> matMul(const Matrix<T> &a, const Matrix<T> &b,
    T zero, Add add, Mul mul) {
    int n = sz(a), k = sz(b), m = sz(b[0]);
    Matrix<T> c(n,vector<T>(m,zero));
    rep(i,0,n) rep(j,0,k) rep(t,0,m)
        c[i][t] = add(c[i][t],mul(a[i][j],b[j][t]));
    return c;
} // d12123
template<class T, class Add, class Mul>
Matrix<T> matPow(Matrix<T> a, uint64_t e,
    T zero, T one, Add add, Mul mul) {
    int n = sz(a);
    Matrix<T> r(n,vector<T>(n,zero));
    rep(i,0,n) r[i][i] = one;
    while (e) {
        if (e&1) r = matMul(r,a,zero,add,mul);
        e >>= 1;
        if (e) a = matMul(a,a,zero,add,mul);
    }
    return r;
```

```
} // 014690
```

Eliminação gaussiana. `gauss(a,det)`: escalona **in-place**, retorna posto e escreve determinante módulo `mod` (0 se retangular; 1 para 0×0). Módulo primo, entradas normalizadas; $O(nm\min(n,m) + \min(n,m)\log \texttt{mod})$. `linearSystem(a,m,x,basis)` recebe matriz aumentada $n \times (m+1)$, sem modificá-la; retorna falso se inconsistente. Caso contrário, soluções $x + \sum c_i \, \texttt{basis}[i]$. Aceita $n = 0$ e $m = 0$; saída ocupa $O(m^2)$.

5dcc86 - math/gaussian-elimination.cpp - 44 lines

```
int gauss(vector<vector<ll>> &a, ll &det) {
    int n = sz(a), m = n ? sz(a[0]) : 0, r = 0;
    det = 1;
    rep(c,0,m) {
        int p = r;
        while (p < n && !a[p][c]) p++;
        if (p == n) continue;
        if (p != r) det = (mod - det) % mod;
        swap(a[p], a[r]);
        ll pivot = (a[r][c] + mod) % mod;
        det = det * pivot % mod;
        ll iv = powm(pivot, mod - 2);
        rep(j,c,m) a[r][j] = a[r][j] * iv % mod;
        rep(i,0,n) if (i != r && a[i][c]) {
            ll x = a[i][c];
            rep(j,c,m) a[i][j] = (a[i][j] - x * a[r][j]) % mod;
        }
        r++;
    }
    rep(i,0,n) rep(j,0,m) if (a[i][j] < 0) a[i][j] += mod;
    if (n != m || r < n) det = 0;
    return r;
} // 14ff56
bool linearSystem(vector<vector<ll>> a, int m,
        vector<ll> &x, vector<vector<ll>> &basis) {
    ll det;
    gauss(a,det);
    vi row(m,-1);
    rep(i,0,sz(a)) {
        int j = 0;
        while (j <= m && !a[i][j]) j++;
        if (j == m) return false;
        if (j < m) row[j] = i;
    }
    x.assign(m,0); basis.clear();
    rep(j,0,m) if (row[j] >= 0) x[j] = a[row[j]][m];
    rep(f,0,m) if (row[f] < 0) {
        vector<ll> v(m); v[f] = 1;
        rep(j,0,m) if (row[j] >= 0)
            v[j] = (mod-a[row[j]][f])%mod;
        basis.pb(v);
    }
    return true;
} // dbc0b6
```

Base de XOR. `XorBasis<B=63>`: `add(x)` retorna 1 se x é XOR de entradas anteriores; senão, insere e retorna 0. `query(x=0,hi=0)` minimiza $x \oplus$ XOR de um subconjunto; `hi` $= 1$ maximiza. `w=reduce(x)` minimiza x in-place: original $=$ resíduo $x \oplus$ XOR das entradas selecionadas por `w`. Bit j seleciona a j-ésima entrada independente em ordem de inserção (0-based). Guarde índices originais externamente quando `add` retornar 0. Para reconstruir uma dependência, reduza uma cópia antes de inserir o original. Acrescentar bit 1 às entradas força dependências de cardinalidade par. $O(B\lceil B/64 \rceil)$ por operação; memória $O(B + B\lceil B/64 \rceil)$ palavras.

992a74 - math/xor-basis.cpp - 22 lines

```
template<int B = 63>
struct XorBasis {
    using T = bitset<B>;
    struct Row { int i; T x,w; };
    vector<Row> b;
    T reduce(T &x, bool hi = 0) const {
        T w;
        for (auto &r : b) if (x[r.i] != hi) x ^= r.x, w ^= r.w;
        return w;
    } // 38829e
    bool add(T x) {
        auto w = reduce(x);
        for (int i = B-1; i >= 0; --i) if (x[i]) {
            w[sz(b)] = 1;
            int j = 0;
            while (j < sz(b) && b[j].i > i) ++j;
            b.insert(b.begin()+j,{i,x,w}); return 0;
        }
        return 1;
    } // a3661d
    T query(T x = {}, bool hi = 0) const { reduce(x,hi); return x; }
```

```
};
```

Interpolação de Lagrange. `lagrange(xs,ys,x)` avalia o interpolador: $O(n^2 + n \log \texttt{mod})$. `lagrange(ys,x)` usa abscissas $0, \ldots, n-1$: $O(n + \log \texttt{mod})$. Requer módulo primo, resíduos normalizados e abscissas distintas módulo `mod`; na segunda, $1 \leq n \leq \texttt{mod}$.

29e498 - math/lagrangian-interpolation.cpp - 26 lines

```
ll lagrange(const vector<ll> &xs, const vector<ll> &ys, ll x) {
    int n = sz(xs); ll ans = 0;
    rep(i,0,n) if (xs[i] == x) return ys[i];
    rep(i,0,n) {
        ll a = ys[i], b = 1;
        rep(j,0,n) if (i != j) a = a * (x - xs[j] + mod) % mod, b = b * (xs[i] -
        ↪  xs[j] + mod) % mod;
        ans = (ans + a * powm(b, mod - 2)) % mod;
    }
    return ans;
} // 51ae24

ll lagrange(const vector<ll> &y, ll x) {
    int n = sz(y);
    x = (x % mod + mod) % mod;
    if (x < n) return y[x];
    vector<ll> p(n + 1, 1), s(n + 1, 1), a(n);
    rep(i,0,n) p[i + 1] = p[i] * (x - i + mod) % mod;
    for (int i = n - 1; i >= 0; --i) s[i] = s[i + 1] * (x - i + mod) % mod;
    a[0] = 1;
    rep(i,1,n) a[i] = a[i - 1] * i % mod;
    a[n - 1] = powm(a[n - 1], mod - 2);
    for (int i = n - 1; i; --i) a[i - 1] = a[i] * i % mod;
    ll ans = 0;
    rep(i,0,n) ans = (ans + p[i] * s[i + 1] % mod * a[i] % mod * a[n - 1 - i] % mod *
    ↪  (((n - 1 - i) & 1) ? mod - y[i] : y[i])) % mod;
    return ans;
} // 05868c
```

Polinômio de Lagrange. `lagrangePoly(xs,ys)`: coeficientes por grau crescente. Requer $n \geq 1$, módulo primo, resíduos normalizados e abscissas distintas módulo `mod`. $O(n^2 + n \log \texttt{mod})$.

7d0710 - math/lagrangian-polynomial.cpp - 19 lines

```
vector<ll> lagrangePoly(const vector<ll> &xs, const vector<ll> &ys) {
    int n = sz(xs);
    vector<ll> p = {1}, f(n);
    rep(i,0,n) {
        p.pb(0);
        for (int j = i + 1; j; --j) p[j] = (p[j - 1] + mod - p[j] * xs[i] % mod) %
        ↪  mod;
        p[0] = p[0] * (mod - xs[i]) % mod;
    }
    rep(i,0,n) {
        vector<ll> q(n);
        q[n - 1] = p[n];
        for (int j = n - 1; j; --j) q[j - 1] = (p[j] + q[j] * xs[i]) % mod;
        ll den = 1;
        rep(j,0,n) if (i != j) den = den * (xs[i] - xs[j] + mod) % mod;
        ll c = ys[i] * powm(den, mod - 2) % mod;
        rep(j,0,n) f[j] = (f[j] + c * q[j]) % mod;
    }
    return f;
} // 7d0710
```

Séries formais: inversa, logaritmo e exponencial. Retornam os primeiros n coeficientes, $1 \leq n < \texttt{mod}$; termos ausentes valem zero. `fpsInv`: $A(0) \neq 0$; `fpsLog`: $A(0) = 1$; `fpsExp`: $A(0) = 0$. Entradas normalizadas e não vazias; as convoluções intermediárias devem caber no limite da transformada. $O(n \log n)$.

7fcc4f - math/power-series.cpp - 34 lines

```
vector<ll> fpsInv(vector<ll> a, int n) {
    vector<ll> b = {powm(a[0],mod-2)};
    while (sz(b) < n) {
        int m = min(n,2*sz(b));
        vector<ll> cut(a.begin(),a.begin()+min(m,sz(a)));
        auto c = conv(cut,b); c.resize(m);
        for (ll &x : c) x = (mod-x) % mod;
        c[0] = (c[0]+2) % mod;
        b = conv(b,c); b.resize(m);
    }
    b.resize(n); return b;
} // f81aba
vector<ll> fpsLog(vector<ll> a, int n) {
    a.resize(n);
    vector<ll> d(max(0,n-1)), inv(n,1);
    rep(i,1,n) d[i-1] = a[i]*i % mod;
    auto b = fpsInv(a,n), c = conv(d,b);
    c.resize(n);
    for (int i = 2; i < n; ++i) inv[i] = mod - (mod/i)*inv[mod%i] % mod;
```

```
    for (int i = n-1; i > 0; --i) c[i] = c[i-1]*inv[i] % mod;
    c[0] = 0; return c;
} // fb2594
vector<ll> fpsExp(vector<ll> a, int n) {
    a.resize(n);
    vector<ll> b = {1};
    while (sz(b) < n) {
        int m = min(n,2*sz(b));
        auto c = fpsLog(b,m);
        rep(i,0,m) c[i] = (a[i]-c[i]+mod) % mod;
        c[0] = (c[0]+1) % mod;
        b = conv(b,c); b.resize(m);
    }
    return b;
} // 810d41
```

Berlekamp-Massey. `berlekampMassey(s)`: coeficientes c da menor recorrência compatível, $s_i = \sum_{j=0}^{m-1} c_j s_{i-j-1}$. Módulo primo, termos normalizados; sequência toda zero retorna vazio. Para recuperar ordem m, forneça ao menos $2m$ termos. $O(n^2 + n \log \texttt{mod})$.

68a616 - math/berlekamp-massey.cpp - 19 lines

```
vi berlekampMassey(vi s) {
    vi c(1, 1), b(1, 1);
    int l = 0, m = 1; ll bb = 1;
    rep (n, 0, sz(s)) {
        ll d = 0;
        rep (i, 0, l + 1) d = (d + 1LL * c[i] * s[n - i]) % mod;
        if (d < 0) d += mod;
        if (!d) { m++; continue; }
        vi t = c;
        ll x = d * powm(bb, mod - 2) % mod;
        if (sz(c) < sz(b) + m) c.resize(sz(b) + m);
        rep (i, 0, sz(b)) c[i + m] = (c[i + m] - x * b[i]) % mod;
        if (2 * l <= n) l = n + 1 - l, b = t, bb = d, m = 1;
        else m++;
    }
    c.erase(begin(c));
    for (int &x : c) x = (mod - x) % mod;
    return c;
} // 68a616
```

Recorrência linear. `linRec(s,tr,k)`: termo $k \geq 0$, 0-based; `s` contém os primeiros $m = |\texttt{tr}| > 0$ termos e $s_i = \sum_{j=0}^{m-1} \texttt{tr}_j s_{i-j-1}$. $O(m^2 \log(k+1))$.

5f7ef4 - math/linear-recurrence.cpp - 23 lines

```
int linRec(vi s, vi tr, ll k) { // s = first m terms, 0-indexed kth term
    int n = sz(tr);
    if (k < n) return s[k];
    auto mul = [&](vi a, vi b) {
        vi c(2 * n + 1);
        rep (i, 0, n + 1) rep (j, 0, n + 1)
            c[i + j] = (c[i + j] + 1LL * a[i] * b[j]) % mod;
        for (int i = 2 * n; i > n; --i) rep (j, 0, n)
            c[i - 1 - j] = (c[i - 1 - j] + 1LL * c[i] * tr[j]) % mod;
        c.resize(n + 1);
        return c;
    }; // 610f7a
    vi pol(n + 1), e(n + 1);
    pol[0] = 1;
    e[1] = 1;
    for (++k; k; k >>= 1) {
        if (k & 1) pol = mul(pol, e);
        e = mul(e, e);
    }
    ll ans = 0;
    rep (i, 0, n) ans = (ans + 1LL * pol[i + 1] * s[i]) % mod;
    return ans;
} // 5f7ef4
```

Bostan–Mori. `bostanMori(p,q,k)`: $[x^k]P/Q$, $k \geq 0$, $Q(0) \neq 0$, $\deg P < \deg Q$. `fastRec(s,c,k)` usa $d = |c| > 0$ termos iniciais e $s_i = \sum_{j=0}^{d-1} c_j s_{i-j-1}$. Coeficientes normalizados; $O(d \log d \log(k+1))$ mais a convolução inicial.

8f1b17 - math/bostan-mori.cpp - 20 lines

```
ll bostanMori(vector<ll> p, vector<ll> q, ll k) {
    while (k) {
        auto neg = q;
        for (int i = 1; i < sz(q); i += 2) neg[i] = (mod - neg[i]) % mod;
        auto a = conv(p,neg), b = conv(q,neg);
        p.clear(), q.clear();
        for (int i = k & 1; i < sz(a); i += 2) p.pb(a[i]);
        for (int i = 0; i < sz(b); i += 2) q.pb(b[i]);
        k >>= 1;
    }
    return p.empty() ? 0 : p[0] * powm(q[0],mod-2) % mod;
```

```
} // a87071
ll fastRec(vector<ll> s, vector<ll> c, ll k) {
    int d = sz(c);
    vector<ll> q(d+1,1);
    rep(i,0,d) q[i+1] = (mod-c[i]) % mod;
    auto p = conv(s,q);
    p.resize(d);
    return bostanMori(p,q,k);
} // 37d63c
```

Integração adaptativa (Simpson). `integrate`: `{valor,ok}` para $\int_a^b f(x)\,dx$. `eps>0`: tolerância absoluta; `depth>=0`, `budget>=3`: limites de profundidade e avaliações. `ok=false` se atingir um limite antes da tolerância estimada. Requer valores finitos de f, inclusive nos extremos, e aritmética finita no intervalo. O estimador é heurístico: pode ignorar picos estreitos ou oscilações; separe descontinuidades conhecidas. Tempo proporcional às avaliações; pilha $O(\texttt{depth})$.

810958 - math/integrate.cpp - 28 lines

```
template<class F>
pair<ld,bool> integrate(F f, ld a, ld b, ld eps = 1e-9L,
        int depth = 25, int budget = 100000) {
    assert(eps > 0 && depth >= 0 && budget >= 3);
    if (a == b) return {0,true};
    ld sign = 1; if (a > b) swap(a,b), sign = -1;
    auto simpson = [](ld a, ld b, ld fa, ld fm, ld fb) {
        return (b-a)*(fa+4*fm+fb)/6;
    }; // 24e9b6
    bool ok = true;
    auto rec = [&](auto&& self, ld a, ld b, ld fa, ld fm, ld fb,
            ld s, ld tol, int dep) -> ld {
        ld m = a+(b-a)/2, l = a+(m-a)/2, r = m+(b-m)/2;
        if (!dep || budget < 2 || l == a || r == b) {
            ok = false; return s;
        }
        budget -= 2;
        ld fl = f(l), fr = f(r);
        ld sl = simpson(a,m,fa,fl,fm), sr = simpson(m,b,fm,fr,fb);
        ld delta = sl+sr-s;
        if (abs(delta) <= 15*tol) return sl+sr+delta/15;
        ld left = self(self,a,m,fa,fl,fm,sl,tol/2,dep-1);
        return left+self(self,m,b,fm,fr,fb,sr,tol/2,dep-1);
    }; // 001bc0
    ld fa = f(a), fm = f(a+(b-a)/2), fb = f(b); budget -= 3;
    ld ans = rec(rec,a,b,fa,fm,fb,simpson(a,b,fa,fm,fb),eps,depth);
    return {sign*ans,ok};
} // 368475
```

Estruturas de Dados

Busca binária. `binSearch(lo,hi,cond)`: predicado monótono falso→verdadeiro, inicialmente `!cond(lo)` e `cond(hi)`. Atualiza os limites para `hi=lo+1`. Requer `hi-lo` representável em `ll`. $O(\log(\texttt{hi}-\texttt{lo}))$ avaliações.

889dcd - dsa/binary-search.cpp - 6 lines

```
void binSearch(ll &lo, ll &hi, auto cond) {
    while (hi - lo > 1) {
        ll mid = lo + (hi - lo) / 2;
        if (cond(mid)) hi = mid; else lo = mid;
    }
} // 889dcd
```

Busca ternária. `ternarySearch(l,r,f)`: índice de máximo em $[l,r]$ inteiro. Requer função crescente até o máximo e decrescente depois (platô apenas no máximo). $O(\log(r-l+1))$ avaliações.

c23378 - dsa/ternary-search.cpp - 10 lines

```
int ternarySearch(ll l, ll r, auto f) {
    while (r - l > 2) {
        ll m1 = l + (r - l) / 3, m2 = l + (r - l) / 3 * 2;
        if (f(m1) < f(m2)) l = m1;
        else r = m2;
    }
    int mx = l;
    rep(i, l + 1, r + 1) if (f(i) > f(mx)) mx = i;
    return mx;
} // c23378
```

Maior subsequência crescente. `LIS(a)`: `{id,par}`, índices de uma LIS estrita e predecessores. `par[i]` permite reconstruir uma LIS terminando em i; -1 no início. Índices 0-based; $O(n \log n)$.

990943 - dsa/lis.cpp - 18 lines

```
pair<vi, vi> LIS(vi &a) {
    int n = sz(a);
    vi st, id, p(n);
    rep(i, 0, n) {
```

```
        int j = lower_bound(all(st), a[i]) - begin(st);
        if (j < sz(st)) {
            st[j] = a[i];
            p[i] = j > 0 ? id[j - 1] : -1;
            id[j] = i;
        } else {
            st.pb(a[i]);
            p[i] = j > 0 ? id[j - 1] : -1;
            id.pb(i);
        }
    }
    for (int i = sz(id) - 1; i > 0; i--) id[i - 1] = p[id[i]];
    return {id, p};
} // 990943
```

Subset sum. `subSum(x)`: $x_i \geq 0$, $S = \sum x_i$. Retorna `p[s]` = índice usado para atingir s, ou -1; soma 0 é possível apesar de `p[0]=-1`. Reconstrua com $s \leftarrow s - x_{p[s]}$ até 0. $O(n(1 + S/64) + S)$.

a4f267 - dsa/fast-subset-sum.cpp - 22 lines

```
vi subSum(vi x) {
    using ull = unsigned long long;
    int s = reduce(all(x));
    vector<ull> bs(s / 64 + 1);
    bs[0] = 1;
    vi p(s + 1, -1);
    rep(i,0,sz(x)) {
        int a = x[i], q = a / 64, r = a % 64;
        for (int j = sz(bs) - 1; j >= 0; --j) {
            ull pre = bs[j], add = 0;
            if (j >= q) add |= (q ? bs[j - q] : pre) << r;
            if (r && j > q) add |= bs[j - q - 1] >> (64 - r);
            bs[j] |= add;
            for (ull k = bs[j] ^ pre; k; k &= k - 1) {
                int t = __builtin_ctzll(k);
                int v = 64 * j + t;
                if (v <= s) p[v] = i;
            }
        }
    }
    return p;
} // a4f267
```

Disjoint set union com rollback. `merge(x,y)` retorna se uniu componentes. `rollback()` desfaz a última chamada a `merge`, inclusive sem efeito, e retorna se desfez uma união; requer histórico não vazio. `root/merge`: $O(\log n)$; rollback: $O(1)$. Sem compressão de caminhos. Memória $O(n + q)$ para q chamadas ainda não desfeitas.

a811cc - dsa/dsu.cpp - 24 lines

```
struct DSU {
    vi par, cnt, hist;
    DSU(int n): par(n), cnt(n, 1) {
        iota(all(par), 0);
    }
    int root(int x) {
        if (par[x] == x) return x;
        return root(par[x]);
    } // 07c331
    bool merge(int x, int y) {
        x = root(x), y = root(y);
        if (x == y) { hist.pb(-1); return false; }
        if (cnt[x] < cnt[y]) swap(x, y);
        hist.pb(y);
        cnt[x] += cnt[y], par[y] = x;
        return true;
    } // 751296
    bool rollback() {
        int y = hist.back(); hist.pop_back();
        if (y < 0) return false;
        cnt[par[y]] -= cnt[y]; par[y] = y;
        return true;
    } // 234579
};
```

Union-find de intervalos paralelos. `RangeUnionFind(n)`, $n > 0$. `merge(a,b,len,f)` une $a + i$ com $b + i$ para $0 \leq i <$ `len`; intervalos podem se sobrepor. `f(x,y)` é chamado após cada união efetiva de componentes de posições: x sobrevive, y é absorvido. Para Q operações e callback $O(1)$: $O((n \log n + Q)\alpha(n))$ tempo total, $O(n \log n)$ memória.

bca612 - dsa/range-unionfind.cpp - 24 lines

```
struct RangeUnionFind {
    vector<vi> p;
    RangeUnionFind(int n): p(bit_width(unsigned(n))) {
        for (int k = 0; k < int(p.size()); ++k)
            p[k].assign(n-(1<<k)+1,-1);
```

```
    }
    int root(int x, int k = 0) {
        return p[k][x] < 0 ? x : p[k][x] = root(p[k][x],k);
    } // 712469
    void join(int a, int b, int k, auto &f) {
        int x = root(a,k), y = root(b,k);
        if (x == y) return;
        if (p[k][x] > p[k][y]) swap(x,y);
        p[k][x] += p[k][y]; p[k][y] = x;
        if (!k) { f(x,y); return; }
        int d = 1<<(k-1);
        join(a,b,k-1,f); join(a+d,b+d,k-1,f);
    } // 24c309
    void merge(int a, int b, int len, auto f) {
        if (!len) return;
        int k = bit_width(unsigned(len))-1, d = len-(1<<k);
        join(a,b,k,f); join(a+d,b+d,k,f);
    } // 38fa4f
};
```

Sparse table. `build(v)`, `query(l,r)` em $[l, r)$ não vazio. `op` associativa e idempotente. Construção $O(n \log n)$; consulta $O(1)$.

9310ff - dsa/sparse-table.cpp - 15 lines

```
template <class S, S (*op)(S, S)> struct SparseTable {
    vector<vector<S>> t;
    void build(vector<S> &v) {
        int n = sz(v), levels = max(1, (int)bit_width((unsigned)n));
        t.assign(levels, vector<S>(n));
        t[0] = v;
        rep(k,1,levels)
            for (int i = 0; i + (1 << k) <= n; i++)
                t[k][i] = op(t[k-1][i], t[k-1][i+(1 << (k-1))]);
    } // 891c13
    S query(int l, int r) {
        int k = bit_width((unsigned)(r-l))-1;
        return op(t[k][l], t[k][r-(1 << k)]);
    } // 0a29bf
};
```

Árvore de Fenwick. `Fenwick(n)`: índices 0-based. `add(i,v)` soma v; `sum(r)` soma $[0, r)$. $O(\log n)$ por operação.

9e504c - dsa/fenwick-tree.cpp - 13 lines

```
struct Fenwick {
    vector<ll> t;
    int n;
    Fenwick(int N) : t(N), n(N) {}
    void add(int i, ll val) {
        for (i++; i <= n; i += i & -i) t[i - 1] += val;
    } // 638501
    ll sum(int r) {
        ll s = 0;
        for (; r; r -= r & -r) s += t[r - 1];
        return s;
    } // 025015
};
```

Deque de monoide. `MonoidDeque<S,op,e>`: `query()` agrega da frente para trás; `e()` se vazio. `op` associativa, não necessariamente comutativa. Atualizações $O(1)$ amortizado; consultas $O(1)$, com operações/cópias $O(1)$; memória $O(n)$.

461f2e - dsa/monoid-deque.cpp - 31 lines

```
template<class S, S (*op)(S,S), S (*e)()>
struct MonoidDeque {
    vector<pair<S,S>> t[2];
    int size() const { return t[0].size()+t[1].size(); }
    S agg(int s) const { return t[s].empty() ? e() : t[s].back().second; }
    void push(int s, S x) {
        S a = agg(s);
        t[s].push_back({x,s ? op(a,x) : op(x,a)});
    } // 35a245
    void pop(int s) {
        if (t[s].empty()) {
            vector<pair<S,S>> v;
            v.swap(t[1-s]);
            int n = v.size(), k = (n+1)/2;
            for (int i = k-1; i >= 0; --i) push(s,v[i].first);
            for (int i = k; i < n; ++i) push(1-s,v[i].first);
        }
        t[s].pop_back();
    } // 9486fe
    void push_front(S x) { push(0,x); }
    void push_back(S x) { push(1,x); }
    void pop_front() { pop(0); }
    void pop_back() { pop(1); }
    S front() const {
```

```
        return t[0].empty() ? t[1].front().first : t[0].back().first;
    } // c1d8e1
    S back() const {
        return t[1].empty() ? t[0].front().first : t[1].back().first;
    } // 4e19c0
    S query() const { return op(agg(0),agg(1)); }
};
```

Segtree simples. Monoide `op,e`. `Segtree(n)`: índices 0-based; `set(i,x)`, `query(l,r)` em $[l, r)$. $O(\log n)$ por operação.

883831 - dsa/simple-segtree.cpp - 20 lines

```
struct Segtree {
    using S = int;
    const S e = 0;
    S op(S a, S b) { return max(a, b); }
    vector<S> t;
    int n;
    Segtree(int N): t(2 * N, e), n(N) {}
    void set(int i, S val) {
        t[i += n] = val;
        while (i >>= 1) t[i] = op(t[i << 1], t[i << 1 | 1]);
    } // 5ffb48
    S query(int l, int r) {
        S al = e, ar = e;
        for (l += n, r += n; l < r; l >>= 1, r >>= 1) {
            if (l & 1) al = op(al, t[l++]);
            if (r & 1) ar = op(t[--r], ar);
        }
        return op(al, ar);
    } // 532d77
};
```

Segtree. `set(i,x)`, `query(l,r)` em $[l, r)$; monoide `op,e`, inclusive não comutativo. Índices 0-based. $O(\log n)$ por operação.

b76e87 - dsa/segtree.cpp - 18 lines

```
template<class S, S (*op)(S, S), S (*e)()>
struct Segtree {
    vector<S> t;
    int n;
    Segtree(int N): t(2 * N, e()), n(N) {}
    void set(int i, S value) {
        t[i += n] = value;
        for (i >>= 1; i; i >>= 1) t[i] = op(t[i << 1], t[i << 1 | 1]);
    } // 630e57
    S query(int l, int r) {
        S al = e(), ar = e();
        for (l += n, r += n; l < r; l >>= 1, r >>= 1) {
            if (l & 1) al = op(al, t[l++]);
            if (r & 1) ar = op(t[--r], ar);
        }
        return op(al, ar);
    } // 9ee903
};
```

Lazy segtree. `set(i,x)`, `apply(l,r,u)`, `sum(l,r)` em $[l, r)$, $n > 0$. `compose(novo,antigo)` aplica antigo antes de novo. `mapping` deve respeitar `op`; $O(\log n)$ por operação.

21bbc4 - dsa/lazy-segtree.cpp - 50 lines

```
template<class S, S (*op)(S, S), S (*e)(), class U, S (*mapping)(U, S),
         U (*compose)(U, U), U (*id)()>
struct LazySegtree {
    vector<S> t;
    vector<U> d;
    int n;
    LazySegtree(int N): t(4 * N, e()), d(4 * N, id()), n(N) {}
    void applyNode(int i, U u) {
        t[i] = mapping(u, t[i]);
        d[i] = compose(u, d[i]);
    } // 3e5d0d
    void push(int i) {
        applyNode(i << 1, d[i]);
        applyNode(i << 1 | 1, d[i]);
        d[i] = id();
    } // 7836a8
    void pull(int i) { t[i] = op(t[i << 1], t[i << 1 | 1]); }
    void set(int j, S val) { set(j, val, 0, n, 1); }
    void set(int j, S val, int tl, int tr, int i) {
        if (tl + 1 == tr) {
            t[i] = val;
            d[i] = id();
            return;
        }
        push(i);
        int tm = (tl + tr) / 2;
        if (j < tm) set(j, val, tl, tm, i << 1);
```

```
        else set(j, val, tm, tr, i << 1 | 1);
        pull(i);
    } // f9c2fd
    void apply(int l, int r, U u) { apply(l, r, u, 0, n, 1); }
    void apply(int l, int r, U u, int tl, int tr, int i) {
        if (r <= tl || tr <= l) return;
        if (l <= tl && tr <= r) return applyNode(i, u);
        push(i);
        int tm = (tl + tr) / 2;
        apply(l, r, u, tl, tm, i << 1);
        apply(l, r, u, tm, tr, i << 1 | 1);
        pull(i);
    } // 3f0fb7
    S sum(int l, int r) { return sum(l, r, 0, n, 1); }
    S sum(int l, int r, int tl, int tr, int i) {
        if (r <= tl || tr <= l) return e();
        if (l <= tl && tr <= r) return t[i];
        push(i);
        int tm = (tl + tr) / 2;
        return op(sum(l, r, tl, tm, i << 1),
                  sum(l, r, tm, tr, i << 1 | 1));
    } // b976f6
};
```

Max add segtree. `MaxAddSegtree(n)`: `update(l,r,v)` soma v em $[l, r)$; `query(l,r)` retorna o máximo. $O(\log n)$ por operação.

c008ec - dsa/max-add-segtree.cpp - 41 lines

```
struct MaxAddSegtree {
    int n, h;
    vector<i64> t, d;
    MaxAddSegtree(int n) : n(n), h(32 - __builtin_clz(n)), t(2 * n), d(n) {}
    void apply(int p, i64 v) {
        t[p] += v;
        if (p < n) d[p] += v;
    } // 94936b
    void build(int p) {
        while (p > 1) p >>= 1, t[p] = max(t[p<<1], t[p<<1|1]) + d[p];
    } // c20980
    void push(int p) {
        for (int s = h; s > 0; --s) {
            int i = p >> s;
            if (d[i]) {
                apply(i<<1, d[i]);
                apply(i<<1|1, d[i]);
                d[i] = 0;
            }
        }
    } // 452305
    void update(int l, int r, i64 v) {
        l += n, r += n;
        int l0 = l, r0 = r;
        for (; l < r; l >>= 1, r >>= 1) {
            if (l & 1) apply(l++, v);
            if (r & 1) apply(--r, v);
        }
        build(l0); build(r0 - 1);
    } // 8a7429
    i64 query(int l, int r) {
        l += n, r += n;
        push(l); push(r - 1);
        i64 res = -1e18;
        for (; l < r; l >>= 1, r >>= 1) {
            if (l & 1) res = max(res, t[l++]);
            if (r & 1) res = max(res, t[--r]);
        }
        return res;
    } // f655b5
};
```

Segtree beats dinâmica. `SegtreeBeats`: domínio inteiro $[0, n)$, $n > 0$. `set(p,x)`, `apply(l,r,u)`, `query(l,r)` em $[l, r)$. `op,e` formam um monoide; `init(l,r)` fornece o agregado inicial do intervalo, compatível com `op`. `mapping(u,s)` aplica em `s` e retorna verdadeiro, ou retorna falso **sem alterar** `s` para forçar descida; deve aceitar folhas e identidade. `compose(novo,antigo)` aplica antigo antes de novo, com neutro `id`. Consultas também podem alocar. Com callbacks/cópias $O(1)$: custo proporcional aos nós visitados, memória aos criados; limite amortizado depende de `mapping`.

4df04f - dsa/segtree-beats.cpp - 61 lines

```
template<class S, S (*op)(S,S), S (*e)(), class U,
         bool (*mapping)(U,S&), U (*compose)(U,U), U (*id)(),
         S (*init)(ll,ll)>
struct SegtreeBeats {
    struct Node { S s; U tag; int a = -1, b = -1; };
    vector<Node> t;
    ll n;
```

```
    SegtreeBeats(ll n): n(n) { make(0,n); }
    int make(ll l, ll r) {
        t.push_back({init(l,r),id()});
        return int(t.size())-1;
    } // 821514
    void pull(int i) { t[i].s = op(t[t[i].a].s,t[t[i].b].s); }
    void allApply(int i, ll l, ll r, U u) {
        if (mapping(u,t[i].s)) {
            t[i].tag = compose(u,t[i].tag); return;
        }
        assert(r-l > 1);
        push(i,l,r);
        ll m = l+(r-l)/2;
        allApply(t[i].a,l,m,u); allApply(t[i].b,m,r,u);
        pull(i);
    } // 532810
    void push(int i, ll l, ll r) {
        ll m = l+(r-l)/2;
        if (t[i].a < 0) {
            int a = make(l,m), b = make(m,r);
            t[i].a = a; t[i].b = b;
        }
        U u = t[i].tag; t[i].tag = id();
        allApply(t[i].a,l,m,u); allApply(t[i].b,m,r,u);
    } // 9d23bc
    void set(ll p, S x) { set(p,x,0,0,n); }
    void set(ll p, S x, int i, ll l, ll r) {
        if (r-l == 1) { t[i].s = x; t[i].tag = id(); return; }
        push(i,l,r);
        ll m = l+(r-l)/2;
        if (p < m) set(p,x,t[i].a,l,m);
        else set(p,x,t[i].b,m,r);
        pull(i);
    } // 864489
    void apply(ll l, ll r, U u) {
        if (l < r) apply(l,r,u,0,0,n);
    } // 65a4f2
    void apply(ll l, ll r, U u, int i, ll a, ll b) {
        if (r <= a || b <= l) return;
        if (l <= a && b <= r) return allApply(i,a,b,u);
        push(i,a,b);
        ll m = a+(b-a)/2;
        apply(l,r,u,t[i].a,a,m); apply(l,r,u,t[i].b,m,b);
        pull(i);
    } // c2b5e1
    S query(ll l, ll r) { return l < r ? query(l,r,0,0,n) : e(); }
    S query(ll l, ll r, int i, ll a, ll b) {
        if (r <= a || b <= l) return e();
        if (l <= a && b <= r) return t[i].s;
        push(i,a,b);
        ll m = a+(b-a)/2;
        return op(query(l,r,t[i].a,a,m),query(l,r,t[i].b,m,b));
    } // a8b812
};
```

Segtree com rollback: chmax e mínimo. `RollbackChmax(n,initial)`, $n > 0$. `insert({l,r,x})` faz $a_i \leftarrow \max(a_i, x)$ em $[l, r)$; `query(l,r)` retorna mínimo (`LLONG_MAX` se vazio), ambas $O(\log n)$. `rollback()` desfaz uma inserção, inclusive sem efeito; requer histórico não vazio, custo proporcional aos nós salvos. Consultas não modificam o histórico. Memória $O(n + k \log n)$ para k inserções ativas.

ed7c85 - dsa/rollback-segtree.cpp - 38 lines

```
struct RollbackChmax {
    using Value = tuple<int,int,ll>;
    struct Node { ll mn, tag; };
    vector<Node> t;
    vector<pair<int,Node>> log;
    vector<int> marks;
    int n;
    RollbackChmax(int n, ll initial=0)
        : t(4*n,{initial,LLONG_MIN}), n(n) {}
    void update(int l, int r, ll x, int i, int a, int b) {
        if (r <= a || b <= l || t[i].mn >= x) return;
        log.push_back({i,t[i]});
        if (l <= a && b <= r) {
            t[i].mn = max(t[i].mn,x);
            t[i].tag = max(t[i].tag,x); return;
        }
        int m = (a+b)/2;
        update(l,r,x,2*i,a,m); update(l,r,x,2*i+1,m,b);
        t[i].mn = max(t[i].tag,min(t[2*i].mn,t[2*i+1].mn));
    } // 81bdfd
    void insert(Value v) {
        marks.push_back(sz(log));
        auto [l,r,x] = v; update(l,r,x,1,0,n);
    } // c9ca32
```

```
    void rollback() {
        while (sz(log) > marks.back()) {
            auto [i,node] = log.back(); log.pop_back(); t[i] = node;
        }
        marks.pop_back();
    } // 83daef
    ll query(int l, int r, int i, int a, int b) const {
        if (r <= a || b <= l) return LLONG_MAX;
        if (l <= a && b <= r) return t[i].mn;
        int m = (a+b)/2;
        return max(t[i].tag,min(query(l,r,2*i,a,m),query(l,r,2*i+1,m,b)));
    } // 9f6f58
    ll query(int l, int r) const { return query(l,r,1,0,n); }
};
```

Deletar offline. `OfflineDeletion<DS>`: registre `insert(x)`/`remove(x)`, depois `run(ds)`. Requer `DS::Value` ordenável, `insert(x)`, `rollback()` para desfazer uma inserção e `handle(k)` para consultar após a operação k (1-based). Semântica de conjunto; operações sem efeito também avançam o tempo. $O(Q \log Q)$ inserções/rollbacks, além de Q chamadas a `handle`.

121ed5 - dsa/offline-deletions.cpp - 45 lines

```
template<class DS>
struct OfflineDeletion {
    using V = typename DS::Value;
    int t = 0, s = 1;
    map<V, int> on;
    vector<tuple<int, int, V>> in;
    vector<vector<V>> st;
    void insert(V x) {
        if (!on.count(x)) on[x] = t;
        t++;
    } // 9112d3
    void remove(V x) {
        auto it = on.find(x);
        if (it != on.end())
            in.eb(it->se, t, x), on.erase(it);
        t++;
    } // 2ffdb3
    void add(int l, int r, V x) {
        for (l += s, r += s; l < r; l >>= 1, r >>= 1) {
            if (l & 1) st[l++].pb(x);
            if (r & 1) st[--r].pb(x);
        }
    } // 8e0d5a
    void dfs(DS &ds, int i = 1, int l = 0, int r = -1) {
        if (r < 0) r = s;
        int c = sz(st[i]);
        for (V x: st[i]) ds.insert(x);
        if (r - l == 1) {
            if (l < t) ds.handle(r);
        } else {
            int m = (l + r) / 2;
            dfs(ds, i << 1, l, m);
            dfs(ds, i << 1 | 1, m, r);
        }
        while (c--) ds.rollback();
    } // add77c
    void run(DS &ds) {
        for (auto [x, l]: on) in.eb(l, t, x);
        on.clear();
        while (s < t) s <<= 1;
        st.assign(2 * s, {});
        for (auto [l, r, x]: in) if (l < r) add(l, r, x);
        dfs(ds);
    } // 04f871
};
```

Consultas de Mo. `moOrd(q,n)`: permutação dos índices das consultas `{l,r}`. Bloco $\sqrt{n}$: $O(Q \log Q)$ para ordenar e $O((n+Q)\sqrt{n})$ movimentos, vezes o custo de adicionar/remover.

c2c90b - dsa/mo-queries.cpp - 11 lines

```
vi moOrd(vector<pii> &q, int n) {
    int B = max(1, (int)sqrt(n));
    vi o(sz(q));
    iota(all(o), 0);
    sort(all(o), [&](int i, int j) {
        int a = q[i].fi / B, b = q[j].fi / B;
        if (a != b) return a < b;
        return (a & 1) ? q[i].se > q[j].se : q[i].se < q[j].se;
    } /* f3c775 */);
    return o;
} // c2c90b
```

Wavelet matrix. `wavMat(a)`: $0 \le a_i < 2^H$. Consultas em $[l, r)$ com altura inicial H: `rangeKth(...,k)`, $0 \le k < r - l$; `cntLess(...,x)`, $0 \le x < 2^H$. Construção $O(Hn)$; consulta $O(H)$.

399393 - dsa/wavelet-matrix.cpp - 36 lines

```cpp
const int H = 31;
using WM = array<vi, H>;
WM wavMat(vector<ll> a) {
    WM wm;
    int n = sz(a);
    for (int i = H - 1; i >= 0; i--) {
        wm[i].resize(n + 1);
        rep(j, 1, n + 1) wm[i][j] = a[j - 1] >> i & 1;
        stable_partition(all(a), [&](ll x) { return ~x >> i & 1; });
        rep(j, 1, n + 1) wm[i][j] += wm[i][j - 1];
    }
    return wm;
} // 9aeaa7

tuple<int, int, int, int> getSplit(WM &wm, int h, int l, int r) {
    int a1 = wm[h][l], a0 = l - a1,
        b1 = wm[h][r], b0 = r - b1,
        n = sz(wm[0]) - 1, c = n - wm[h][n];
    return {a0, b0, c + a1, c + b1};
} // f64ad6

ll rangeKth(WM &wm, int h, int l, int r, int k) {
    if (h == 0) return 0;
    auto [l0, r0, l1, r1] = getSplit(wm, h - 1, l, r);
    int sl = r0 - l0;
    if (k < sl) return rangeKth(wm, h - 1, l0, r0, k);
    return (1 << (h - 1)) + rangeKth(wm, h - 1, l1, r1, k - sl);
} // 97e884

int cntLess(WM &wm, int h, int l, int r, ll x) {
    if (h == 0) return 0;
    auto [l0, r0, l1, r1] = getSplit(wm, h - 1, l, r);
    if (x >> (h - 1) & 1)
        return r0 - l0 + cntLess(wm, h - 1, l1, r1, x);
    return cntLess(wm, h - 1, l0, r0, x);
} // fcb25d
```

Wavelet matrix: posições e retângulos. `wavPos(wm)` retorna `pos[h][i]`: posição do índice original i após a partição do nível h, em $O(Hn)$. `rectOp(wm,l,r,a,b,f)` cobre $i \in [l, r)$, $a \le x_i < b$ por $O(H)$ chamadas disjuntas `f(h,L,R)` na ordem de `pos[h]`. Requer $0 \le a \le b \le 2^H$.

445682 - dsa/wavelet-matrix-extended.cpp - 26 lines

```cpp
WM wavPos(WM &wm) {
    WM pos;
    int n = sz(wm[0])-1;
    vi ord(n); iota(all(ord),0);
    for (int h = H-1; h >= 0; --h) {
        pos[h].resize(n);
        rep(i,0,n) {
            int k = ord[i], ones = wm[h][k];
            ord[i] = wm[h][k+1] > ones ? n-wm[h][n]+ones : k-ones;
            pos[h][i] = ord[i];
        }
    }
    return pos;
} // d118d3

void rectOp(WM &wm, int l, int r, ll a, ll b, auto f) {
    auto go = [&](auto &&go, int h, int l, int r, ll lo, ll hi) -> void {
        if (l == r || b <= lo || hi <= a || a == b) return;
        if (h < H && a <= lo && hi <= b) { f(h,l,r); return; }
        auto [l0,r0,l1,r1] = getSplit(wm,h-1,l,r);
        ll mid = (lo+hi)/2;
        go(go,h-1,l0,r0,lo,mid);
        go(go,h-1,l1,r1,mid,hi);
    }; // 305538
    go(go,H,l,r,0,1LL<<H);
} // 51d66e
```

Trie de XOR. `XorTrie<B=30>`: multiconjunto de inteiros sem sinal em $[0, 2^B)$, $1 \le B \le 64$. `add(x)` insere; `add(x,-1)` remove uma ocorrência existente. `kth(k,x=0)` retorna o k-ésimo menor **valor de XOR** com x, $0 \le k <$ `size()`. $O(B)$ por operação; nós não são liberados, memória $O(BD)$ para D valores distintos já inseridos.

e3688f - dsa/xor-trie.cpp - 32 lines

```cpp
template<int B = 30> struct XorTrie {
    using T = uint64_t;
    struct Node { int ch[2] = {-1,-1}, cnt = 0; };
    vector<Node> t{Node{}};
    int size() const { return t[0].cnt; }
    int count(T x) const {
        int p = 0;
```

```
        for (int i = B-1; i >= 0 && p >= 0; --i) p = t[p].ch[x>>i&1];
        return p < 0 ? 0 : t[p].cnt;
    } // cc4712
    void add(T x, int d = 1) {
        int p = 0; t[p].cnt += d;
        for (int i = B-1; i >= 0; --i) {
            int b = x>>i&1;
            if (t[p].ch[b] < 0) {
                t[p].ch[b] = t.size(); t.emplace_back();
            }
            p = t[p].ch[b]; t[p].cnt += d;
        }
    } // f19b65
    T kth(int k, T x = 0) const {
        int p = 0;
        T ans = 0;
        for (int i = B-1; i >= 0; --i) {
            int b = x>>i&1, q = t[p].ch[b];
            int c = q < 0 ? 0 : t[q].cnt;
            if (k >= c) k -= c, b ^= 1, ans |= T(1)<<i;
            p = t[p].ch[b];
        }
        return ans;
    } // e421b4
};
```

Árvore cartesiana. `cartTree(n,le)`: pai de cada índice; a raiz é seu próprio pai. `le(i,j)` decide se i fica acima de j, inclusive desempate. $O(n)$.

caeaba - dsa/cartesian-tree.cpp - 20 lines

```
vi cartTree(int n, auto le) {
    int last;
    vi st, par(n);
    rep (i, 0, n) {
        par[i] = i, last = -1;
        while (sz(st) && le(par[i], st.back())) {
            if (last != -1) par[last] = st.back();
            last = st.back();
            st.pop_back();
        }
        if (last != -1) par[last] = i;
        st.pb(i);
    }
    while (sz(st) > 1) {
        last = st.back();
        st.pop_back();
        par[last] = st.back();
    }
    return par;
} // caeaba
```

Treap implícita. `split(t,k,a,b)` separa os primeiros k elementos; `sep(t,l,r,a,b,c)` isola $[l, r)$ em `b`. Índices 0-based em `ins,era,kth`. $O(\log n)$ esperado.

ad716f - dsa/treap.cpp - 69 lines

```
struct Node;
using TN = Node*;

struct Node {
    int y = rng(), cnt, v;
    ll sum;
    bool rev = 0;
    TN l = 0, r = 0;
    Node(int v) : cnt(1), v(v), sum(v) {}
};

int cnt(TN t) { return t ? t->cnt : 0; }
ll sum(TN t) { return t ? t->sum : 0; }

void apply_rev(TN t) {
    if (!t) return;
    t->rev ^= 1;
    swap(t->l, t->r);
} // 907c51
void push(TN t) {
    if (!t || !t->rev) return;
    apply_rev(t->l);
    apply_rev(t->r);
    t->rev = 0;
} // b46a2a
void pull(TN t) {
    t->cnt = 1 + cnt(t->l) + cnt(t->r);
    t->sum = t->v + sum(t->l) + sum(t->r);
} // 6ee5eb

void split(TN t, int k, TN& a, TN& b) {
```

```
    if (!t) return a = b = 0, void();
    push(t);
    if (cnt(t->l) >= k) split(t->l, k, a, t->l), b = t;
    else split(t->r, k - cnt(t->l) - 1, t->r, b), a = t;
    pull(t);
} // b34b6d
TN merge(TN a, TN b) {
    if (!a || !b) return a ? a : b;
    if (a->y > b->y) return push(a), a->r = merge(a->r, b), pull(a), a;
    return push(b), b->l = merge(a, b->l), pull(b), b;
} // 529963

void ins(TN& t, int p, int v) {
    TN a, b;
    split(t, p, a, b);
    t = merge(merge(a, new Node(v)), b);
} // c2b578
void era(TN& t, int p) {
    TN a, b, c;
    split(t, p, a, b);
    split(b, 1, b, c);
    delete b;
    t = merge(a, c);
} // cf32a5
TN kth(TN t, int p) {
    while (t) {
        push(t);
        if (cnt(t->l) == p) return t;
        if (cnt(t->l) > p) t = t->l;
        else p -= cnt(t->l) + 1, t = t->r;
    }
    return 0;
} // 4ba488

void sep(TN t, int l, int r, TN& a, TN& b, TN& c) {
    split(t, r, b, c);
    split(b, l, a, b);
} // 4fa0ee
```

Árvore implícita persistente (RBST). `RBST<T>`: sequência persistente, raiz 0 vazia. `split(root,k)` separa os primeiros k elementos; `merge(a,b)` concatena. `kth(root,k)` retorna ID do nó na posição k (0-based); valor em `v[id]`. $O(\log n)$ esperado por operação.

e0a07d - dsa/rbst.cpp - 55 lines

```
mt19937_64 rng64(random_device{}());

template<class T>
struct RBST {
    vector<int> l = {0}, r = {0};
    vector<ll> c = {0};
    vector<T> v = {T{}};
    ll cnt(int x) { return c[x]; }
    void cp(int &x) {
        l.pb(l[x]), r.pb(r[x]), c.pb(c[x]), v.pb(v[x]);
        x = sz(c) - 1;
    } // db2c4c
    int node(ll x) {
        l.pb(0), r.pb(0), c.pb(1), v.pb(x);
        return sz(c) - 1;
    } // b57877
    void pull(int x) {
        c[x] = cnt(l[x]) + 1 + cnt(r[x]);
    } // c5949b
    int merge(int a, int b) {
        if (!a || !b) return a | b;
        bool q = rng64() % (cnt(a) + cnt(b)) < cnt(a);
        int x = q ? a : b;
        cp(x);
        if (q) r[x] = merge(r[x], b);
        else l[x] = merge(a, l[x]);
        pull(x);
        return x;
    } // d92d85
    pair<int, int> split(int x, ll k) { // first k, rest
        if (!x) return {};
        cp(x);
        ll m = cnt(l[x]);
        if (k <= m) {
            auto [a, b] = split(l[x], k);
            l[x] = b;
            pull(x);
            return {a, x};
        } else {
            auto [a, b] = split(r[x], k - m - 1);
            r[x] = a;
            pull(x);
```

```
            return {x, b};
        }
    } // 87294b
    int kth(int x, ll k) { // node id, 0-indexed
        while (x) {
            ll m = cnt(l[x]);
            if (k < m) x = l[x];
            else if (k == m) return x;
            else k -= m + 1, x = r[x];
        }
        return 0;
    } // 488337
};
```

Extensões GNU. Requer GNU libstdc++ (`g++`). `V=null_type`: conjunto; outro tipo: mapa. Na árvore, `order_of_key(x)` conta chaves anteriores a x pelo comparador; `find_by_order(k)` retorna iterador, 0-based. Operações $O(\log n)$. Para repetidos, use `{valor,id}` com IDs únicos; comparador estrito, nunca `less_equal`. Hash: $O(1)$ esperado amortizado com boa dispersão, $O(n)$ por busca no pior caso; use hash aleatório contra entradas adversárias. Rehash invalida iteradores/referências. Heap: `push/join` $O(1)$; `pop/erase/modify` $O(\log n)$ amortizado. Handles sobrevivem a alterações e `join`, até remover o elemento. `rope`: Sequência com cópias compartilhadas; alterações preservam outras versões. Índices 0-based; `substr/erase(pos,len)` usam quantidade, não extremo final. Em recortes, a posição de inserção refere-se à sequência após a remoção. Acesso por índice $O(h)$, altura da árvore; inserir k elementos novos exige ao menos $O(k)$. Rebalanceamento e liberação de nós podem custar tempo linear; não assuma toda edição $O(\log n)$. Sem agregados ou reversão lazy. Bitsets: buscas retornam `size()` se não houver bit ligado; `find_next(i)` busca após i. Operações binárias exigem tamanhos iguais. No bitset dinâmico, `clear()` remove todos os bits (tamanho zero); `reset()` apenas zera os valores.

```
#include <functional>
#include <utility>
#include <ext/pb_ds/assoc_container.hpp>
#include <ext/pb_ds/tree_policy.hpp>
#include <ext/pb_ds/priority_queue.hpp>
namespace pb = __gnu_pbds;

template<class K, class V = pb::null_type, class C = std::less<K>>
using OrderedTree = pb::tree<K,V,C,pb::rb_tree_tag,
    pb::tree_order_statistics_node_update>;
template<class K, class V = pb::null_type, class H = std::hash<K>>
using HashTable = pb::gp_hash_table<K,V,H>;
template<class T, class C = std::less<T>>
using Heap = pb::priority_queue<T,C,pb::pairing_heap_tag>;

void pbdsExample() {
    OrderedTree<std::pair<int,int>> s; // {valor,id}, id >= 0
    s.insert({5,0}); s.insert({5,1});
    s.order_of_key({5,-1}); // 0: quantidade < 5
    s.order_of_key({6,-1}); // 2: quantidade < 6
    auto it = s.find_by_order(1); // {5,1}; end() se k >= size()
    s.erase(it); // remove apenas esta ocorrencia
    OrderedTree<int,int> m; m[4] = 9; // mapa com rank por chave
    HashTable<int,int> h; h[3] = 7;
    HashTable<int> seen; seen.insert(3);
    Heap<int> a, b; // max-heap; std::greater<int> para minimo
    auto p = a.push(4); // Heap<int>::point_iterator
    a.modify(p,1);
    auto q = b.push(2);
    a.join(b); // transfere b para a; b fica vazio
    a.erase(q); // handle de b agora pertence a a
    a.top(); // 1
    a.pop(); // invalida p
}

#include <ext/rope>
using __gnu_cxx::rope;
using __gnu_cxx::crope; // rope<char>

void ropeExample() {
    int v[] = {10,20,30};
    rope<int> a(v,v+3);
    auto old = a; // snapshot
    auto part = a.substr(1,2);
    a.erase(1,2); // posicao, quantidade
    a.insert(0,part); // [20,30,10]; old = [10,20,30]
    a.replace(0,99); // [99,30,10]; part = [20,30]
    a[1]; // 30
    crope s("abc"); s += "def";
}

#include <bitset>
#include <tr2/dynamic_bitset>
```

```cpp
void bitsetExample(int n, int x) { // n > 0, x >= 0
    std::bitset<100> b;
    b.set(3); b.set(70);
    for (size_t i = b._Find_first(); i < b.size(); i = b._Find_next(i)) {
        // processar i
    }
    std::tr2::dynamic_bitset<> d(n);
    d[0] = 1; d |= d << x; // subset sum
    for (size_t i = d.find_first(); i < d.size(); i = d.find_next(i)) {
        // processar i
    }
    d.resize(n+1); // preserva bits; novos bits = 0
}
```

Máximo de sufixo. `SuffixMax<K,V> st(e)`; `add(x,y)` insere, `query(X)` retorna máximo y com $x \geq X$, ou `e` se vazio. $O(\log n)$ amortizado por inserção/consulta.

508151 - dsa/suffix-max.cpp - 20 lines

```cpp
template<class K, class V>
struct SuffixMax {
    map<K, V> m;
    V e;
    SuffixMax(V e = V{}): e(e) {}
    void add(K x, V y) {
        auto it = m.lower_bound(x);
        if (it != end(m) && it->se >= y) return;
        it = m.insert_or_assign(it, x, y);
        while (it != begin(m)) {
            auto p = prev(it);
            if (y < p->se) break;
            m.erase(p);
        }
    } // 3a4c71
    V query(K x) {
        auto it = m.lower_bound(x);
        return it == end(m) ? e : it->se;
    } // 15381b
};
```

CHT estático e pontos de troca. `StaticCHT(v)`: máximo de retas $ax+b$, construção $O(n \log n)$. `h[i]` é ótima em $[\texttt{start}[i], \texttt{start}[i+1])$ (último até $+\infty$); `query(x)` em $O(\log n)$, requer retas e x finito. Interseções em `ld`: há erro de arredondamento.

0111d5 - dsa/static-cht.cpp - 27 lines

```cpp
struct StaticCHT {
    struct Line { ll a, b; ld at(ld x) const { return a*x+b; } };
    vector<Line> h;
    vector<ld> start;
    StaticCHT(vector<Line> v) {
        sort(all(v), [](Line a, Line b) {
            return tie(a.a,a.b) < tie(b.a,b.b);
        } /* dd4249 */);
        for (auto l : v) {
            ld x = -numeric_limits<ld>::infinity();
            while (!h.empty()) {
                auto p = h.back();
                if (l.a != p.a) {
                    x = ((ld)p.b-l.b)/((ld)l.a-p.a);
                    if (x > start.back()) break;
                }
                h.pop_back(); start.pop_back();
            }
            if (h.empty()) x = -numeric_limits<ld>::infinity();
            h.push_back(l); start.push_back(x);
        }
    }
    int index(ld x) const {
        return upper_bound(all(start),x)-start.begin()-1;
    } // 9977ee
    ld query(ld x) const { return h[index(x)].at(x); }
};
```

Árvore de Li Chao. `LiChao(N)` mantém o mínimo em $[0, N)$, $N > 0$. `insert(line)`: reta inteira, $O(\log N)$; `insert(line,L,R)`: ativa em $[L, R)$, $O(\log^2 N)$. `query(x)` retorna a melhor reta em $O(\log N)$. Para máximo, inverta comparações e sentinela.

a7d6fc - dsa/li-chao-extended.cpp - 35 lines

```cpp
struct Line {
    ll a, b;
    ll operator()(int x) { return a * x + b; }
};
struct LiChao {
    vector<Line> t;
    int n;
    LiChao(int N): t(4 * N, {0, LLONG_MAX}), n(N) {}
```

```
    void insert(Line line) { insert(line, 0, n); }
    void insert(Line line, int L, int R) { insert(line, L, R, 0, n, 1); }
    void insert(Line line, int L, int R, int tl, int tr, int i) {
        int tm = tl + (tr - tl) / 2;
        if (L >= tr || R <= tl) return;
        if (L <= tl && tr <= R) {
            if (line(tm) < t[i](tm)) swap(line, t[i]);
            if (tl + 1 == tr) return;
            if (t[i](tl) < line(tl)) insert(line, L, R, tm, tr, i << 1 | 1);
            else insert(line, L, R, tl, tm, i << 1);
        } else {
            if (tl + 1 == tr) return;
            insert(line, L, R, tm, tr, i << 1 | 1);
            insert(line, L, R, tl, tm, i << 1);
        }
    } // 3dcc38
    Line query(int x) {
        Line line = {0, LLONG_MAX};
        for (int tl = 0, tr = n, i = 1;;) {
            if (t[i](x) < line(x)) line = t[i];
            if (tl + 1 == tr) return line;
            int tm = tl + (tr-tl)/2;
            if (x < tm) tr = tm, i <<= 1;
            else tl = tm, i = i << 1 | 1;
        }
    } // cc25f8
};
```

Simplex. `Simplex(A,b,c).solve(x)` maximiza $c \cdot x$ com $Ax \leq b$, $x \geq 0$. Escreve uma solução em `x`; retorna $-$INF se inviável, $+$INF se ilimitado.

becd4c - dsa/simplex.cpp - 57 lines

```
struct Simplex {
    static constexpr ld EPS = 1e-10;
    static constexpr ld INF = 1e100;
    int m, n;
    vi B, N;
    vector<vector<ld>> D;
    Simplex(vector<vector<ld>> A, vector<ld> b, vector<ld> c)
        : m(sz(b)), n(sz(c)), B(m), N(n + 1), D(m + 2, vector<ld>(n + 2)) {
        rep(i,0,m) rep(j,0,n) D[i][j] = A[i][j];
        rep(i,0,m) B[i] = n + i, D[i][n] = -1, D[i][n + 1] = b[i];
        rep(j,0,n) N[j] = j, D[m][j] = -c[j];
        N[n] = -1, D[m + 1][n] = 1;
    }
    void pivot(int r, int s) {
        ld inv = 1 / D[r][s];
        rep(i,0,m+2) if (i != r)
            rep(j,0,n+2) if (j != s) D[i][j] -= D[r][j] * D[i][s] * inv;
        rep(j,0,n+2) if (j != s) D[r][j] *= inv;
        rep(i,0,m+2) if (i != r) D[i][s] *= -inv;
        D[r][s] = inv;
        swap(B[r], N[s]);
    } // eb9c6e
    bool simplex(int ph) {
        int x = ph == 1 ? m + 1 : m;
        while (1) {
            int s = -1;
            rep(j,0,n+1) if (!(ph == 2 && N[j] == -1) &&
                (s == -1 || D[x][j] < D[x][s] - EPS ||
                (abs(D[x][j] - D[x][s]) <= EPS && N[j] < N[s]))) s = j;
            if (D[x][s] >= -EPS) return 1;
            int r = -1;
            rep(i,0,m) if (D[i][s] > EPS &&
                (r == -1 || D[i][n + 1] / D[i][s] < D[r][n + 1] / D[r][s] - EPS ||
                (abs(D[i][n + 1] / D[i][s] - D[r][n + 1] / D[r][s]) <= EPS && B[i] <
                ↪  B[r]))) r = i;
            if (r == -1) return 0;
            pivot(r, s);
        }
    } // fee585
    ld solve(vector<ld> &x) {
        int r = 0;
        rep(i,1,m) if (D[i][n + 1] < D[r][n + 1]) r = i;
        if (D[r][n + 1] < -EPS) {
            pivot(r, n);
            if (!simplex(1) || D[m + 1][n + 1] < -EPS) return -INF;
            rep(i,0,m) if (B[i] == -1) {
                int s = -1;
                rep(j,0,n+1) if (s == -1 || D[i][j] < D[i][s] - EPS ||
                    (abs(D[i][j] - D[i][s]) <= EPS && N[j] < N[s])) s = j;
                pivot(i, s);
            }
        }
        if (!simplex(2)) return INF;
        x.assign(n, 0);
        rep(i,0,m) if (B[i] < n) x[B[i]] = D[i][n + 1];
```

```
        return D[m][n + 1];
    } // 38c01f
};
```

Convolução

$c_k = \sum_{i+j=k} a_i b_j$; $O(n \log n)$.

FFT

e42ddb - dsa/fft.cpp - 41 lines

```
using cd = complex<ld>;
const ld PI = acosl(-1.0L);

void fft(vector<cd> &a) {
    int n = sz(a);
    static vector<cd> rt(2, 1);
    for (static int k = 2; k < n; k <<= 1) {
        rt.resize(n);
        cd x = polar((ld)1, PI / k);
        rep(i, k, 2 * k) rt[i] = i & 1 ? rt[i / 2] * x : rt[i / 2];
    }
    for (int i = 1, j = 0; i < n; i++) {
        int bit = n >> 1;
        for (; j & bit; bit >>= 1) j ^= bit;
        j ^= bit;
        if (i < j) swap(a[i], a[j]);
    }
    for (int k = 1; k < n; k <<= 1)
        for (int i = 0; i < n; i += k << 1)
            rep(j, 0, k) {
                cd z = rt[j + k] * a[i + j + k];
                a[i + j + k] = a[i + j] - z;
                a[i + j] += z;
            }
} // 29b822

vector<ld> conv(const vector<ld> &a, const vector<ld> &b) {
    if (a.empty() || b.empty()) return {};
    int s = sz(a) + sz(b) - 1, n = 1;
    while (n < s) n <<= 1;
    vector<cd> in(n), out(n);
    rep(i, 0, sz(a)) in[i].real(a[i]);
    rep(i, 0, sz(b)) in[i].imag(b[i]);
    fft(in);
    for (cd &x : in) x *= x;
    rep(i, 0, n) out[i] = in[-i & (n - 1)] - conj(in[i]);
    fft(out);
    vector<ld> res(s);
    rep(i, 0, s) res[i] = imag(out[i]) / (4 * n);
    return res;
} // 312a10
```

NTT

`conv(a,b)`: coeficientes normalizados módulo `mod`. O tamanho arredondado para potência de 2 deve ser $\leq 2^{23}$ nesta implementação; `ntt` modifica o vetor e exige tamanho potência de 2.

bec19a - dsa/ntt.cpp - 38 lines

```
// const int mod = 998244353, ROOT = 3;
// could also be mod = 469762049, ROOT = 3
void ntt(vector<ll> &a) {
    int n = sz(a);
    static vector<ll> r = {1, 1};
    for (int t = sz(r); t < n; t <<= 1) {
        r.resize(t << 1);
        ll z = powm(ROOT, (mod - 1) / (t << 1));
        for (int i = t; i < t << 1; i++)
            r[i] = r[i >> 1] * (i & 1 ? z : 1) % mod;
    }
    for (int i = 1, j = 0; i < n; i++) {
        int bit = n >> 1;
        for (; j & bit; bit >>= 1) j ^= bit;
        j ^= bit;
        if (i < j) swap(a[i], a[j]);
    }
    for (int m = 1; m < n; m <<= 1)
        for (int l = 0; l < n; l += m << 1)
            rep(i, 0, m) {
                ll x = a[l + i], z = r[m + i] * a[l + m + i] % mod;
                a[l + i] = (x + z < mod ? x + z : x + z - mod);
                a[l + m + i] = (x < z ? x - z + mod : x - z);
            }
} // 3904ec

vector<ll> conv(vector<ll> &a, vector<ll> &b) {
    if (a.empty() || b.empty()) return {};
    int s = sz(a) + sz(b) - 1, n = 1;
    while (n < s) n <<= 1;
```

```
    int inv = powm(n, mod - 2);
    vector<ll> L(a), R(b), out(n);
    L.resize(n), R.resize(n);
    ntt(L), ntt(R);
    rep(i, 0, n) out[-i & (n - 1)] = L[i] * R[i] % mod * inv % mod;
    ntt(out);
    return {out.begin(), out.begin() + s};
} // ad22f1
```

Convolução de XOR. `xorConv(a,b)`: $c_k = \sum_{i \oplus j = k} a_i b_j$; $O(n \log n)$.

fd3fd7 - dsa/xor-convolution.cpp - 20 lines

```
void fwht(vector<ll> &a) {
    for (int k = 1; k < sz(a); k <<= 1)
        for (int i = 0; i < sz(a); i += 2 * k)
            rep (j, i, i + k) {
                ll x = a[j], y = a[j + k];
                a[j] = x + y; // mod: a[j] = (x + y) % mod;
                a[j + k] = x - y; // mod: a[j + k] = (x - y + mod) % mod;
            }
} // ae23e1

vector<ll> xorConv(vector<ll> a, vector<ll> b) {
    int n = 1;
    while (n < max(sz(a), sz(b))) n <<= 1;
    a.resize(n), b.resize(n);
    fwht(a), fwht(b);
    rep (i, 0, n) a[i] *= b[i]; // mod: a[i] = a[i] * b[i] % mod;
    fwht(a);
    rep (i, 0, n) a[i] /= n; // mod: a[i] = a[i] * invN % mod;
    return a;
} // 0bc2c7
```

Zeta e Mobius. `subset/superset`: máscaras em $[0, 2^B)$, $O(B2^B)$. `div/mul`: divisores/múltiplos em $[1, n]$ (vetor de tamanho $n+1$; índice 0 ignorado), $O(n \log \log n)$ para $n \geq 3$. Passe `inv=true` para a inversa de Möbius; evite overflow em `int`.

1abb33 - dsa/zeta.cpp - 38 lines

```
vi subsetZeta(vi a, bool inv = false) {
    for (int j = 1; j < sz(a); j <<= 1)
        rep(i,1,sz(a)) if (i & j) inv ? a[i] -= a[i^j] : a[i] += a[i^j];
    return a;
} // 2e2d58
vi supersetZeta(vi a, bool inv = false) {
    for (int j = 1; j < sz(a); j <<= 1)
        rep(i,1,sz(a)) if (i & j) inv ? a[i^j] -= a[i] : a[i^j] += a[i];
    return a;
} // c2aec0
vi enumPrimes(int n) {
    vi pr, spf(n);
    rep(i, 2, n) {
        if (spf[i] == 0) spf[i] = i, pr.pb(i);
        for (int j: pr) {
            if (i * j >= n) break;
            spf[i * j] = j;
            if (j == spf[i]) break;
        }
    }
    return pr;
} // 46cddf
vi divZeta(vi a, bool inv = false) {
    int n = sz(a)-1;
    for (int p : enumPrimes(n+1)) rep(j,1,n/p+1) {
        int i = inv ? n/p+1-j : j;
        inv ? a[i*p] -= a[i] : a[i*p] += a[i];
    }
    return a;
} // 2db431
vi mulZeta(vi a, bool inv = false) {
    int n = sz(a)-1;
    for (int p : enumPrimes(n+1)) rep(j,1,n/p+1) {
        int i = inv ? j : n/p+1-j;
        inv ? a[i] -= a[i*p] : a[i] += a[i*p];
    }
    return a;
} // 486237
```

Convolução de subconjuntos. Convolução de OR para subconjuntos disjuntos. $O(n \log(n)^2)$.

a32e26 - dsa/subset-convolution.cpp - 16 lines

```
#define T popcount(unsigned(i))
vi subsetConv(vi &a, vi &b) {
    int n = sz(a), m = bit_width(unsigned(n));
    vector ca(m, vi(n)), cb(m, vi(n)), c(m, vi(n));
    rep(i, 0, n) ca[T][i] = a[i], cb[T][i] = b[i];
    rep(i, 0, m) ca[i] = subsetZeta(ca[i]), cb[i] = subsetZeta(cb[i]);
    rep(i, 0, n) rep(j, 0, m) rep(k, 0, m - j) {
```

```
        c[j + k][i] += ll(ca[j][i]) * cb[k][i] % mod;
        if (c[j + k][i] >= mod) c[j + k][i] -= mod;
    }
    rep(i, 0, m) c[i] = subsetZeta(c[i],true);
    vi r(n);
    rep(i, 0, n) r[i] = c[T][i];
    return r;
} // 66fa32
#undef T
```

Grafos

0-1 BFS. `bfs01(g,s)`: distâncias de `s`; `g[v]` contém `{to,w}`, $w \in \{0,1\}$. Inalcançáveis: 10^9. $O(V+E)$.

72f0a3 - graph/bfs01.cpp - 16 lines

```
vi bfs01(vector<vector<pii>> &g, int s) {
    const int INF = 1e9;
    vi dist(sz(g), INF);
    deque<int> q;
    dist[s] = 0;
    q.push_back(s);
    while (sz(q)) {
        int v = q.front(); q.pop_front();
        for (auto [to, w] : g[v]) if (dist[v] + w < dist[to]) {
            dist[to] = dist[v] + w;
            if (w) q.push_back(to);
            else q.push_front(to);
        }
    }
    return dist;
} // 72f0a3
```

Dijkstra. `dijkstra(g,s)`: distâncias de `s`; `g[v]` contém `{to,w}`, $w \geq 0$. Inalcançáveis: 2^{62}. $O((V+E)\log V)$.

4bb2a0 - graph/dijkstra.cpp - 18 lines

```
using pil = pair<int, ll>;
vector<ll> dijkstra(vector<vector<pil>> &g, int s) {
    const ll INF = 1LL << 62;
    vector<ll> dist(sz(g), INF);
    priority_queue<pll> q;
    dist[s] = 0;
    q.push({0, s});
    while (sz(q)) {
        auto [nd, v] = q.top(); q.pop();
        ll d = -nd;
        if (d != dist[v]) continue;
        for (auto [to, w] : g[v]) if (d + w < dist[to]) {
            dist[to] = d + w;
            q.push({-dist[to], to});
        }
    }
    return dist;
} // 781494
```

Ciclo de peso médio mínimo. `minMeanCycle(n,edges)` para arcos `(u,v,peso)`: retorna fração `{numerador,denominador}` não reduzida, denominador positivo, ou `nullopt` se acíclico. Aceita pesos negativos, laços, paralelas e grafo desconexo. $O(V(V+E))$ tempo e $O(V^2)$ memória.

cc02e2 - graph/min-mean-cycle.cpp - 23 lines

```
optional<pair<i128,int>> minMeanCycle(
        int n, const vector<tuple<int,int,ll>>& edges) {
    const i128 inf = i128(1)<<120;
    vector<vector<i128>> d(n+1,vector<i128>(n,inf));
    fill(d[0].begin(),d[0].end(),0);
    for (int k = 1; k <= n; ++k)
        for (auto [u,v,w] : edges) if (d[k-1][u] != inf)
            d[k][v] = min(d[k][v],d[k-1][u]+w);
    using Fraction = pair<i128,int>;
    auto less = [](Fraction a, Fraction b) {
        return a.first*b.second < b.first*a.second;
    }; // 0524e4
    optional<Fraction> ans;
    for (int v = 0; v < n; ++v) if (d[n][v] != inf) {
        Fraction worst = {d[n][v]-d[0][v],n};
        for (int k = 1; k < n; ++k) if (d[k][v] != inf) {
            Fraction cur = {d[n][v]-d[k][v],n-k};
            if (less(worst,cur)) worst = cur;
        }
        if (!ans || less(worst,*ans)) ans = worst;
    }
    return ans;
} // cc02e2
```

Caminho Euleriano. `eulerWalk(g,m,s)`: `{vértices,ids}`, ou ambos vazios se impossível a partir de `s`. IDs em $[0, m)$; em grafo não direcionado, use o mesmo ID nas duas direções. $O(n + m)$.

483eba - graph/eulerian-walk.cpp - 23 lines

```cpp
pair<vi, vi> eulerWalk(vector<vector<pii>>& g, int m, int s = 0) {
    int n = sz(g);
    vi path, walk, d(n), vis(m), it(n);
    vector<pii> st = {{s, -1}};
    d[s]++; // remove to force cycles
    while (sz(st)) {
        auto [v, e] = st.back();
        if (it[v] == sz(g[v])) {
            path.pb(v), walk.pb(e);
            st.pop_back();
        } else {
            auto [u, id] = g[v][it[v]++];
            if (!vis[id]) {
                d[v]--, d[u]++;
                vis[id] = 1;
                st.eb(u, id);
            }
        }
    }
    for (int x : d) if (x < 0) return {{}, {}};
    if (sz(walk) != m + 1) return {{}, {}};
    return { vi(path.rbegin(), path.rend()), vi(walk.rbegin() + 1, walk.rend()) };
} // 483eba
```

Componentes do grafo complementar. `complementCC(g)`: listas de vértices das componentes do complemento de um grafo simples não direcionado. $O(n+m)$ tempo, $O(n)$ memória adicional.

c2ee42 - graph/complement-components.cpp - 21 lines

```cpp
vector<vi> complementCC(vector<vi> &g) {
    int n = g.size();
    vi nxt(n+1), mark(n, -1);
    iota(nxt.begin(),nxt.end(),1); nxt[n] = 0;
    vector<vi> comps;
    while (nxt[n] != n) {
        int s = nxt[n]; nxt[n] = nxt[s];
        vi q{s};
        for (int i = 0; i < int(q.size()); ++i) {
            int u = q[i];
            for (int v : g[u]) mark[v] = u;
            for (int p = n; nxt[p] != n;) {
                int v = nxt[p];
                if (mark[v] == u) p = v;
                else nxt[p] = nxt[v], q.push_back(v);
            }
        }
        comps.push_back(move(q));
    }
    return comps;
} // c2ee42
```

Árvore de pontes. `bridges(g)`: componentes após remover as pontes de um grafo não direcionado. Aresta uv é ponte sse `comp[u]!=comp[v]`. $O(V + E)$.

c3d125 - graph/bridges.cpp - 31 lines

```cpp
vi bridges(vector<vi> &g) {
    int n = sz(g), ti = 0, cc = 0;
    vi tin(n), low(n), st, comp;
    comp.assign(n, -1);
    auto make = [&](int v) {
        while (1) {
            int x = st.back(); st.pop_back();
            comp[x] = cc;
            if (x == v) break;
        }
        cc++;
    }; // ea296f
    auto dfs = [&](auto self, int v, int p) -> void {
        tin[v] = low[v] = ++ti;
        st.pb(v);
        int usedPar = 0;
        for (int to : g[v]) {
            if (to == p && !usedPar) { usedPar = 1; continue; }
            if (!tin[to]) {
                self(self, to, v);
                low[v] = min(low[v], low[to]);
                if (low[to] > tin[v]) make(to);
            } else low[v] = min(low[v], tin[to]);
        }
    }; // 45b0f5
    rep(i,0,n) if (!tin[i]) {
        dfs(dfs, i, -1);
        make(i);
```

```
    }
    return comp;
} // c3d125
```

Árvore de bloco e corte. `bct(g)`: floresta bipartida entre vértices originais $[0, n)$ e blocos $[n, |t|)$. `|t[v]|>1` sse v é articulação. Vértices isolados também geram um bloco. $O(V + E)$.

7605c4 - graph/block-cut-tree.cpp - 35 lines

```
vector<vi> bct(vector<vi> &g) {
    int n = sz(g), ti = 0;
    vector<vi> t(n);
    vi num(n), low(n), seen(n);
    vector<pii> st;
    auto add = [&](int a, int b) {
        int id = sz(t), it = id + 1;
        t.pb({});
        while (1) {
            auto [x, y] = st.back(); st.pop_back();
            if (seen[x] != it) seen[x] = it, t[x].pb(id), t[id].pb(x);
            if (seen[y] != it) seen[y] = it, t[y].pb(id), t[id].pb(y);
            if (x == a && y == b) break;
        }
    }; // 1a9f9b
    auto dfs = [&](auto self, int v, int p) -> void {
        num[v] = low[v] = ++ti;
        int ch = 0;
        for (int to : g[v]) if (to != p) {
            if (!num[to]) {
                st.eb(v, to);
                ch++;
                self(self, to, v);
                low[v] = min(low[v], low[to]);
                if (low[to] >= num[v]) add(v, to);
            } else if (num[to] < num[v]) {
                st.eb(v, to);
                low[v] = min(low[v], num[to]);
            }
        }
        if (p == -1 && !ch) t.pb({v}), t[v].pb(sz(t) - 1);
    }; // e48eef
    rep(i, 0, n) if (!num[i]) dfs(dfs, i, -1);
    return t;
} // 7605c4
```

Numeração st. `stNumber(g,s,t)`: ordem com s primeiro, t último e cada vértice interno com um vizinho anterior e outro posterior. Assume que o grafo é biconexo após adicionar a aresta `s-t`. É necessário considerar o caso com dois vértices e nenhuma aresta separadamente. $O(V + E)$.

207b6a - graph/st-numbering.cpp - 26 lines

```
vi stNumber(vector<vi> &g, int s, int t) {
    int n = sz(g), ti = 0;
    vi in(n, -1), low(n), p(n, -1), by(n), sig(n, -1), num(n);
    auto dfs = [&](auto &&self, int v) -> void {
        in[v] = low[v] = ti, by[ti++] = v;
        auto go = [&](int u) {
            if (u == p[v]) return;
            if (in[u] < 0) p[u] = v, self(self, u), low[v] = min(low[v], low[u]);
            else low[v] = min(low[v], in[u]);
        }; // 72ea91
        if (v == s && in[t] < 0) p[t] = s, self(self, t); // virtual edge
        for (int u : g[v]) if (v != s || u != t) go(u);
    }; // 22a9e5
    dfs(dfs, s);
    list<int> l = {s, t};
    vector<list<int>::iterator> it(n);
    it[s] = l.begin(), it[t] = next(l.begin());
    rep(i,2,n) {
        int v = by[i], q = p[v], x = by[low[v]];
        if (sig[x] < 0) it[v] = l.insert(it[q], v), sig[q] = 1;
        else it[v] = l.insert(next(it[q]), v), sig[q] = -1;
    }
    int i = 0;
    for (int v : l) num[v] = i++;
    return num;
} // 207b6a
```

Componentes fortemente conexas e grafo de condensação. `scc(g)`: `{cnt,comp}`, número de SCCs e componente de cada vértice; $O(V + E)$. `condGraph(g,cnt,comp)`: DAG sem arestas repetidas; $O(V + E \log(E + 1))$.

2c057a - graph/scc.cpp - 31 lines

```
pair<int, vi> scc(vector<vi> &g) {
    int n = sz(g);
    vector<vi> rg(n);
    rep (v, 0, n) for (int u : g[v]) rg[u].pb(v);
```

```
    vi vis(n), ord, comp(n, -1);
    auto dfs1 = [&](auto self, int v) -> void {
        vis[v] = 1;
        for (int u : g[v]) if (!vis[u]) self(self, u);
        ord.pb(v);
    }; // 0fb686
    auto dfs2 = [&](auto self, int v, int c) -> void {
        comp[v] = c;
        for (int u : rg[v]) if (comp[u] < 0) self(self, u, c);
    }; // 1482a1
    rep (i, 0, n) if (!vis[i]) dfs1(dfs1, i);
    reverse(all(ord));
    int cnt = 0;
    for (int v : ord) if (comp[v] < 0) dfs2(dfs2, v, cnt++);
    return {cnt, comp};
} // a8c254

vector<vi> condGraph(vector<vi> &g, int cnt, vi &comp) {
    vector<vi> dag(cnt);
    rep (v, 0, sz(g)) for (int u : g[v])
        if (comp[v] != comp[u]) dag[comp[v]].pb(comp[u]);
    rep (i, 0, cnt) {
        sort(all(dag[i]));
        dag[i].erase(unique(all(dag[i])), end(dag[i]));
    }
    return dag;
} // a5cf62
```

2-SAT. `TwoSat ts(n);` `clause(a,va,b,vb)` adiciona $(x_a = \texttt{va}) \vee (x_b = \texttt{vb})$, índices 0-based. `solve()` retorna viabilidade; só então leia `val`. $O(V + E)$.

aaa6ea - graph/2-sat.cpp - 20 lines

```
struct TwoSat {
    int n;
    vector<vi> g;
    vi val;
    TwoSat(int n) : n(n), g(2 * n) {}
    void clause(int a, bool va, int b, bool vb) {
        int x = 2*a+va, y = 2*b+vb;
        g[x ^ 1].pb(y);
        g[y ^ 1].pb(x);
    } // 2f4cde
    bool solve() {
        auto [cnt, comp] = scc(g);
        val.assign(n, 0);
        rep (i, 0, n) {
            if (comp[2 * i] == comp[2 * i + 1]) return 0;
            val[i] = comp[2 * i] < comp[2 * i + 1];
        }
        return 1;
    } // a54ac0
};
```

Floresta de grafo funcional. `root[i]`: representante do ciclo/componente de `i`. $O(n)$.

988efe - graph/functional-forest.cpp - 18 lines

```
vi funcForest(vi &f) {
    vi deg(sz(f)), root(sz(f), -1), ord;
    for (int v : f) deg[v]++;
    queue<int> q;
    rep (i, 0, sz(f)) if (!deg[i]) q.push(i);
    while (sz(q)) {
        int v = q.front(); q.pop();
        ord.pb(v);
        if (!--deg[f[v]]) q.push(f[v]);
    }
    rep (i, 0, sz(f)) if (deg[i]) {
        root[i] = i;
        for (int v = f[i]; v != i; v = f[v]) root[v] = i, deg[v] = 0;
        deg[i] = 0;
    }
    for (int i = sz(ord) - 1; i >= 0; --i) root[ord[i]] = root[f[ord[i]]];
    return root;
} // 988efe
```

Dominator tree. u domina v se todo caminho de s a v passa por u. `domTree(g,s)` retorna `idom[v]`, o dominador imediato de v; `idom[s]=s`, -1 para inalcançáveis. $O((V + E)\alpha(V))$ na prática.

72f9c4 - graph/dominator-tree.cpp - 34 lines

```
vi domTree(vector<vi> &g, int s) {
    int n = sz(g), t = 0;
    vector<vi> rg(n + 1), bucket(n + 1);
    vi arr(n), rev(n + 1), par(n + 1), sdom(n + 1), dom(n + 1), dsu(n + 1), mn(n +
    ↪  1);
    auto dfs = [&](auto &&self, int u) -> void {
        arr[u] = ++t; rev[t] = u;
```

```
        sdom[t] = dom[t] = dsu[t] = mn[t] = t;
        for (int v : g[u]) {
            if (!arr[v]) self(self, v), par[arr[v]] = arr[u];
            rg[arr[v]].pb(arr[u]);
        }
    }; // 73ed34
    auto find = [&](auto &&self, int u, int x = 0) -> int {
        if (u == dsu[u]) return x ? -1 : u;
        int v = self(self, dsu[u], x + 1);
        if (v < 0) return u;
        if (sdom[mn[dsu[u]]] < sdom[mn[u]]) mn[u] = mn[dsu[u]];
        return dsu[u] = v, x ? v : mn[u];
    }; // 6abce4
    dfs(dfs, s);
    for (int i = t; i; i--) {
        for (int j : rg[i]) sdom[i] = min(sdom[i], sdom[find(find, j)]);
        if (i > 1) bucket[sdom[i]].pb(i);
        for (int j : bucket[i]) {
            int y = find(find, j);
            dom[j] = sdom[y] == sdom[j] ? sdom[j] : y;
        }
        if (i > 1) dsu[i] = par[i];
    }
    rep(i, 2, t + 1) if (dom[i] != sdom[i]) dom[i] = dom[dom[i]];
    vi idom(n, -1);
    rep(i, 1, t + 1) idom[rev[i]] = rev[dom[i]];
    return idom;
} // 72f9c4
```

Árvore geradora mínima direcionada. `dmst(n,r,e)`: custo da arborescência mínima enraizada em r, ou -1 se impossível. Arestas `E{a,b,w}`: $a \to b$ com custo w. $O(E \log(V))$.

849978 - graph/directed-mst.cpp - 65 lines

```
struct E { int a, b; ll w; };
struct H { int a; ll w, d = 0; H *l = 0, *r = 0; };
void P(H *x) {
    if (!x || !x->d) return;
    x->w += x->d;
    if (x->l) x->l->d += x->d;
    if (x->r) x->r->d += x->d;
    x->d = 0;
} // 089de9
H* M(H *a, H *b) {
    if (!a || !b) return a ? a : b;
    P(a), P(b);
    if (a->w > b->w) swap(a, b);
    a->r = M(a->r, b);
    swap(a->l, a->r);
    return a;
} // 995744
H* pop(H *x) {
    P(x);
    return M(x->l, x->r);
} // 9a6e69

ll dmst(int n, int r, vector<E> es) {
    vector<H*> h(n);
    for (auto e : es) h[e.b] = M(h[e.b], new H{e.a, e.w});
    vi uf(n), se(n, -1);
    iota(all(uf), 0);
    auto f = [&](auto &&f, int x) -> int {
        return uf[x] == x ? x : uf[x] = f(f, uf[x]);
    }; // 570c1e
    se[r] = r;
    vector<pair<int, ll>> st;
    ll ans = 0;
    rep(s,0,n) {
        int u = f(f, s);
        if (se[u] >= 0) continue;
        st.clear();
        while (se[u] < 0) {
            se[u] = s;
            while (h[u] && f(f, h[u]->a) == u) h[u] = pop(h[u]);
            if (!h[u]) return LLONG_MAX;
            P(h[u]);
            ll c = h[u]->w;
            ans += c;
            st.eb(u, c);
            u = f(f, h[u]->a);
            if (se[u] == s) {
                H *x = 0;
                int v = u;
                for (;;) {
                    auto [w, c] = st.back();
                    st.pop_back();
                    h[w]->d -= c;
```

```
                    x = M(x, h[w]);
                    uf[w] = v;
                    if (w == v) break;
                }
                u = v;
                h[u] = x;
                se[u] = -1;
            }
        }
    }
    return ans;
} // bad540
```

Matching bipartido máximo. `hopKarp(g,l,r)`: g da esquerda para a direita; inicialize `l,r` com -1. Escreve parceiros (ou -1) e retorna o tamanho do matching. $O(E\sqrt{V})$.

2d4485 - graph/hopcroft-karp.cpp - 31 lines

```
int hopKarp(vector<vi> &g, vi &l, vi &r) {
    int n = sz(l), m = sz(r);
    vi d(n);
    auto bfs = [&]() {
        queue<int> q;
        rep(i, 0, n) d[i] = l[i] < 0 ? q.push(i), 0 : -1;
        bool f = 0;
        while (sz(q)) {
            int v = q.front(); q.pop();
            for (int u : g[v]) {
                int w = r[u];
                if (w < 0) f = 1;
                else if (d[w] < 0) d[w] = d[v] + 1, q.push(w);
            }
        }
        return f;
    }; // 0d5d50
    auto dfs = [&](auto &&self, int v) -> bool {
        for (int u : g[v]) {
            int w = r[u];
            if (w < 0 || (d[w] == d[v] + 1 && self(self, w))) {
                l[v] = u, r[u] = v;
                return 1;
            }
        }
        return d[v] = -1, 0;
    }; // a3185e
    int ans = 0;
    while (bfs()) rep(i, 0, n) ans += l[i] < 0 && dfs(dfs, i);
    return ans;
} // 2d4485
```

Matching bipartido com menor custo. `hungarian(a)`: `{custo,match}`, uma coluna distinta por linha; matriz $n \times m$, $n \le m$. $O(n^2 m)$.

c8c7c0 - graph/hungarian.cpp - 33 lines

```
pair<ll, vi> hungarian(vector<vector<ll>> a) {
    int n = sz(a), m = sz(a[0]);
    const ll INF = 4e18;
    vector<ll> u(n + 1), v(m + 1), minv(m + 1);
    vi p(m + 1), way(m + 1), ans(n);
    rep(i, 1, n + 1) {
        p[0] = i;
        int j0 = 0;
        fill(all(minv), INF);
        vector<char> used(m + 1);
        do {
            used[j0] = 1;
            int i0 = p[j0], j1 = 0;
            ll delta = INF;
            rep(j, 1, m + 1) if (!used[j]) {
                ll cur = a[i0 - 1][j - 1] - u[i0] - v[j];
                if (cur < minv[j]) minv[j] = cur, way[j] = j0;
                if (minv[j] < delta) delta = minv[j], j1 = j;
            }
            rep(j, 0, m + 1)
                if (used[j]) u[p[j]] += delta, v[j] -= delta;
                else minv[j] -= delta;
            j0 = j1;
        } while (p[j0]);
        do {
            int j1 = way[j0];
            p[j0] = p[j1];
            j0 = j1;
        } while (j0);
    }
    rep(j, 1, m + 1) if (p[j]) ans[p[j] - 1] = j - 1;
    return {-v[0], ans};
} // c8c7c0
```

Encontrar matching geral. `matching(g)`: parceiros em grafo não direcionado, -1 se livre. $O(n^3)$.

35862b - graph/matching.cpp - 51 lines

```cpp
vi matching(vector<vi> &g) {
    int n = sz(g), T = 0;
    vi m(n, -1), p(n), b(n), q(n), u(n), w(n), l(n);
    auto F = [&](int x, int y) {
        for (++T;; x = p[m[x]]) {
            x = b[x];
            l[x] = T;
            if (m[x] < 0) break;
        }
        for (;; y = p[m[y]]) {
            y = b[y];
            if (l[y] == T) return y;
        }
    }; // 43ebed
    auto G = [&](int x, int c, int y) {
        for (; b[x] != c; x = p[m[x]])
            w[b[x]] = w[b[m[x]]] = 1, p[x] = y, y = m[x];
    }; // 1f6aa5
    rep(s, 0, n) if (m[s] < 0) {
        fill(all(u), 0), fill(all(p), -1), iota(all(b), 0);
        int a = 0, z = 0, x = -1;
        q[z++] = s, u[s] = 1;
        while (a < z && x < 0) {
            int v = q[a++];
            for (int y : g[v]) {
                if (b[v] == b[y] || m[v] == y)
                    continue;
                if (y == s || (~m[y] && ~p[m[y]])) {
                    int c = F(v, y);
                    fill(all(w), 0);
                    G(v, c, y);
                    G(y, c, v);
                    rep(i, 0, n) if (w[b[i]])
                        b[i] = c, !u[i] && (u[i] = 1, q[z++] = i);
                } else if (!~p[y]) {
                    p[y] = v;
                    if (!~m[y]) {
                        x = y;
                        break;
                    }
                    y = m[y], u[y] = 1, q[z++] = y;
                }
            }
        }
        while (~x) {
            int y = p[x], v = m[y];
            m[x] = y, m[y] = x, x = v;
        }
    }
    return m;
} // 35862b
```

Matching geral. `matchingSize(n,es,its=2)`: tamanho estimado por matriz de Tutte; módulo primo ímpar. **Monte Carlo** com possível subestimação; `its` repetições. Ignora self-loops. $O(\texttt{its} \cdot n^3)$ para módulo fixo.

8ce777 - graph/matching-size.cpp - 17 lines

```cpp
int matchingSize(int n, vector<pii> &es, int its = 2) {
    uniform_int_distribution<int> dist(1, mod - 1);
    int ans = 0;
    rep(it,0,its) {
        vector<vector<ll>> a(n, vector<ll>(n));
        for (auto [u, v] : es) if (u != v) {
            int x = dist(rng);
            a[u][v] += x;
            if (a[u][v] >= mod) a[u][v] -= mod;
            a[v][u] -= x;
            if (a[v][u] < 0) a[v][u] += mod;
        }
        ll det;
        ans = max(ans, gauss(a, det) / 2);
    }
    return ans;
} // 8ce777
```

Fluxo máximo (Dinic). `addEdge(u,v,cap)`: direcionada, capacidade não negativa. `flow(s,t,mx)` retorna o fluxo adicional, até `mx`; requer $s \neq t$ e mantém o residual entre chamadas. $O(V^2E)$ para fluxo máximo. O fluxo de cada aresta original está em `e[id].f`. Após fluxo máximo sem limite, vértices alcançáveis por residual positivo formam o lado da fonte de um corte mínimo.

e5f6d7 - graph/dinic.cpp - 53 lines

```cpp
struct Dinic {
    struct Edge {
```

```
    ll s, t, f, c;
};
vector<Edge> e;
vector<vi> adj;
vi ne, lvl;
Dinic(int n): adj(n), ne(n), lvl(n) {}
void addEdge(int a, int b, ll c) {
    adj[a].pb(sz(e));
    e.eb(a, b, 0, c);
    adj[b].pb(sz(e));
    e.eb(b, a, 0, 0);
} // 5cff3f
bool bfs(int s, int t) {
    vi q{s};
    fill(all(lvl), -1);
    lvl[s] = 0;
    for (int j = 0; j < sz(q); ++j) {
        int x = q[j];
        for (int i: adj[x]) {
            if (e[i].f == e[i].c) continue;
            if (lvl[e[i].t] != -1) continue;
            lvl[e[i].t] = lvl[x] + 1;
            q.pb(e[i].t);
        }
    }
    return lvl[t] == -1;
} // f66943
ll dfs(int x, int t, ll f) {
    if (f == 0) return 0;
    if (x == t) return f;
    for (int &te = ne[x]; te < sz(adj[x]); te++) {
        int i = adj[x][te], y = e[i].t;
        if (lvl[y] != lvl[x] + 1) continue;
        if (ll df = dfs(y, t, min(f, e[i].c - e[i].f))) {
            e[i].f += df;
            e[i ^ 1].f -= df;
            return df;
        }
    }
    return 0;
} // d7b05b
ll flow(int s, int t, ll mx = LLONG_MAX) {
    ll mf = 0;
    while (mf < mx) {
        if (bfs(s, t)) break;
        fill(all(ne), 0);
        while (ll f = dfs(s, t, mx - mf)) mf += f;
    }
    return mf;
} // b19820
};
```

Árvore de Gomory–Hu. Grafo não direcionado com capacidades não negativas; arestas `(u,v,c)`, paralelas permitidas. Retorna arestas da árvore: o menor peso no caminho entre u, v é o valor do corte mínimo entre eles. $n-1$ fluxos máximos; vértices desconexos ficam ligados por peso zero.

088e89 - graph/gomory-hu.cpp - 16 lines

```
vector<tuple<int,int,ll>> gomoryHu(int n, vector<tuple<int,int,ll>> es) {
    vi par(n);
    vector<tuple<int,int,ll>> tree;
    for (int i = 1; i < n; ++i) {
        Dinic g(n);
        for (auto [u,v,c] : es) if (u != v) {
            g.addEdge(u,v,c), g.addEdge(v,u,c);
        }
        ll cut = g.flow(i,par[i]);
        g.bfs(i,par[i]);
        tree.emplace_back(i,par[i],cut);
        for (int j = i + 1; j < n; ++j)
            if (par[j] == par[i] && g.lvl[j] >= 0) par[j] = i;
    }
    return tree;
} // 088e89
```

Fluxo de custo mínimo. `addEdge(u,v,cap,cost)`; `flow(s,t,k)` retorna `{fluxo,custo}` adicional até k, mantendo o residual. Requer $s \neq t$, sem self-loops e sem ciclos negativos no residual inicial; custos negativos são permitidos. $O(VE + FE \log V)$, F = número de aumentos.

e07252 - graph/mcf.cpp - 58 lines

```
struct MCMF {
    struct E { int to, rev; ll cap, cost; };
    int n;
    vector<vector<E>> g;
    MCMF(int n) : n(n), g(n) {}
    void addEdge(int u, int v, ll cap, ll cost) {
```

```
        g[u].eb(v, sz(g[v]), cap, cost);
        g[v].eb(u, sz(g[u]) - 1, 0, -cost);
    } // 82791b
    pll flow(int s, int t, ll k = 4e18) {
        const ll INF = 4e18;
        ll fl = 0, cost = 0;
        vector<ll> p(n, INF), d(n), f(n);
        vi pv(n), pe(n), inq(n);
        queue<int> q;
        p[s] = 0, q.push(s), inq[s] = 1;
        while (sz(q)) {
            int u = q.front(); q.pop();
            inq[u] = 0;
            rep(i,0,sz(g[u])) {
                auto &e = g[u][i];
                if (e.cap && p[e.to] > p[u] + e.cost) {
                    p[e.to] = p[u] + e.cost;
                    if (!inq[e.to]) q.push(e.to), inq[e.to] = 1;
                }
            }
        }
        rep(i,0,n) if (p[i] == INF) p[i] = 0;
        while (fl < k) {
            fill(all(d), INF);
            priority_queue<pll, vector<pll>, greater<pll>> pq;
            d[s] = 0, f[s] = k - fl, pq.emplace(0, s);
            while (sz(pq)) {
                auto [du, u] = pq.top(); pq.pop();
                if (du != d[u]) continue;
                rep(i,0,sz(g[u])) {
                    auto &e = g[u][i];
                    if (!e.cap) continue;
                    ll nd = du + e.cost + p[u] - p[e.to];
                    if (d[e.to] > nd) {
                        d[e.to] = nd, pv[e.to] = u, pe[e.to] = i;
                        f[e.to] = min(f[u], e.cap);
                        pq.emplace(nd, e.to);
                    }
                }
            }
            if (d[t] == INF) break;
            rep(i,0,n) if (d[i] < INF) p[i] += d[i];
            ll x = f[t];
            fl += x, cost += x * p[t];
            for (int v = t; v != s; v = pv[v]) {
                auto &e = g[pv[v]][pe[v]];
                e.cap -= x, g[v][e.rev].cap += x;
            }
        }
        return {fl, cost};
    } // e0d612
};
```

Grafo com sequência de graus. `havelHakimi(deg)`: arestas de um grafo simples com esses graus. Requer sequência gráfica válida; não detecta impossibilidade. $O(n+m)$.

f8d639 - graph/havel-hakimi.cpp - 38 lines

```
vector<pii> havelHakimi(vi deg) {
    int n = sz(deg), mx = n;
    vector<pii> ed;
    ed.reserve(accumulate(all(deg), 0LL) / 2);
    vi head(n + 1, -1), nxt(n, -1), prv(n, -1), take;
    auto rem = [&](int v) {
        int d = deg[v], p = prv[v], q = nxt[v];
        if (p == -1) head[d] = q;
        else nxt[p] = q;
        if (q != -1) prv[q] = p;
        prv[v] = nxt[v] = -1;
    }; // 93ce62
    auto ins = [&](int v) {
        int d = deg[v];
        prv[v] = -1, nxt[v] = head[d];
        if (head[d] != -1) prv[head[d]] = v;
        head[d] = v;
    }; // 21411e
    rep(i,0,n) if (deg[i]) ins(i);
    while (mx && head[mx] == -1) --mx;
    while (mx) {
        int v = head[mx];
        rem(v);
        int d = deg[v];
        take.clear(), take.reserve(d);
        int cur = mx;
        rep(_,0,d) {
            while (cur && head[cur] == -1) --cur;
            int u = head[cur]; rem(u);
```

```
            take.pb(u);
        }
        for (int u : take) ed.eb(v, u), --deg[u];
        deg[v] = 0;
        for (int u : take) if (deg[u]) ins(u);
        while (mx && head[mx] == -1) --mx;
    }
    return ed;
} // f8d639
```

Enumeração de triângulos. `triangles(g,emit)`: grafo simples não direcionado. Chama `emit(u,v,w)` uma vez por triângulo; ordem dos vértices não garantida. $O(n + m\sqrt{m})$ tempo para callback $O(1)$, $O(n + m)$ memória adicional.

1675a5 - graph/triangles.cpp - 14 lines

```
void triangles(vector<vi> &g, auto emit) {
    int n = g.size();
    vector<vi> out(n);
    vi mark(n, -1);
    for (int u = 0; u < n; ++u)
        for (int v : g[u])
            if (pair{g[u].size(),u} < pair{g[v].size(),v})
                out[u].push_back(v);
    for (int u = 0; u < n; ++u) {
        for (int v : out[u]) mark[v] = u;
        for (int v : out[u]) for (int w : out[v])
            if (mark[w] == u) emit(u,v,w);
    }
} // 1675a5
```

Coloração de arestas. `edgeColoring(n,edges)`: cor de cada aresta na ordem de entrada, em $[0, \Delta]$, onde Δ é o grau máximo. Requer grafo não direcionado simples. Não garante o mínimo de cores. $O(nm + n)$, memória $O(n(\Delta + 1))$.

45567b - graph/edge-coloring.cpp - 27 lines

```
vi edgeColoring(int n, vector<pii> edges) {
    vi cc(n+1), color(sz(edges)), fan(n), free(n), loc;
    for (auto [u,v] : edges) ++cc[u], ++cc[v];
    int u, v, colors = *max_element(all(cc))+1;
    vector<vi> adj(n,vi(colors,-1));
    for (auto edge : edges) {
        tie(u,v) = edge; fan[0] = v;
        loc.assign(colors,0);
        int at = u, end = u, d, c = free[u], ind = 0, i = 0;
        while (d = free[v], !loc[d] && (v = adj[u][d]) != -1)
            loc[d] = ++ind, cc[ind] = d, fan[ind] = v;
        cc[loc[d]] = c;
        for (int cd = d; at != -1; cd ^= c^d, at = adj[at][cd])
            swap(adj[at][cd],adj[end = at][cd^c^d]);
        while (adj[fan[i]][d] != -1) {
            int left = fan[i], right = fan[++i], e = cc[i];
            adj[u][e] = left; adj[left][e] = u;
            adj[right][e] = -1; free[right] = e;
        }
        adj[u][d] = fan[i]; adj[fan[i]][d] = u;
        for (int y : {fan[0],u,end})
            for (int &z = free[y] = 0; adj[y][z] != -1; ++z);
    }
    rep(i,0,sz(edges))
        for (tie(u,v) = edges[i]; adj[u][color[i]] != v; ++color[i]);
    return color;
} // 45567b
```

Árvores

Diâmetro de uma árvore. `treeDiam(n,g)` retorna a sequência de vértices no diâmetro da árvore. $O(n)$.

897b26 - tree/tree-diameter.cpp - 14 lines

```
vi treeDiam(int n, vector<vi> &g) {
    int k;
    vi d(n), t(n), s(n), p(n), dia;
    rep(a,0,2) {
        k = 1, p[t[0]] = -1, s.assign(n, 0);
        for (int i: t) {
            s[i] = 1;
            for (int j: g[i]) if (!s[j]) d[j] = d[i] + 1, t[k++] = j, p[j] = i;
        }
        t[0] = max_element(all(d)) - begin(d);
    }
    for (int i = t[0]; i != -1; i = p[i]) dia.pb(i);
    return dia;
} // 897b26
```

Matching em árvores. Parceiro de cada vértice em um matching máximo; `-1` se livre. $O(n)$.

6e0886 - tree/tree-matching.cpp - 17 lines

```
vi treeMatching(vector<vi> &g, int root = 0) {
    int n = sz(g);
    vi par(n, -1), ord{root}, mat(n, -1);
    rep(i,0,sz(ord)) {
        int v = ord[i];
        for (int u : g[v]) if (u != par[v]) {
            par[u] = v;
            ord.pb(u);
        }
    }
    for (int i = n - 1; i > 0; --i) {
        int v = ord[i], p = par[v];
        if (mat[v] == -1 && mat[p] == -1)
            mat[v] = p, mat[p] = v;
    }
    return mat;
} // 6e0886
```

Isomorfismo de subárvores. `TreeIso t`; `t.classify(g,r=0)`: classes exatas de isomorfismo de subárvores não ordenadas e sem rótulos, sob a raiz r; árvore não vazia. Reutilize a mesma instância para comparar árvores diferentes. Para n vértices e K classes já registradas: $O(n \log(n+K))$ tempo, memória proporcional às assinaturas armazenadas mais $O(n)$.

b78b52 - tree/isomorphism.cpp - 18 lines

```
struct TreeIso {
    map<vi,int> ids;
    vi classify(vector<vi> &g, int r = 0) {
        int n = g.size();
        vi p(n, -1), ord{r}, ans(n);
        for (int i = 0; i < n; ++i)
            for (int v : g[ord[i]]) if (v != p[ord[i]])
                p[v] = ord[i], ord.push_back(v);
        for (int i = n-1; i >= 0; --i) {
            int u = ord[i];
            vi a;
            for (int v : g[u]) if (v != p[u]) a.push_back(ans[v]);
            sort(a.begin(),a.end());
            ans[u] = ids.emplace(move(a),ids.size()).first->second;
        }
        return ans;
    } // 7c6f56
};
```

LCA. `LCA lca(g,root)`, `lca.lca(a,b)`. Construção $O(n \log n)$; consulta $O(1)$.

1d44d9 - tree/lca.cpp - 23 lines

```
using RMQ = SparseTable<int, [](int a, int b) { return min(a, b); }>;

struct LCA {
    int t = 0;
    vi tm, path, ret;
    RMQ rmq;
    LCA(vector<vi> &g, int r = 0) : tm(g.size()) {
        dfs(g, r, -1);
        rmq.build(ret);
    }
    void dfs(vector<vi> &g, int x, int pre) {
        tm[x] = t++;
        for (int y: g[x]) if (y != pre) {
            path.pb(x), ret.pb(tm[x]);
            dfs(g, y, x);
        }
    } // 2e7ca5
    int lca(int x, int y) {
        if (x == y) return x;
        tie(x, y) = minmax(tm[x], tm[y]);
        return path[rmq.query(x, y)];
    } // e8105a
};
```

HLD. `HLD(g,r)`: árvore não vazia. Subárvore em $[\texttt{tin}[u], \texttt{tout}[u])$; `path(u,v)` retorna pares de extremos **inclusivos**, na ordem $u \to v$ (podem ser decrescentes). Construção $O(n)$; $O(\log n)$ intervalos por caminho.

c869f5 - tree/hld.cpp - 39 lines

```
struct HLD {
    vi cnt, nxt, par, tin, tout, d;
    int T = 0;
    HLD(vector<vi> g, int r = 0) :
        cnt(sz(g)), nxt(sz(g)), par(sz(g), -1),
        tin(sz(g)), tout(sz(g)), d(sz(g)) {
        auto dfs1 = [&](auto &&self, int u) -> void {
            cnt[u] = 1, nxt[u] = u;
            if (par[u] != -1) g[u].erase(find(all(g[u]), par[u]));
            for (int &v: g[u]) {
```

```
            par[v] = u, d[v] = d[u] + 1;
            self(self, v);
            cnt[u] += cnt[v];
            if (cnt[v] > cnt[g[u][0]]) swap(v, g[u][0]);
        }
    }; // c30f49
    auto dfs2 = [&](auto &&self, int u) -> void {
        tin[u] = T++;
        for (int v: g[u]) {
            if (v == g[u][0]) nxt[v] = nxt[u];
            self(self, v);
        }
        tout[u] = T;
    }; // f292ec
    dfs1(dfs1, r), dfs2(dfs2, r);
}
vector<pii> path(int u, int v) {
    vector<pii> a, b;
    while (nxt[u] != nxt[v]) {
        if (d[nxt[u]] < d[nxt[v]])
            b.eb(tin[nxt[v]], tin[v]), v = par[nxt[v]];
        else
            a.eb(tin[u], tin[nxt[u]]), u = par[nxt[u]];
    }
    (d[u] < d[v] ? b : a).eb(tin[u], tin[v]);
    a.insert(end(a), rbegin(b), rend(b));
    return a;
} // 32f9ea
};
```

Árvore virtual. `virtree(...,v)`: arestas `{pai,filho}`. `v` vira o fechamento por LCA, ordenado por `tin` e sem duplicatas; `tin,tout` da mesma raiz, subárvore [`tin`, `tout`). Para k vértices e LCA $O(1)$: $O(k \log k)$.

57ab2b - tree/virtual-tree.cpp - 17 lines

```
vector<pii> virtree(vector<vi> &t, LCA &lca, vi &tin, vi &tout, vi &v) {
    if (v.empty()) return {};
    auto cmp = [&](int a, int b) { return tin[a] < tin[b]; };
    sort(all(v), cmp);
    int m = sz(v);
    rep(i,0,m-1) v.pb(lca.lca(v[i], v[i+1]));
    sort(all(v), cmp);
    v.erase(unique(all(v)), v.end());
    vector<pii> e;
    vi st = {v[0]};
    rep(i,1,sz(v)) {
        while (tout[st.back()] <= tin[v[i]]) st.pop_back();
        e.eb(st.back(), v[i]);
        st.pb(v[i]);
    }
    return e;
} // 57ab2b
```

Árvore de reconstrução de Kruskal. `KruskalTree(n,e)`, arestas `(peso,u,v)` não orientadas. Folhas $[0, n)$; campos: `ch` filhos, `w` peso, `cnt` número de folhas, `p` pai; raízes em `roots`, pai -1. Para folhas distintas conexas, peso do LCA = menor limiar que as conecta. Floresta desconexa permitida. $O(n + m \log m)$, até $2n - 1$ nós.

590540 - tree/kruskal-tree.cpp - 23 lines

```
struct KruskalTree {
    vi p, cnt, roots;
    vector<array<int,2>> ch;
    vector<ll> w;
    KruskalTree(int n, vector<tuple<ll,int,int>> e)
        : p(n,-1), cnt(n,1), ch(n,{-1,-1}), w(n) {
        vi d(n); iota(all(d),0);
        auto f = [&](int u) {
            while (u != d[u]) u = d[u] = d[d[u]];
            return u;
        }; // 76ce70
        sort(all(e));
        for (auto [c,u,v] : e) {
            u = f(u); v = f(v); if (u == v) continue;
            int k = sz(p);
            p[u] = p[v] = k; p.pb(-1);
            ch.pb({u,v}); w.pb(c);
            cnt.pb(cnt[u]+cnt[v]);
            d[u] = d[v] = k; d.pb(k);
        }
        rep(i,0,sz(p)) if (p[i] < 0) roots.pb(i);
    }
};
```

Decomposição em árvore de centróides. `centroidDecomp(g)`: pais na árvore de centróides, raiz com -1. Requer árvore não vazia. $O(n \log n)$.

98b5c8 - tree/centroid-decomp.cpp - 22 lines

```cpp
vi centroidDecomp(vector<vi> &g) {
    int n = sz(g);
    vi par(n, -1), sub(n), dead(n);
    auto dfs = [&](auto &&self, int v, int p) -> int {
        sub[v] = 1;
        for (int u : g[v]) if (u != p && !dead[u])
            sub[v] += self(self, u, v);
        return sub[v];
    }; // 95667b
    auto cen = [&](auto &&self, int v, int p, int m) -> int {
        for (int u : g[v]) if (u != p && !dead[u] && sub[u] > m / 2)
            return self(self, u, v, m);
        return v;
    }; // c873ed
    auto build = [&](auto &&self, int v, int p) -> void {
        int m = dfs(dfs, v, -1), c = cen(cen, v, -1, m);
        par[c] = p, dead[c] = 1;
        for (int u : g[c]) if (!dead[u]) self(self, u, c);
    }; // 6a8e97
    build(build, 0, -1);
    return par;
} // 98b5c8
```

Histograma de distâncias em árvore. `treeDistHist(g)`: `ans[d]` conta pares não ordenados de vértices distintos à distância `d`. Implemente `conv(vector<ll>, vector<ll>)` usando fft com `double`. $O(n \log^2 n)$.

bfa638 - tree/dist-hist.cpp - 42 lines

```cpp
vector<ll> treeDistHist(vector<vi>& g) {
    int n = sz(g);
    vi sub(n), dead(n);
    vector<ll> ans(n);
    auto solve = [&](auto&& self, int s) -> void {
        auto dfs = [&](auto&& dfs, int u, int p) -> int {
            sub[u] = 1;
            for (int v : g[u]) if (v != p && !dead[v])
                sub[u] += dfs(dfs, v, u);
            return sub[u];
        }; // 067880
        int N = dfs(dfs, s, -1), c = s, p = -1;
        while (1) {
            int v = -1;
            for (int x : g[c]) if (x != p && !dead[x] && sub[x] > N / 2)
                v = x;
            if (v == -1) break;
            p = c, c = v;
        }
        vector<ll> a(1, 1);
        for (int v : g[c]) if (!dead[v]) {
            vector<ll> b(1);
            auto dep = [&](auto&& dep, int u, int p, int d) -> void {
                if (sz(b) <= d) b.resize(d + 1);
                b[d]++;
                for (int v : g[u]) if (v != p && !dead[v]) dep(dep, v, u, d + 1);
            }; // 105348
            dep(dep, v, c, 1);
            if (sz(a) < sz(b)) a.resize(sz(b));
            rep(i, 1, sz(b)) a[i] += b[i];
            auto q = conv(b, b);
            rep(i, 1, min(n, sz(q))) ans[i] -= q[i];
        }
        auto q = conv(a, a);
        rep(i, 1, min(n, sz(q))) ans[i] += q[i];
        dead[c] = 1;
        for (int v : g[c]) if (!dead[v]) self(self, v);
    }; // 8f8432
    solve(solve, 0);
    rep(i, 1, n) ans[i] /= 2;
    return ans;
} // bfa638
```

Link-cut tree. `LinkCut<S,op,e> t(n)`: floresta sem arestas, vértices **1 a n**. Monoide como na segtree. `set(u,x)`; `query(u,v)` agrega o caminho na ordem $u \to v$, incluindo ambos os extremos, e requer conexão; aceita operação não comutativa. `link/cut(u,v)`: falso se criaria ciclo/aresta ausente. Operações $O(\log n)$ amortizado com `op` e cópias $O(1)$; memória $O(n)$. Agrega apenas caminhos, não subárvores.

cf9cc7 - tree/link-cut.cpp - 70 lines

```cpp
template<class S, S (*op)(S, S), S (*e)()>
struct LinkCut {
    struct Node { int c[2] = {}, p = 0; bool rev = 0;
        S val = e(), fwd = e(), bwd = e(); };
    vector<Node> t;
    LinkCut(int n) : t(n+1) {}
    bool auxRoot(int x) {
        int p = t[x].p;
```

```
    return !p || (t[p].c[0] != x && t[p].c[1] != x);
} // a99fc7
void pull(int x) {
    auto &a = t[t[x].c[0]], &b = t[t[x].c[1]];
    t[x].fwd = op(op(a.fwd,t[x].val),b.fwd);
    t[x].bwd = op(op(b.bwd,t[x].val),a.bwd);
} // 3d98f0
void flip(int x) {
    if (x) {
        swap(t[x].c[0],t[x].c[1]);
        swap(t[x].fwd,t[x].bwd); t[x].rev ^= 1;
    }
} // 6aa904
void push(int x) {
    if (t[x].rev) flip(t[x].c[0]), flip(t[x].c[1]), t[x].rev = 0;
} // a7444f
void rotate(int x) {
    int p = t[x].p, g = t[p].p, d = t[p].c[1] == x;
    int b = t[x].c[d^1];
    if (!auxRoot(p)) t[g].c[t[g].c[1] == p] = x;
    t[x].p = g; t[x].c[d^1] = p; t[p].p = x;
    t[p].c[d] = b; if (b) t[b].p = p;
    pull(p); pull(x);
} // b293c3
void splay(int x) {
    vector<int> path{x};
    for (int y = x; !auxRoot(y); ) path.push_back(y = t[y].p);
    for (auto it = path.rbegin(); it != path.rend(); ++it) push(*it);
    while (!auxRoot(x)) {
        int p = t[x].p, g = t[p].p;
        if (!auxRoot(p)) rotate((t[p].c[1] == x) ==
            (t[g].c[1] == p) ? p : x);
        rotate(x);
    }
} // bf3fc3
void access(int x) {
    for (int y = x, last = 0; y; last = y, y = t[y].p) {
        splay(y); t[y].c[1] = last; pull(y);
    }
    splay(x);
} // 5a956c
void makeRoot(int x) { access(x); flip(x); }
int root(int x) {
    access(x); push(x);
    while (t[x].c[0]) x = t[x].c[0], push(x);
    splay(x); return x;
} // 398918
bool connected(int u, int v) { return root(u) == root(v); }
bool link(int u, int v) {
    makeRoot(u); if (root(v) == u) return false;
    t[u].p = v; return true;
} // 67d999
bool cut(int u, int v) {
    makeRoot(u); access(v);
    if (t[v].c[0] != u || t[u].c[1]) return false;
    t[v].c[0] = t[u].p = 0; pull(v); return true;
} // 8455bd
void set(int u, S val) { access(u); t[u].val = val; pull(u); }
S query(int u, int v) {
    makeRoot(u); access(v); return t[v].fwd;
} // ae1361
};
```

Strings

Hashing. `H`: `H()` é vazio/neutro; `H(x)` tem um símbolo. `a.concat(b)` retorna ab sem alterar os operandos; igualdade compara hash e comprimento. Base aleatória compartilhada, módulo $2^{61}-1$; colisões possíveis. Concatenação $O(1)$; cada hash guarda $\texttt{pw} = \texttt{base}^{\texttt{cnt}}$.

ddb4d3 - strings/hashing.cpp - 13 lines

```
struct H {
    static constexpr ll M = (1LL<<61)-1;
    inline static const ll base = 256+mt19937_64(random_device{}())()%(M-257);
    ll h = 0, pw = 1;
    int cnt = 0;
    H() = default;
    H(ll x, int n=1, ll p=base): h((x%M+M)%M), pw(p), cnt(n) {}
    static ll mul(ll a, ll b) { return (__int128)a*b%M; }
    H concat(H b) const {
        return {mul(h,b.pw)+b.h, cnt+b.cnt, mul(pw,b.pw)};
    } // 14f320
    bool operator==(const H&) const = default;
};
```

KMP. `kmp(s)` retorna a prefix function em $O(n)$.

cb52b3 - strings/kmp.cpp - 10 lines

```
vi kmp(string s) {
    int n = sz(s), len = 0;
    vi pre(n);
    rep (i, 1, n) {
        while (len > 0 && s[i] != s[len]) len = pre[len - 1];
        if (s[i] == s[len]) len++;
        pre[i] = len;
    }
    return pre;
} // cb52b3
```

Z-function. `zFunc(s)`: `z[i]` = LCP de `s` com `s[i..n)`; `z[0]=n`. Aceita string vazia. $O(n)$.

fc69f3 - strings/z-function.cpp - 12 lines

```
vi zFunc(string s) {
    int n = sz(s), l = 0, r = 0;
    vi z(n);
    if (!n) return z;
    rep(i, 1, n) {
        if (i < r) z[i] = min(r - i, z[i - l]);
        while (i + z[i] < n && s[z[i]] == s[i + z[i]]) z[i]++;
        if (i + z[i] > r) l = i, r = i + z[i];
    }
    z[0] = n;
    return z;
} // fc69f3
```

Aho-Corasick. `AC`: adicione padrões `a-z` com `add`; chame `build` uma vez, depois `next(v,c)`. `add` retorna nó terminal; ocorrências exigem propagação pelos links. Construção $O(26|\texttt{AC}|)$; percurso $O(|\texttt{texto}|)$.

e20924 - strings/aho-corasick.cpp - 36 lines

```
struct AC {
    static constexpr int A = 26;
    struct N {
        array<int, A> to{};
        int link = 0;
        N() { to.fill(-1); }
    };
    vector<N> t{{}};
    vi ord;
    int add(string& s) {
        int v = 0;
        for (char c : s) {
            int x = c - 'a';
            if (t[v].to[x] == -1) t[v].to[x] = sz(t), t.eb();
            v = t[v].to[x];
        }
        return v;
    } // bd83ba
    void build() {
        ord.clear();
        rep(c,0,A) {
            int u = t[0].to[c];
            if (u == -1) t[0].to[c] = 0;
            else ord.pb(u);
        }
        for (int i = 0; i < sz(ord); ++i) {
            int v = ord[i];
            rep(c,0,A) {
                int& u = t[v].to[c];
                if (u == -1) u = t[t[v].link].to[c];
                else t[u].link = t[t[v].link].to[c], ord.pb(u);
            }
        }
    } // 341c62
    int next(int v, char c) { return t[v].to[c - 'a']; }
};
```

Array de sufixos e LCP. `sufArr(s)` retorna `{sa,lcp}`: sem sufixo vazio; `lcp[i]` entre `sa[i-1]` e `sa[i]`, `lcp[0]=0`. Requer $n > 0$; $O(n \log n)$.

4e3b00 - strings/suffix-array.cpp - 35 lines

```
pair<vi,vi> sufArr(string &s) { // Could also be vector<T> &s
    int n = sz(s), k = 0;
    vi c(n), d(n), e(n), sb(n), sa(n), cnt(n + 1);
    iota(all(sa), 0);
    sort(all(sa), [&](int i, int j) { return s[i] < s[j]; });
    c[sa[0]] = 1;
    for (int i = 1; i < n; i++)
        c[sa[i]] = c[sa[i - 1]] + (s[sa[i]] != s[sa[i - 1]]);
    for (int k = 1; c[sa[n - 1]] != n; k <<= 1) {
        rep (i, 0, n) d[i] = i + k < n ? c[i + k] : 0;
        auto srt = [&](auto &C) {
            fill(all(cnt), 0);
            rep (i, 0, n) cnt[C[i] + 1]++;
```

```
            rep (i, 0, n) cnt[i + 1] += cnt[i];
            for (int i: sa) sb[cnt[C[i]]++] = i;
            swap(sa, sb);
        }; // 3af78c
        srt(d); srt(c);
        e[sa[0]] = 1;
        rep(i,1,n) {
            int a = sa[i-1], b = sa[i];
            e[b] = e[a] + (c[a] != c[b] || d[a] != d[b]);
        }
        swap(c, e);
    }
    d[0] = 0;
    rep(i,0,n) {
        if (c[i] == 1) { k = 0; continue; }
        int j = sa[c[i] - 2];
        while (i + k < n && j + k < n && s[i + k] == s[j + k]) k++;
        d[c[i] - 1] = k;
        if (k) k--;
    }
    return {sa, d};
} // 4e3b00
```

Árvore de sufixos. `ST(s,sa,lcp)`: árvore de sufixos em $O(n)$. Nós em `t`: `[l,r)` rotula a aresta do pai `p`; `dep` guarda profundidade em caracteres. `suf>=0` marca o fim do sufixo iniciado em `suf`; sem terminador único, pode ocorrer em nó interno.

31e155 - strings/suffix-tree.cpp - 31 lines

```
struct ST {
    struct N {
        int l, r, p, suf;
        vi ch;
    };
    vector<N> t;
    vi dep;
    int nn(int l, int r, int p, int suf = -1) {
        t.pb({l, r, p, suf, {}});
        dep.pb(p == -1 ? 0 : dep[p] + r - l);
        if (p != -1) t[p].ch.pb(sz(t) - 1);
        return sz(t) - 1;
    } // aa5bfe
    ST(string& s, vi& sa, vi& lcp) {
        int n = sz(s), rt = nn(0, 0, -1), last = rt;
        rep(i,0,n) {
            int x = sa[i], k = i ? lcp[i] : 0;
            while (dep[last] > k) last = t[last].p;
            if (dep[last] < k) {
                int y = t[last].ch.back(); t[last].ch.pop_back();
                int z = nn(t[y].l, t[y].l + k - dep[last], last);
                t[y].p = z;
                t[y].l += k - dep[last];
                t[z].ch.pb(y);
                dep[z] = k;
                last = z;
            }
            last = nn(x + k, n, last, x);
        }
    }
};
```

Autômato de sufixos. `SAM sam`; `extend(c)` para letras `a-z`. `ord()` ordena estados por `len` crescente; propague `cnt` pelos links em ordem inversa. `t[v].end`: fim 0-based de uma ocorrência; comprimento L dá intervalo $[\texttt{end} - L + 1, \texttt{end} + 1)$. Construção $O(n)$.

73ae80 - strings/suffix-automaton.cpp - 40 lines

```
struct SAM {
    struct N {
        int len = 0, link = -1, end = -1;
        array<int, 26> to;
        ll cnt = 0; // propagate to links in reverse ord()
        N() { to.fill(-1); }
    };
    vector<N> t{{}};
    int last = 0;
    void extend(char c) {
        int x = c - 'a', cur = sz(t), p = last;
        t.eb();
        t[cur].len = t[last].len + 1;
        t[cur].cnt = 1; t[cur].end = t[cur].len - 1;
        for (; p != -1 && t[p].to[x] == -1; p = t[p].link) t[p].to[x] = cur;
        if (p == -1) t[cur].link = 0;
        else {
            int q = t[p].to[x];
            if (t[p].len + 1 == t[q].len) t[cur].link = q;
            else {
                int cl = sz(t);
```

```
                t.pb(t[q]);
                t[cl].len = t[p].len + 1;
                t[cl].cnt = 0;
                for (; p != -1 && t[p].to[x] == q; p = t[p].link) t[p].to[x] = cl;
                t[q].link = t[cur].link = cl;
            }
        }
        last = cur;
    } // c46433
    vi ord() {
        int mx = 0;
        for (auto &u : t) mx = max(mx, u.len);
        vi cnt(mx + 1), ord(sz(t));
        for (auto &u : t) cnt[u.len]++;
        rep(i,1,mx+1) cnt[i] += cnt[i - 1];
        for (int i = sz(t) - 1; i >= 0; i--) ord[--cnt[t[i].len]] = i;
        return ord;
    } // a8d426
};
```

Maior substring comum. `lcs(a,b)`: letras `a-z`; $O(|a| + |b|)$.

9de7c0 - strings/longest-common-substring.cpp - 16 lines

```
string lcs(string a, string b) {
    SAM sam;
    for (char c: a) sam.extend(c);
    int v = 0, l = 0, best = 0, pos = 0;
    rep(i,0,sz(b)) {
        int c = b[i] - 'a';
        while (v && sam.t[v].to[c] == -1) {
            v = sam.t[v].link;
            l = sam.t[v].len;
        }
        if (sam.t[v].to[c] != -1) v = sam.t[v].to[c], l++;
        else l = 0;
        if (l > best) best = l, pos = i;
    }
    return b.substr(pos - best + 1, best);
} // 9de7c0
```

Manacher. `manacher(s)`: `{d1,d2}`. Comprimentos `2d1`$[i] - 1$ (centro i) e `2d2`$[i]$ (centro entre $i-1, i$). $O(n)$.

59be6a - strings/manacher.cpp - 17 lines

```
pair<vi, vi> manacher(string& s) {
    int n = sz(s);
    vi d1(n), d2(n);
    for (int i = 0, l = 0, r = -1; i < n; i++) {
        int k = i > r ? 1 : min(d1[l + r - i], r - i + 1);
        while (i - k >= 0 && i + k < n && s[i - k] == s[i + k]) k++;
        d1[i] = k--;
        if (i + k > r) l = i - k, r = i + k;
    }
    for (int i = 0, l = 0, r = -1; i < n; i++) {
        int k = i > r ? 0 : min(d2[l + r - i + 1], r - i + 1);
        while (i - k - 1 >= 0 && i + k < n && s[i - k - 1] == s[i + k]) k++;
        d2[i] = k--;
        if (i + k > r) l = i - k - 1, r = i + k;
    }
    return {d1, d2}; // {odd, even}
} // 59be6a
```

Árvore de palíndromos (eertree). `add(c)` acrescenta uma letra `a-z` e retorna o nó do maior sufixo palíndromo atual. Nós $v \geq 2$ representam palíndromos distintos; 0 e 1 são raízes de comprimentos -1 e 0. `link`: maior sufixo palíndromo próprio; `end`: fim de uma ocorrência, 0-based. `occurrences()[v]` conta todas as ocorrências, inclusive sobrepostas; contagem não destrutiva. `cnt` conta apenas vezes como maior sufixo. Construção $O(n)$ para alfabeto fixo; contagem $O(n)$, memória $O(26n)$.

2b833c - strings/palindromic-tree.cpp - 31 lines

```
struct Eertree {
    struct Node {
        int len, link = 0, cnt = 0, end = -1;
        array<int,26> to{};
        Node(int len): len(len) {}
    };
    vector<Node> t = {Node(-1),Node(0)};
    vi s{-1}; int last = 1;
    int getLink(int v,int c) {
        while (s[sz(s)-2-t[v].len] != c) v = t[v].link;
        return v;
    } // 31d1f0
    int add(char ch) {
        int c = ch-'a'; s.push_back(c);
        int v = getLink(last,c);
        if (!t[v].to[c]) {
            int u = sz(t); t.emplace_back(t[v].len+2);
```

```
            t[u].link = t[u].len == 1 ? 1 : t[getLink(t[v].link,c)].to[c];
            t[u].end = sz(s)-2;
            t[v].to[c] = u;
        }
        last = t[v].to[c]; ++t[last].cnt;
        return last;
    } // efdd12
    vi occurrences() const {
        vi count;
        for (auto &v : t) count.push_back(v.cnt);
        for (int v = sz(t)-1; v >= 2; --v) count[t[v].link] += count[v];
        return count;
    } // 49c36a
};
```

Séries de sufixos palíndromos. Para estender a eertree, mantenha $d[v] = \texttt{len}[v] - \texttt{len}[\texttt{link}[v]]$ nos nós não vazios. Agrupe nós consecutivos de mesmo d na cadeia de suffix links em séries aritméticas: há $O(\log n)$ por prefixo. O *series link* pula o grupo; com $u = \texttt{link}[v]$,

$$\texttt{sl}[v] = \begin{cases} \texttt{sl}[u], & d[v] = d[u], \\ u, & d[v] \neq d[u]. \end{cases}$$

Inicialize $d = 0$ e $\texttt{sl} = 1$ nas duas raízes do snippet; calcule os campos ao criar cada nó. Parta de `last` e siga `sl` enquanto $\texttt{len}[v] > 0$: a série de v tem comprimentos de $\texttt{len}[v]$ até $\texttt{len}[\texttt{sl}[v]] + d[v]$, em passos de $-d[v]$. Para acelerar DP de partição em palíndromos, agregue as transições de cada série; percorrer todos seus nós ainda pode custar $O(n)$ por prefixo.

Decomposição de Lyndon. `lyndon(s)`: fronteiras $p = (0, \ldots, n)$ da fatoração; fatores $s[p_i, p_{i+1})$ em ordem lexicográfica não crescente. String vazia retorna `{0}`. $O(n)$.

751105 - strings/lyndon.cpp - 14 lines

```
vi lyndon(string s) {
    int n = sz(s);
    vi p = {0};
    for (int i = 0; i < n;) {
        int j = i + 1, k = i;
        while (j < n && s[k] <= s[j]) {
            if (s[k] < s[j]) k = i;
            else k++;
            j++;
        }
        while (i <= k) p.pb(i += j - k);
    }
    return p;
} // 751105
```

Encontrar runs em uma string. `getRuns(s)`: triplas `{p,l,r}` de repetições maximais em $[l, r)$, período mínimo p, $r - l \geq 2p$. $O(n \log^2 n)$ nesta implementação.

934209 - strings/runs.cpp - 38 lines

```
using tiii = tuple<int, int, int>;

vector<tiii> getRuns(string &S) {
    int N = sz(S);
    vector<vector<pii>> B(N + 1);
    auto go = [&](string &l, string &r, int M, int o) {
        int n = sz(l), m = sz(r);
        string s = l, t = r + l + r;
        reverse(all(s));
        auto z = zFunc(s), w = zFunc(t);
        rep(p,1,n+1) {
            int a = p == n ? n : min(p + z[p], n), b = min(w[n + m - p], m);
            if (a + b >= 2 * p) B[p].eb(M - (o ? b : a), M + (o ? a : b));
        }
    }; // 9dd4a1
    auto dfs = [&](auto &&self, int L, int R) -> void {
        if (R - L < 2) return;
        int M = (L + R) >> 1;
        self(self, L, M), self(self, M, R);
        string l(begin(S) + L, begin(S) + M), r(begin(S) + M, begin(S) + R);
        go(l, r, M, 0);
        reverse(all(l)), reverse(all(r));
        go(r, l, M, 1);
    }; // dea6cf
    dfs(dfs, 0, N);
    vector<tiii> res;
    set<pii> done;
    rep(p,0,N+1) {
        auto &v = B[p];
        sort(all(v), [](pii a, pii b) { return a.fi == b.fi ? a.se > b.se : a.fi <
        ↪  b.fi; });
        int R = -1;
        for (auto [l, r] : v) if (R < r) {
```

```
            R = r;
            if (done.emplace(l, r).se) res.eb(p, l, r);
        }
    }
    return res;
} // d4dc26
```

Geometria

Base. `Pt=complex<ll>`; produtos devem caber em `ll`. Inteiros: `EPS=0`. Nas entradas marcadas *Reais*, troque `Pt`, `dot` e `cross` para `ld`.

673f55 - geometry/core.cpp - 9 lines

```
using Pt = complex<ll>;
#define xx real()
#define yy imag()

ll dot(Pt a, Pt b) { return (conj(a) * b).xx; }
ll cross(Pt a, Pt b) { return (conj(a) * b).yy; }
Pt perp(Pt a) { return Pt(-a.yy, a.xx); }
const ld EPS = 1e-9;
int sgn(ld x) { return (x > EPS) - (x < -EPS); }
```

Interseção de retas. `lineInter`: Reais; retas infinitas ab, cd, não paralelas e não degeneradas.

2fc473 - geometry/line-intersection.cpp - 3 lines

```
Pt lineInter(Pt a, Pt b, Pt c, Pt d) {
    return a + (b - a) * cross(c - a, d - c) / cross(b - a, d - c);
} // 2fc473
```

Interseção de segmentos. `segInter` inclui extremos; `properInter` exige cruzamento estrito.

f6de72 - geometry/segment-intersection.cpp - 16 lines

```
bool segInter(Pt a, Pt b, Pt c, Pt d) {
    int ab_c = sgn(cross(b - a, c - a)), ab_d = sgn(cross(b - a, d - a));
    int cd_a = sgn(cross(d - c, a - c)), cd_b = sgn(cross(d - c, b - c));
    if (ab_c * ab_d < 0 && cd_a * cd_b < 0) return 1;
    if (!ab_c && sgn(dot(c - a, c - b)) <= 0) return 1;
    if (!ab_d && sgn(dot(d - a, d - b)) <= 0) return 1;
    if (!cd_a && sgn(dot(a - c, a - d)) <= 0) return 1;
    if (!cd_b && sgn(dot(b - c, b - d)) <= 0) return 1;
    return 0;
} // 4f106a

bool properInter(Pt a, Pt b, Pt c, Pt d) {
    int ab_c = sgn(cross(b - a, c - a)), ab_d = sgn(cross(b - a, d - a));
    int cd_a = sgn(cross(d - c, a - c)), cd_b = sgn(cross(d - c, b - c));
    return ab_c * ab_d < 0 && cd_a * cd_b < 0;
} // 66a13a
```

Área orientada. `polyArea`: dobro da área orientada; positivo em sentido anti-horário. $O(n)$.

b23e06 - geometry/polygon-area.cpp - 5 lines

```
ll polyArea(vector<Pt> &pt) { // ld: change ret type to ld
    ll n = sz(pt), s = 0; // ld: change to ld s
    rep (i, 0, n) s += cross(pt[i], pt[(i + 1) % n]);
    return s;
} // b23e06
```

Convex hull. `convHull`: índices do hull anti-horário; exclui colineares nas arestas. Remova duplicatas e trate $n \leq 1$ fora. $O(n \log n)$.

b43fd9 - geometry/convex-hull.cpp - 22 lines

```
vi convHull(vector<Pt> &pt) {
    int n = sz(pt), m;
    vi h, ord(n);
    auto add = [&]() {
        vi st;
        for (int i: ord) {
            while ((m = sz(st)) > 1) {
                Pt a = pt[st[m - 1]], b = pt[st[m - 2]], c = pt[i];
                if (cross(b - a, c - a) < 0) break; // 1) > for clockwise, <= to
                ↪ include non-vertices
                st.pop_back();
            }
            st.pb(i);
        }
        st.pop_back();
        h.insert(h.end(), all(st));
    }; // 1938a0
    iota(all(ord), 0);
    auto top = [](auto a) { return pair(a.xx, a.yy); };
    sort(all(ord), [&](int i, int j) { return top(pt[i]) > top(pt[j]); });
    add(), reverse(all(ord)), add();
    return h;
```

```
} // b43fd9 // 1) ld: use sgn(cross(...)) < 0
```

Ponto em convexo. `pointInPoly`: convexo anti-horário, $n \geq 3$, sem repetir o primeiro vértice nem manter colineares consecutivos. `border=true` inclui a borda. $O(\log n)$.

1ce015 - geometry/point-in-polygon.cpp - 10 lines

```
bool pointInPoly(vector<Pt> &pt, Pt q, bool border = true) {
    int n = pt.size(), t = border, l = 1, r = n - 1; // ld: ld t = border ? EPS : 0
    auto v = [&](int i) { return cross(q - pt[0], pt[i] - pt[0]); };
    if (v(l) >= t || v(r) <= -t) return 0;
    while (r - l > 1) {
        int m = (l + r) / 2;
        (v(m) > 0 ? r : l) = m; // ld: use v(m) > EPS
    }
    return cross(pt[r] - pt[l], pt[l] - q) < t; // ld: use the ld t above
} // 1ce015
```

Corte de polígono. `polyCut`: parte do polígono à esquerda da reta orientada $a \to b$, incluindo a borda; $a \neq b$. Reais. $O(n)$.

a60a48 - geometry/polygon-cut.cpp - 10 lines

```
vector<Pt> polyCut(vector<Pt> p, Pt a, Pt b) {
    vector<Pt> q;
    rep(i,0,sz(p)) {
        Pt c = p[i], d = p[(i + 1) % sz(p)];
        int sc = sgn(cross(b - a, c - a)), sd = sgn(cross(b - a, d - a));
        if (sc >= 0) q.pb(c);
        if (sc * sd < 0) q.pb(lineInter(a, b, c, d));
    }
    return q;
} // a60a48
```

Interseção de semiplanos. `Halfplane(a,b)` mantém a esquerda de $a \to b$, com $a \neq b$. `halfplaneInter` intersecta também com $[-B, B]^2$, $B > 0$; retorna polígono anti-horário ou vazio se sem área positiva. Reais. Não distingue regiões originalmente ilimitadas. Paralelos/repetidos permitidos; tolerância `EPS`. $O(n \log n)$.

a56079 - geometry/halfplane-intersection.cpp - 34 lines

```
struct Halfplane {
    Pt p, d; ld angle;
    Halfplane(Pt a, Pt b): p(a), d((b-a)/abs(b-a)), angle(arg(d)) {}
    bool out(Pt q) const { return cross(d,q-p) < -EPS; }
};
Pt hpInter(Halfplane a, Halfplane b) {
    return a.p + a.d * (cross(b.p-a.p,b.d)/cross(a.d,b.d));
} // f8014a
vector<Pt> halfplaneInter(vector<Halfplane> h, ld bound) {
    vector<Pt> box = {{-bound,-bound},{bound,-bound},
        {bound,bound},{-bound,bound}};
    rep(i,0,4) h.emplace_back(box[i],box[(i+1)%4]);
    sort(all(h), [](auto a, auto b) { return a.angle < b.angle; });
    vector<Halfplane> q = h;
    int l = 0, r = 0;
    for (auto a : h) {
        while (r-l>1 && a.out(hpInter(q[r-2],q[r-1]))) --r;
        while (r-l>1 && a.out(hpInter(q[l],q[l+1]))) ++l;
        if (r>l && abs(cross(a.d,q[r-1].d)) < EPS) {
            if (dot(a.d,q[r-1].d) < 0) return {};
            if (!a.out(q[r-1].p)) continue;
            --r;
        }
        q[r++] = a;
    }
    while (r-l>2 && q[l].out(hpInter(q[r-2],q[r-1]))) --r;
    while (r-l>2 && q[r-1].out(hpInter(q[l],q[l+1]))) ++l;
    if (r-l<3) return {};
    vector<Pt> p;
    rep(i,l,r) p.pb(hpInter(q[i],q[i+1<r ? i+1 : l]));
    ld area = 0;
    rep(i,0,sz(p)) area += cross(p[i]-p[0],p[(i+1)%sz(p)]-p[0]);
    return area > EPS ? p : vector<Pt>{};
} // 4682fa
```

Soma de Minkowski. `minkowski`: convexos anti-horários, sem vértice final repetido; saída anti-horária, podendo manter colineares. $O(|a| + |b|)$.

09f042 - geometry/minkowski.cpp - 17 lines

```
vector<Pt> minkowski(vector<Pt> a, vector<Pt> b) {
    auto norm = [&](vector<Pt>& p) {
        auto lt = [](Pt a, Pt b) { return a.xx != b.xx ? a.xx < b.xx : a.yy < b.yy;
        ↪ };
        rotate(p.begin(), min_element(all(p), lt), p.end());
        p.pb(p[0]), p.pb(p[1]);
    }; // f5b3e9
    int n = sz(a), m = sz(b);
    norm(a), norm(b);
    vector<Pt> c;
```

```
    for (int i = 0, j = 0; i < n || j < m; ) {
        c.pb(a[i] + b[j]);
        ll z = cross(a[i + 1] - a[i], b[j + 1] - b[j]); // ld: keep z in ld
        if (z >= 0 && i < n) i++; // ld: use z >= -EPS
        if (z <= 0 && j < m) j++; // ld: use z <= EPS
    }
    return c;
} // 09f042 //ld: a.xx != b.xx -> sgn(a.xx - b.xx)
```

Círculo–reta. `circLineInter`: pontos do círculo com a reta $a + td$, $d \neq 0$. Reais.

21e90b - geometry/circle-line.cpp - 8 lines

```
vector<Pt> circLineInter(Pt c, ld r, Pt a, Pt d) {
    Pt f = a + d * dot(c - a, d) / norm(d);
    ld h2 = r * r - norm(c - f);
    if (sgn(h2) < 0) return {};
    ld dt = sqrt(max((ld)0, h2)) / abs(d);
    if (!sgn(h2)) return {f};
    return {f - dt * d, f + dt * d};
} // 21e90b
```

Círculo–círculo. `circCircInter`: Reais; trate círculos coincidentes fora.

d3b1f8 - geometry/circle-circle.cpp - 12 lines

```
vector<Pt> circCircInter(Pt c1, ld r1, Pt c2, ld r2) {
    Pt d = c2 - c1;
    ld dist = abs(d);
    if (!sgn(dist)) return {};
    ld a = (norm(d) + r1 * r1 - r2 * r2) / (2 * dist);
    ld h2 = r1 * r1 - a * a;
    if (sgn(h2) < 0) return {};
    Pt mid = c1 + d * (a / dist);
    if (!sgn(h2)) return {mid};
    Pt p = perp(d) * (sqrt(max((ld)0, h2)) / dist);
    return {mid - p, mid + p};
} // d3b1f8
```

Tangentes comuns. `circTangents`: pares de pontos de tangência, externas e internas. Reais; trate centros iguais fora e remova duplicatas se necessário.

0d00d5 - geometry/circle-tangents.cpp - 14 lines

```
vector<pair<Pt, Pt>> circTangents(Pt c1, ld r1, Pt c2, ld r2) {
    vector<pair<Pt, Pt>> res;
    for (int s: {1, -1}) {
        ld r = s * r2, dr = r1 - r;
        Pt d = c2 - c1;
        ld l2 = norm(d), h2 = max((ld)0, l2 - dr * dr);
        if (sgn(l2 - dr * dr) < 0) continue;
        for (int t: {1, -1}) {
            Pt n = (dr * d + perp(d) * (t * sqrt(h2))) / l2;
            res.pb({c1 + r1 * n, c2 + r * n});
        }
    }
    return res;
} // 0d00d5
```

Círculo mínimo. `mec`: círculo `{o,r}` mínimo. Reais. Vetor vazio retorna raio -1. $O(n)$ esperado.

fd30f4 - geometry/enclosing-circle.cpp - 33 lines

```
struct C {
    Pt o;
    ld r;
};

bool in(C c, Pt p) { return abs(p - c.o) <= c.r + EPS; }

C circ(Pt a, Pt b) { return {(a + b) / (ld)2, abs(a - b) / 2}; }

C circ(Pt a, Pt b, Pt c) {
    if (fabsl(cross(b - a, c - a)) < EPS) {
        C x = circ(a, b), y = circ(a, c), z = circ(b, c);
        if (in(x, c)) return x;
        if (in(y, b)) return y;
        return z;
    }
    Pt o = a + perp((b - a) * norm(c - a) - (c - a) * norm(b - a))
        / cross(b - a, c - a) / (ld)2;
    return {o, abs(a - o)};
} // a6aab7

C mec(vector<Pt> p) {
    shuffle(all(p), rng);
    C c{{0, 0}, -1};
    rep(i,0,sz(p)) if (!in(c, p[i])) {
        c = {p[i], 0};
        rep(j,0,i) if (!in(c, p[j])) {
            c = circ(p[i], p[j]);
            rep(k,0,j) if (!in(c, p[k])) c = circ(p[i], p[j], p[k]);
```

```
        }
    }
    return c;
} // effcbe
```

Área círculo–polígono. `circlePolyArea`: área não negativa da interseção de um polígono simples com o disco de centro `c` e raio $r \geq 0$. Reais; aceita ambas as orientações e vértices consecutivos repetidos. $O(n)$.

070e93 - geometry/circle-polygon-area.cpp - 23 lines

```
ld circlePolyArea(const vector<Pt> &p, Pt c, ld r) {
    if (r <= 0 || p.size() < 3) return 0;
    ld ans = 0, r2 = r * r;
    for (int i = 0, n = p.size(); i < n; i++) {
        Pt a = p[i] - c, b = p[(i + 1) % n] - c, d = b - a;
        ld len2 = norm(d);
        if (!len2) continue;
        vector<ld> ts = {0, 1};
        ld t = -dot(a, d) / len2, h2 = r2 - norm(a + t * d);
        if (h2 > 0) {
            ld dt = sqrt(h2 / len2);
            for (ld x : {t - dt, t + dt})
                if (0 < x && x < 1) ts.push_back(x);
        }
        sort(ts.begin(), ts.end());
        for (int j = 1; j < (int)ts.size(); j++) {
            Pt u = a + ts[j - 1] * d, v = a + ts[j] * d;
            if (norm((u + v) / (ld)2) < r2) ans += cross(u, v);
            else ans += r2 * atan2(cross(u, v), dot(u, v));
        }
    }
    return abs(ans) / 2;
} // 070e93
```

Par mais próximo. `closestPair`: par de pontos, não índices. Requer $n \geq 2$ e coordenadas inteiras; trate duplicatas antes. Distâncias ao quadrado devem caber em `ll`. $O(n \log n)$.

0c44eb - geometry/closest-pair.cpp - 20 lines

```
bool cmpX(Pt a, Pt b) { return a.xx != b.xx ? a.xx < b.xx : a.yy < b.yy; }

pair<Pt,Pt> closestPair(vector<Pt> p) {
    sort(all(p), cmpX);
    set<pair<ll,ll>> s;
    pair<Pt,Pt> ret{p[0], p[1]};
    ll best = norm(p[0] - p[1]);
    for (int l = 0, i = 0; i < sz(p); i++) {
        ll d = sqrtl((ld)best) + 1;
        while (p[i].xx - p[l].xx > d) s.erase({p[l].yy, p[l].xx}), l++;
        auto it1 = s.lower_bound({p[i].yy - d, LLONG_MIN});
        auto it2 = s.upper_bound({p[i].yy + d, LLONG_MAX});
        for (auto it = it1; it != it2; ++it) {
            Pt q(it->se, it->fi);
            if (ll cur = norm(p[i] - q); cur < best) best = cur, ret = {p[i], q};
        }
        s.insert({p[i].yy, p[i].xx});
    }
    return ret;
} // 5f8da5
```

MST Manhattan. `manhattanMST`: até $4n$ arestas `(peso,u,v)` contendo uma MST de $|\Delta x| + |\Delta y|$. IDs originais, aceita duplicatas. `Pt=complex<ll>`, $|x|, |y| \leq 10^{18}$. $O(n \log n)$ tempo, $O(n)$ memória.

de0e83 - geometry/manhattan-mst.cpp - 22 lines

```
vector<tuple<ll,int,int>> manhattanMST(vector<Pt> p) {
    vi id(sz(p)); iota(all(id),0);
    vector<tuple<ll,int,int>> edges;
    rep(k,0,4) {
        sort(all(id), [&](int i,int j) {
            return p[i].xx+p[i].yy < p[j].xx+p[j].yy;
        } /* 29cc89 */);
        map<ll,int> sweep;
        for (int i : id) {
            for (auto it = sweep.lower_bound(-p[i].yy); it != sweep.end();) {
                int j = it->second;
                Pt d = p[i]-p[j];
                if (d.yy > d.xx) break;
                edges.emplace_back(d.xx+d.yy,i,j);
                it = sweep.erase(it);
            }
            sweep[-p[i].yy] = i;
        }
        for (Pt &v : p) v = k&1 ? Pt(-v.xx,v.yy) : Pt(v.yy,v.xx);
    }
    return edges;
} // de0e83
```

Faces de grafo planar. `faces`: contornos como listas de vértices (podem repetir vértices). Requer grafo simples conexo, embedding sem cruzamentos e ambas as direções em `g`. Inclui face externa. $O((n+m)\log n)$.

9376fd - geometry/planar-graph-faces.cpp - 39 lines

```cpp
ll half(Pt p) {
    return p.yy < 0 || (p.yy == 0 && p.xx < 0);
} // 16546c

ll norm2(Pt a) {
    return a.xx * a.xx + a.yy * a.yy;
} // 3cd1dc

vector<vi> faces(vector<Pt> &p, vector<vi> g) {
    int n = sz(g);
    vector<map<int, int>> id(n);
    rep(i,0,n) {
        sort(all(g[i]), [&](int a, int b) {
            Pt A = p[a] - p[i], B = p[b] - p[i];
            if (half(A) != half(B)) return half(A) < half(B);
            auto c = cross(A, B);
            if (c) return c > 0;
            return norm2(A) < norm2(B); // only tie safety
        } /* 68d3a3 */);
        rep(j,0,sz(g[i])) id[i][g[i][j]] = j;
    }
    vector<vector<char>> vis(n);
    rep(i,0,n) vis[i].assign(sz(g[i]), 0);
    vector<vi> fs;
    rep(s,0,n) rep(e,0,sz(g[s])) if (!vis[s][e]) {
        vi f;
        int u = s, i = e;
        while (!vis[u][i]) {
            vis[u][i] = 1;
            int v = g[u][i];
            f.pb(u);
            int j = id[v][u];
            u = v;
            i = (j - 1 + sz(g[u])) % sz(g[u]);
        }
        fs.pb(f);
    }
    return fs;
} // 02be84
```

Triangulação de Delaunay. `Delaunay d(p)`: pontos inteiros **distintos**, $|x|, |y| \leq 10^8$. `triangles`: triplas anti-horárias de IDs originais; `edges`: arestas não orientadas, inclusive cadeia se todos colineares. Aceita cocirculares e $n < 3$. $O(n \log n)$ tempo, $O(n)$ memória. A MST euclidiana está contida nessas arestas. Adaptado de cp-algorithms, SYury e colaboradores, CC BY-SA 4.0.

d882d7 - geometry/delaunay.cpp - 97 lines

```cpp
struct Delaunay {
    using I = __int128;
    struct Point { ll x,y; };
    struct Edge { int org=-1, next=0; };
    vector<Point> p;
    vi id, free;
    vector<Edge> e;
    vector<array<int,3>> triangles;
    vector<pair<int,int>> edges;
    static int rot(int a) { return (a&~3)|((a+1)&3); }
    int dest(int a) const { return e[a^2].org; }
    int prev(int a) const { return rot(e[rot(a)].next); }
    int left(int a) const { return rot(e[rot(a)^2].next); }
    I orient(int a,int b,int c) const {
        return I(p[b].x-p[a].x)*(p[c].y-p[a].y)
            -I(p[b].y-p[a].y)*(p[c].x-p[a].x);
    } // db6dcd
    bool circle(int a,int b,int c,int d) const {
        I x=p[a].x-p[d].x, y=p[a].y-p[d].y;
        I u=p[b].x-p[d].x, v=p[b].y-p[d].y;
        I s=p[c].x-p[d].x, t=p[c].y-p[d].y;
        return (x*x+y*y)*(u*t-v*s)-(u*u+v*v)*(x*t-y*s)
            +(s*s+t*t)*(x*v-y*u) > 0;
    } // 8e2844
    void splice(int a,int b) {
        int x=rot(e[a].next), y=rot(e[b].next);
        swap(e[x].next,e[y].next); swap(e[a].next,e[b].next);
    } // b349e1
    int make(int a,int b) {
        int k;
        if (free.empty()) { k=sz(e); e.resize(k+4); }
        else { k=free.back(); free.pop_back(); }
        e[k]={a,k}; e[k+2]={b,k+2};
        e[k+1]={-1,k+3}; e[k+3]={-1,k+1}; return k;
```

```
} // f38274
void erase(int a) {
    splice(a,prev(a)); splice(a^2,prev(a^2));
    e[a].org=e[a^2].org=-1; free.push_back(a&~3);
} // c4a2f2
int connect(int a,int b) {
    int c=make(dest(a),e[b].org);
    splice(c,left(a)); splice(c^2,b); return c;
} // e58fdd
pair<int,int> build(int l,int r) {
    if (r-l <= 3) {
        int a=make(l,l+1);
        if (r-l == 2) return {a,a^2};
        int b=make(l+1,l+2); splice(a^2,b);
        I s=orient(l,l+1,l+2);
        if (!s) return {a,b^2};
        int c=connect(b,a);
        return s>0 ? pair{a,b^2} : pair{c^2,c};
    }
    int m=(l+r)/2;
    auto [lo,li]=build(l,m); auto [ri,ro]=build(m,r);
    while (true) {
        if (orient(e[li].org,dest(li),e[ri].org)>0) li=left(li);
        else if (orient(e[ri].org,dest(ri),e[li].org)<0) ri=e[ri^2].next;
        else break;
    }
    int base=connect(ri^2,li);
    if (e[li].org==e[lo].org) lo=base^2;
    if (e[ri].org==e[ro].org) ro=base;
    auto valid=[&](int a) { return orient(e[base].org,dest(base),dest(a))<0; };
    while (true) {
        int a=e[base^2].next, b=prev(base);
        if (valid(a)) while
        ↪  (circle(dest(base),e[base].org,dest(a),dest(e[a].next))) {
            int t=e[a].next; erase(a); a=t;
        }
        if (valid(b)) while
        ↪  (circle(dest(base),e[base].org,dest(b),dest(prev(b)))) {
            int t=prev(b); erase(b); b=t;
        }
        if (!valid(a) && !valid(b)) break;
        if (!valid(a) || (valid(b) && circle(dest(a),e[a].org,e[b].org,dest(b))))
            base=connect(b,base^2);
        else base=connect(base^2,a^2);
    }
    return {lo,ro};
} // 9c7b54
Delaunay(vector<Point> input) {
    int n=sz(input); id.resize(n); iota(all(id),0);
    sort(all(id),[&](int a,int b) {
        return tie(input[a].x,input[a].y)<tie(input[b].x,input[b].y);
    } /* f9b851 */);
    for (int i : id) p.push_back(input[i]);
    if (n<2) return;
    build(0,n); vector<bool> seen(sz(e));
    for (int a=0; a<sz(e); a+=2) if (e[a].org>=0) {
        if (!(a&2)) edges.push_back({id[e[a].org],id[dest(a)]});
        if (seen[a]) continue;
        vi face; int b=a;
        do { seen[b]=true; face.push_back(e[b].org); b=left(b); } while (b!=a);
        if (sz(face)==3 && orient(face[0],face[1],face[2])>0)
            triangles.push_back({id[face[0]],id[face[1]],id[face[2]]});
    }
}
};
```

Dual de Voronoi. O circuncentro de cada triângulo é um vértice. Para aresta interna, ligue os circuncentros adjacentes; para aresta de borda $u \to v$ do triângulo anti-horário, emita do circuncentro o raio na direção $(\Delta y, -\Delta x)$. Triângulos cocirculares produzem segmentos de comprimento zero. Com todos os sítios colineares, use as mediatrizes completas entre vizinhos; um único sítio ocupa o plano.

Maior distância ao sítio mais próximo em um retângulo. Os candidatos são vértices de Voronoi internos, interseções do dual com a borda e cantos do retângulo. Avalie cada candidato usando o sítio da célula/aresta correspondente; nos cantos, encontre o sítio mais próximo.

Convex hull 3D. `hull3D(p)`: faces triangulares com IDs originais, orientadas para fora; aceita duplicatas e faces coplanares trianguladas. Pontos inteiros $|x|, |y|, |z| \leq 10^9$. Retorna vazio se dimensão afim menor que 3. Tempo esperado $O(n^2)$, memória $O(n)$. `volume6(a,b,c,d)>0` indica que d está do lado externo da face orientada abc.

01226b - geometry/hull-3d.cpp - 55 lines

```
struct Point3 {
    ll x,y,z;
    Point3 operator-(Point3 b) const { return {x-b.x,y-b.y,z-b.z}; }
};
```

```
__int128 volume6(Point3 a, Point3 b, Point3 c, Point3 d) {
    b = b-a; c = c-a; d = d-a;
    return (__int128)b.x*( (__int128)c.y*d.z-(__int128)c.z*d.y)
        - (__int128)b.y*( (__int128)c.x*d.z-(__int128)c.z*d.x)
        + (__int128)b.z*( (__int128)c.x*d.y-(__int128)c.y*d.x);
} // 68c5ad
vector<array<int,3>> hull3D(const vector<Point3> &p) {
    int n = sz(p);
    if (n < 4) return {};
    vi id(n); iota(all(id),0);
    auto noncollinear = [&](int a,int b,int c) {
        auto u = p[b]-p[a], v = p[c]-p[a];
        return (__int128)u.x*v.y != (__int128)u.y*v.x ||
            (__int128)u.x*v.z != (__int128)u.z*v.x ||
            (__int128)u.y*v.z != (__int128)u.z*v.y;
    }; // 167577
    int b = 1;
    while (b < n && p[b].x == p[0].x && p[b].y == p[0].y && p[b].z == p[0].z) ++b;
    if (b == n) return {};
    swap(id[1],id[b]); b = 2;
    while (b < n && !noncollinear(id[0],id[1],id[b])) ++b;
    if (b == n) return {};
    swap(id[2],id[b]); b = 3;
    while (b < n && !volume6(p[id[0]],p[id[1]],p[id[2]],p[id[b]])) ++b;
    if (b == n) return {};
    swap(id[3],id[b]);
    vector<array<int,3>> f;
    rep(i,0,4) rep(j,i+1,4) rep(k,j+1,4) {
        array<int,3> t = {id[i],id[j],id[k]};
        if (volume6(p[t[0]],p[t[1]],p[t[2]],p[id[6-i-j-k]]) > 0) swap(t[1],t[2]);
        f.push_back(t);
    }
    rep(k,4,n) {
        unordered_set<uint64_t> boundary;
        vector<array<int,3>> keep;
        for (auto t : f) {
            if (volume6(p[t[0]],p[t[1]],p[t[2]],p[id[k]]) <= 0) {
                keep.push_back(t); continue;
            }
            rep(j,0,3) {
                uint64_t a = t[j], b = t[(j+1)%3];
                if (!boundary.erase((b<<32)|a)) boundary.insert((a<<32)|b);
            }
        }
        for (auto e : boundary)
            keep.push_back({int(e>>32),
                int(uint32_t(e)),id[k]});
        f.swap(keep);
    }
    return f;
} // a9abed
```

Subproblemas

Atribuição em intervalos constantes. `Intervals(n,x=0)`, $n > 0$: domínio $[0, n)$. `assign(l,r,x)` atribui em $[l, r)$. Em `m`, cada chave inicia um trecho constante; a chave n é sentinela. Atribuição: $O(\log S + k)$ para S trechos e k apagados; consulta $O(\log S)$.

f0a368 - problems/interval-assignment.cpp - 22 lines

```
struct Intervals {
    ll n;
    map<ll,ll> m;
    Intervals(ll n, ll x = 0): n(n), m{{0,x},{n,0}} {}
    auto split(ll p) {
        auto it = m.lower_bound(p);
        if (it->first == p) return it;
        return m.emplace(p, prev(it)->second).first;
    } // 031048
    void assign(ll l, ll r, ll x) {
        if (l == r) return;
        auto b = split(r), a = split(l);
        m.erase(a, b);
        a = m.emplace(l, x).first;
        if (a != m.begin() && prev(a)->second == x) {
            a = prev(a); m.erase(next(a));
        }
        b = next(a);
        if (b->first < n && b->second == x) m.erase(b);
    } // 13ec3c
    ll get(ll p) { return prev(m.upper_bound(p))->second; }
};
```

Somas dinâmicas em retângulos. `Fenwick2D(pts)`: registre todos os possíveis pontos de atualização `add(x,y,v)`; limites de consulta livres, valores iniciais zero. `sum(x,y)` soma pontos com coordenadas estritamente menores que x, y;

`rect(x1,y1,x2,y2)` soma $[x_1, x_2) \times [y_1, y_2)$. Para N pontos registrados: construção $O(N \log^2 N)$, operações $O(\log^2 N)$, memória $O(N \log N)$.

2037b0 - problems/fenwick-2d.cpp - 37 lines

```cpp
struct Fenwick2D {
    vector<ll> x;
    vector<vector<ll>> y, t;
    int pos(vector<ll> &v, ll a) {
        return lower_bound(v.begin(), v.end(), a)-v.begin();
    } // bf1da9
    Fenwick2D(vector<pair<ll,ll>> pts) {
        for (auto [a,b] : pts) x.push_back(a);
        sort(x.begin(), x.end());
        x.erase(unique(x.begin(), x.end()), x.end());
        int n = x.size();
        y.resize(n+1); t.resize(n+1);
        for (auto [a,b] : pts)
            for (int i = pos(x,a)+1; i <= n; i += i&-i)
                y[i].push_back(b);
        for (int i = 1; i <= n; ++i) {
            sort(y[i].begin(), y[i].end());
            y[i].erase(unique(y[i].begin(), y[i].end()), y[i].end());
            t[i].resize(y[i].size()+1);
        }
    }
    void add(ll a, ll b, ll v) {
        for (int i = pos(x,a)+1; i < int(t.size()); i += i&-i)
            for (int j = pos(y[i],b)+1; j < int(t[i].size()); j += j&-j)
                t[i][j] += v;
    } // 115c7c
    ll sum(ll a, ll b) {
        ll s = 0;
        for (int i = pos(x,a); i; i -= i&-i)
            for (int j = pos(y[i],b); j; j -= j&-j)
                s += t[i][j];
        return s;
    } // 94b913
    ll rect(ll a, ll b, ll c, ll d) {
        return sum(c,d)-sum(a,d)-sum(c,b)+sum(a,b);
    } // 8b6f44
};
```

Área da união de retângulos. `rectUnion(rects)`: retângulos alinhados aos eixos, `{x1,y1,x2,y2}`, $x_1 \le x_2$, $y_1 \le y_2$. Ignora área zero; aceita duplicatas. Retorna área em `i128`; coordenadas inteiras de módulo $\le 10^{18}$. $O(n \log n)$ tempo, $O(n)$ memória.

b79e15 - problems/rectangle-union.cpp - 33 lines

```cpp
i128 rectUnion(vector<array<ll,4>> rects) {
    vector<tuple<ll,int,ll,ll>> ev;
    vector<ll> y;
    for (auto [a,b,c,d] : rects) if (a < c && b < d) {
        ev.emplace_back(a, 1, b, d); ev.emplace_back(c, -1, b, d);
        y.push_back(b); y.push_back(d);
    }
    if (ev.empty()) return 0;
    sort(ev.begin(), ev.end()); sort(y.begin(), y.end());
    y.erase(unique(y.begin(), y.end()), y.end());
    int n = y.size()-1;
    vector<int> cnt(4*n);
    vector<ll> len(4*n);
    auto upd = [&](auto &&self, int p, int l, int r, int a, int b, int d) -> void {
        if (b <= l || r <= a) return;
        if (a <= l && r <= b) cnt[p] += d;
        else {
            int m = (l+r)/2;
            self(self, 2*p, l, m, a, b, d);
            self(self, 2*p+1, m, r, a, b, d);
        }
        len[p] = cnt[p] ? y[r]-y[l] : (r-l == 1 ? 0 : len[2*p]+len[2*p+1]);
    }; // 6fa5a1
    i128 area = 0;
    ll last = get<0>(ev[0]);
    for (auto [x,d,a,b] : ev) {
        area += i128(x-last)*len[1]; last = x;
        int l = lower_bound(y.begin(), y.end(), a)-y.begin();
        int r = lower_bound(y.begin(), y.end(), b)-y.begin();
        upd(upd, 1, 0, n, l, r, d);
    }
    return area;
} // b79e15
```

Mochila com quantidades limitadas. `boundedKnapsack(W,items)`: itens `{peso, valor,quantidade}`, peso positivo e quantidade não negativa. Retorna valor máximo por peso **exato** em $[0, W]$; `LLONG_MIN` se impossível. Aceita valores negativos; valores/intermediários em `ll`, distintos de `LLONG_MIN`. $O(nW)$ tempo, $O(W)$ memória.

0cf538 - problems/bounded-knapsack.cpp - 21 lines

```cpp
vector<ll> boundedKnapsack(int W, vector<tuple<int,ll,int>> items) {
```

```
    vector<ll> dp(W+1, LLONG_MIN);
    dp[0] = 0;
    for (auto [w,v,c] : items) {
        if (w > W || !c) continue;
        auto old = dp;
        for (int r = 0; r < w; ++r) {
            deque<pair<int,ll>> q;
            for (int s = r, j = 0; s <= W; s += w, ++j) {
                while (!q.empty() && q.front().first < j-c) q.pop_front();
                if (old[s] != LLONG_MIN) {
                    ll x = old[s]-j*v;
                    while (!q.empty() && q.back().second <= x) q.pop_back();
                    q.emplace_back(j, x);
                }
                dp[s] = q.empty() ? LLONG_MIN : q.front().second+j*v;
            }
        }
    }
    return dp;
} // 0cf538
```

DP para todas as raízes. `reroot`: árvore não vazia; resposta por raiz. `merge` associativa com neutro `e`; agrega vizinhos na ordem de `g[u]`. `vertex(x,u)` incorpora u ao agregado; `edge(x,u,v)` transporta o resultado de u para v. $O(n)$ tempo e memória para operações/cópias $O(1)$; sem recursão.

31305a - problems/reroot.cpp - 33 lines

```
template<class T>
vector<T> reroot(vector<vi> &g, T e, auto merge, auto vertex, auto edge) {
    int n = g.size();
    vi p(n, -1), ord{0};
    vector<T> down(n, e), up(n, e), ans(n);
    for (int i = 0; i < n; ++i)
        for (int v : g[ord[i]]) if (v != p[ord[i]])
            p[v] = ord[i], ord.push_back(v);
    for (int i = n-1; i >= 0; --i) {
        int u = ord[i];
        T x = e;
        for (int v : g[u]) if (v != p[u])
            x = merge(x, edge(down[v], v, u));
        down[u] = vertex(x, u);
    }
    for (int u : ord) {
        int m = g[u].size();
        vector<T> a(m), suf(m+1, e);
        for (int i = 0; i < m; ++i) {
            int v = g[u][i];
            a[i] = v == p[u] ? up[u] : edge(down[v], v, u);
        }
        for (int i = m-1; i >= 0; --i) suf[i] = merge(a[i], suf[i+1]);
        ans[u] = vertex(suf[0], u);
        T pre = e;
        for (int i = 0; i < m; ++i) {
            int v = g[u][i];
            if (v != p[u]) up[v] = edge(vertex(merge(pre, suf[i+1]), u), u, v);
            pre = merge(pre, a[i]);
        }
    }
    return ans;
} // a4e86d
```

Restrições de diferenças e paridade. `PotentialDSU<>`: `merge(u,v,w)` impõe $a_v - a_u = w$; retorna falso e rejeita a restrição se contraditória. `diff(u,v)` retorna $a_v - a_u$, ou `nullopt` se desconexos. `PotentialDSU<true>`: substitui diferença por XOR, com $w \in \{0,1\}$. $O(\alpha(n))$ amortizado por operação.

fc9044 - problems/potential-dsu.cpp - 26 lines

```
template<bool parity = false> struct PotentialDSU {
    vector<int> p;
    vector<ll> d;
    PotentialDSU(int n): p(n, -1), d(n) {}
    ll sum(ll a, ll b) { return parity ? (a^b) : a+b; }
    ll neg(ll a) { return parity ? a : -a; }
    int root(int u) {
        if (p[u] < 0) return u;
        int v = p[u], r = root(v);
        d[u] = sum(d[u], d[v]);
        return p[u] = r;
    } // d5f9e3
    bool merge(int u, int v, ll w) {
        int a = root(u), b = root(v);
        w = sum(sum(w, d[u]), neg(d[v]));
        if (a == b) return w == 0;
        if (p[a] > p[b]) swap(a, b), w = neg(w);
        p[a] += p[b]; p[b] = a; d[b] = w;
        return true;
    } // c5b3b3
```

```
    optional<ll> diff(int u, int v) {
        int a = root(u), b = root(v);
        if (a != b) return nullopt;
        return sum(d[v], neg(d[u]));
    } // ae64cc
};
```

Circulação com limites inferiores. `BoundedFlow(n)` usa Dinic. `add(u,v,l,r)` retorna ID, $0 \le l \le r$. Após inserir tudo, chame `feasible()` **uma vez**; se verdadeiro, `get(id)` recupera o fluxo. Para fluxo $s \to t$ de valor em $[L, R]$, adicione $t \to s$ com limites $[L, R]$. Capacidades, balanços e demanda total devem caber em `ll`. Um fluxo máximo em grafo com $n + 2$ vértices e $m + O(n)$ arestas.

fe3172 - problems/bounded-flow.cpp - 21 lines

```
struct BoundedFlow {
    int n;
    Dinic d;
    vector<ll> bal, lo;
    vi ids;
    BoundedFlow(int n): n(n), d(n+2), bal(n) {}
    int add(int u, int v, ll l, ll r) {
        ids.push_back(d.e.size()); lo.push_back(l);
        d.addEdge(u, v, r-l);
        bal[u] -= l; bal[v] += l;
        return ids.size()-1;
    } // dca9a6
    bool feasible() {
        ll need = 0;
        for (int u = 0; u < n; ++u)
            if (bal[u] > 0) d.addEdge(n, u, bal[u]), need += bal[u];
            else d.addEdge(u, n+1, -bal[u]);
        return d.flow(n, n+1, need) == need;
    } // 86befa
    ll get(int id) { return lo[id]+d.e[ids[id]].f; }
};
```

Fórmulas, Teoremas e Fatos

Teoria dos números

Ceiling inteiro. Para $a, b > 0$, $\lceil a/b \rceil = (a + b - 1)/b$ em aritmética inteira.

Contagem / soma de divisores. Se

$$n = \prod p_i^{e_i},$$

então

$$d(n) = \prod (e_i + 1),$$

$$\sigma(n) = \prod \frac{p_i^{e_i+1} - 1}{p_i - 1}.$$

Máximos de divisores e fatores primos. Para $1 \le n \le N$, sejam $D(N) = \max d(n)$ e $W(N) = \max \omega(n)$, onde $d(n)$ conta divisores positivos e $\omega(n)$ conta primos distintos. n_* é o menor inteiro com $d(n_*) = D(N)$.

N	$D(N)$	$W(N)$	n_*
10^1	4	2	6
10^2	12	3	60
10^3	32	4	840
10^4	64	5	7560
10^5	128	6	83160
10^6	240	7	720720
10^7	448	8	8648640
10^8	768	8	73513440
10^9	1344	9	735134400
$2^{31} - 1$	1600	9	2095133040
$2^{32} - 1$	1920	9	3491888400
10^{10}	2304	10	6983776800
10^{11}	4032	10	97772875200
10^{12}	6720	11	963761198400
10^{13}	10752	12	9316358251200
10^{14}	17280	12	97821761637600
10^{15}	26880	13	866421317361600
10^{16}	41472	13	8086598962041600
10^{17}	64512	14	74801040398884800
10^{18}	103680	15	897612484786617600
$2^{63} - 1$	161280	15	9200527969062830400
$2^{64} - 1$	184320	15	18401055938125660800

$W(N)$ é o maior k com $2 \cdot 3 \cdot 5 \cdots p_k \le N$ (produto dos primeiros k primos). Para

obter $D(N)$ em outro limite, enumere $2^{e_1}3^{e_2}\cdots \le N$ com $e_1 \ge e_2 \ge \cdots \ge 1$ e maximize $\prod(e_i+1)$.

Phi de Euler.

$$\varphi(n) = n\prod_{p\mid n}\left(1-\frac{1}{p}\right)$$

Triplas pitagóricas. Soluções positivas primitivas de $a^2+b^2=c^2$:

$$a = m^2-n^2, \qquad b = 2mn, \qquad c = m^2+n^2,$$

Para inteiros $m > n \ge 1$, $\gcd(m,n)=1$ e $m \not\equiv n \pmod 2$. Todas as soluções positivas vêm de múltiplos $k \ge 1$ e troca dos catetos.

Somas de dois quadrados. Para $n > 0$, existem inteiros x, y com $x^2+y^2=n$ sse todo primo $p \equiv 3 \pmod 4$ tem expoente par na fatoração de n. Nesse caso, o número de pares *ordenados e com sinal*, incluindo coordenadas zero, é

$$r_2(n) = 4 \prod_{\substack{p\mid n \\ p\equiv 1 \pmod 4}} (v_p(n)+1),$$

Se a condição falha, $r_2(n)=0$. Para $n=0$, há apenas $(0,0)$.

Somas de três quadrados. Um inteiro não negativo n pode ser escrito como

$$n = x^2+y^2+z^2$$

com x, y, z inteiros sse

$$n \neq 4^a(8b+7)$$

para todos os inteiros $a, b \ge 0$.

Fórmula de Legendre. Para p primo, o maior expoente de p que divide $n!$ é

$$\sum_{k\ge 1}\left\lfloor \frac{n}{p^k}\right\rfloor$$

Teorema de Lucas. Para primo p, se

$$n = \sum n_i p^i, \qquad k = \sum k_i p^i,$$

então

$$\binom{n}{k} \equiv \prod_i \binom{n_i}{k_i} \pmod p.$$

Teorema de Kummer. O expoente de um primo p em $\binom{n}{k}$ é igual ao número de vai-uns ao somar k e $n-k$ na base p.

LTE (versão comum). Se p é um primo ímpar, $p \mid (a-b)$ e $p \nmid ab$, então

$$v_p(a^n-b^n) = v_p(a-b)+v_p(n).$$

Para a versão da soma, exija p primo ímpar, $p \nmid ab$, n ímpar e $p \mid (a+b)$:

$$v_p(a^n+b^n) = v_p(a+b)+v_p(n).$$

Condição geral de compatibilidade do CRT. O sistema

$$x \equiv a_i \pmod{m_i}$$

tem solução sse, para todos i, j,

$$a_i \equiv a_j \pmod{\gcd(m_i, m_j)}.$$

Quando tem solução, ela é única módulo $\operatorname{lcm}(m_1,\ldots,m_k)$.

Inversão de Möbius. Se

$$g(n) = \sum_{d\mid n} f(d),$$

então

$$f(n) = \sum_{d\mid n} \mu(d)\, g\!\left(\frac{n}{d}\right).$$

Truque de agrupar quocientes. O valor de

$$\left\lfloor \frac{n}{i}\right\rfloor$$

muda $O(\sqrt{n})$ vezes para $1 \le i \le n$ ($n \ge 1$). Mais precisamente, todos os i com o mesmo quociente formam um intervalo

$$i \in \left[\left\lfloor \frac{n}{\lfloor n/i\rfloor + 1}\right\rfloor + 1,\ \left\lfloor \frac{n}{\lfloor n/i\rfloor}\right\rfloor\right].$$

Somas de moedas por grafo de resíduos. Moedas inteiras positivas $c_1, \ldots, c_k$, sem limite de uso; seja $a = \min c_i$. Nos resíduos $0, \ldots, a-1$, crie arcos $r \to (r + c_i) \bmod a$ de peso c_i. Dijkstra a partir de 0 dá $d[r]$, a menor soma representável nesse resíduo (∞ se inalcançável). Para inteiro $x \geq 0$,

$$x \text{ é representável} \iff d[x \bmod a] \leq x.$$

Se $\gcd(c_1, \ldots, c_k) = 1$, todos os resíduos são alcançáveis e o maior inteiro não representável é $\max_r d[r] - a$ (-1 se $a = 1$). Para gcd $g > 1$, rejeite $g \nmid x$; caso contrário, divida moedas e alvo por g. Com heap indexado e arcos gerados sob demanda: $O(ka \log(a+1))$ tempo, $O(a+k)$ memória.

Existência de raiz primitiva. Existem raízes primitivas módulo n sse

$$n \in \{1, 2, 4, p^k, 2p^k\},$$

onde p é um primo ímpar.

Teorema de Wilson. Para primo p,

$$(p-1)! \equiv -1 \pmod{p}.$$

Contagem

Coeficiente binomial.

$$\binom{n}{k} = \frac{n!}{k!(n-k)!}$$

Estrelas e barras. Para $n \geq 0$, $k \geq 1$, o número de soluções inteiras não negativas de

$$x_1 + \cdots + x_k = n$$

é

$$\binom{n+k-1}{k-1}.$$

Para $n \geq k$, o número de soluções inteiras positivas é

$$\binom{n-1}{k-1}.$$

Estrelas e barras com cotas inferiores. O número de soluções inteiras de

$$x_1 + \cdots + x_n = S, \qquad x_i \geq l_i$$

é

$$\binom{S - \sum l_i + n - 1}{n-1},$$

para cotas inteiras e $S \geq \sum l_i$; caso contrário, zero.

Número de Catalan.

$$C_n = \frac{1}{n+1}\binom{2n}{n} = \binom{2n}{n} - \binom{2n}{n+1}$$

Conta parênteses balanceados ($2n$ símbolos), árvores binárias enraizadas (n nós internos), triangulações de $(n+2)$-ágonos, matchings perfeitos sem cruzamento ($2n$ pontos) e caminhos direita/cima de $(0,0)$ a (n,n) sem passar acima da diagonal.

Reflexão: caminhos acima de uma barreira. O número de caminhadas de $n \geq 0$ passos ± 1, de altura inteira $a \geq 0$ até $b \geq 0$, sem ficar negativo, é

$$\binom{n}{(n+b-a)/2} - \binom{n}{(n+b+a+2)/2}.$$

Binomial com índice inferior não inteiro ou fora de $[0, n]$ vale zero. Para barreira h, subtraia h das alturas.

Número de Narayana. A quantidade de caminhos de $(0,0)$ a (N,N) que não passam acima da diagonal e com exatamente M picos locais é

$$\frac{1}{N}\binom{N}{M}\binom{N}{M-1}$$

Lema dos ciclos. Para inteiros $A_i \leq 1$ com soma $K > 0$, exatamente K posições iniciais dão rotações com todos os prefixos positivos (conte posições, não sequências distintas).

Desarranjos.

$$D_n = n! \sum_{k=0}^{n} \frac{(-1)^k}{k!}$$

e

$$D_0 = 1, \quad D_1 = 0, \quad D_n = (n-1)(D_{n-1} + D_{n-2}) \ (n \geq 2)$$

Funções de parking. Para $1 \leq m \leq n$, o número de sequências $a = (a_1, a_2, \ldots, a_m)$ tais que $\forall i, 1 \leq a_i \leq n$, e para $k = 1, 2, \ldots, n$, o número de índices i que satisfazem $a_i \leq k$ é pelo menos $m - n + k$ é $(n+1-m)(n+1)^{m-1}$

Números de Stirling do primeiro tipo. $\begin{bmatrix} n \\ k \end{bmatrix}$ é o número de permutações de tamanho n com exatamente k ciclos.

$$\begin{bmatrix} n \\ k \end{bmatrix} = \begin{bmatrix} n-1 \\ k-1 \end{bmatrix} + (n-1) \begin{bmatrix} n-1 \\ k \end{bmatrix}$$

Números de Stirling do segundo tipo. $\begin{Bmatrix} n \\ k \end{Bmatrix}$ é o número de formas de dividir os inteiros de 1 a n em k caixas não vazias e indistinguíveis.

$$\begin{Bmatrix} n \\ k \end{Bmatrix} = \begin{Bmatrix} n-1 \\ k-1 \end{Bmatrix} + k \begin{Bmatrix} n-1 \\ k \end{Bmatrix}$$

Forma fechada:

$$\begin{Bmatrix} n \\ k \end{Bmatrix} = \frac{1}{k!} \sum_{i=0}^{k} (-1)^i \binom{k}{i} (k-i)^n$$

Fatoriais subindo. Para $x^{\overline{n}} = x(x+1)(x+2)\dots(x+n-1)$, temos que

$$x^{\overline{n}} = \sum_{k=0}^{n} \begin{bmatrix} n \\ k \end{bmatrix} x^k$$

e

$$x^n = \sum_{k=0}^{n} (-1)^{n-k} \begin{Bmatrix} n \\ k \end{Bmatrix} x^{\overline{k}}$$

Números de Bell.

$$B_n = \sum_{k=0}^{n} S(n,k)$$

Partições inteiras (Euler). Se $p(n)$ conta somas não ordenadas de inteiros positivos com total n, ponha $p(0) = 1$ e $p(n) = 0$ para $n < 0$. Para $n > 0$, com $g_k^{\pm} = k(3k \pm 1)/2$,

$$p(n) = \sum_{k \geq 1} (-1)^{k-1} \big(p(n - g_k^-) + p(n - g_k^+) \big).$$

Pare quando $g_k^- > n$. Calcula $p(0), \dots, p(N)$ em $O(N\sqrt{N})$ tempo e $O(N)$ memória; válido módulo qualquer inteiro positivo.

Inversão de Lagrange. Extrai coeficientes de séries definidas implicitamente. Se $F(x) = x\phi(F(x))$, $F(0) = 0$, $\phi(0) \neq 0$, então, para $n \geq k \geq 1$,

$$[x^n] F(x)^k = \frac{k}{n} [t^{n-k}] \phi(t)^n.$$

Mais geralmente, $[x^n] H(F(x)) = \frac{1}{n} [t^{n-1}] H'(t) \phi(t)^n$ para $n \geq 1$. Em característica zero; módulo primo p, use $n < p$.

Contagem: simetrias e restrições

Permanente: contar matchings perfeitos (Ryser). Para A de tamanho $n \times n$, $n \geq 1$, o permanente soma $\prod_i A_{i,\pi(i)}$ sobre todas as permutações π. Para a matriz de adjacência bipartida 0/1, conta matchings perfeitos. Com $[n] = \{1, \dots, n\}$,

$$\operatorname{per}(A) = \sum_{S \subseteq [n]} (-1)^{n-|S|} \prod_{i=1}^{n} \left(\sum_{j \in S} A_{ij} \right).$$

Gray code mantém somas de linha e recalcula o produto em $O(n)$ por subconjunto. Tempo $O(n2^n)$, memória auxiliar $O(n)$; sem divisões, válido módulo qualquer inteiro positivo. Para $n = 0$, o permanente é 1.

Lema de Burnside. Para contar configurações distintas a menos de simetrias de um grupo finito G, tire a média do número de configurações fixadas por cada simetria, $|\operatorname{Fix}(g)|$:

$$\#\text{classes} = \frac{1}{|G|} \sum_{g \in G} |\operatorname{Fix}(g)|.$$

Módulo m, só divida por inverso se $\gcd(|G|, m) = 1$; caso contrário, calcule a soma módulo $m|G|$ e divida o representante por $|G|$ nos inteiros.

Exatamente k propriedades. Seja S_j a soma das quantidades de objetos que satisfazem simultaneamente cada escolha de j entre m propriedades; S_0 é o total de objetos. O número que satisfaz *exatamente* k é

$$\sum_{j=k}^{m} (-1)^{j-k} \binom{j}{k} S_j.$$

Um objeto com t propriedades contribui $\binom{t}{j}$ a S_j; não conte apenas uma vez a união das interseções.

Árvores rotuladas (Prüfer). Para $n \geq 2$ vértices distintos, há n^{n-2} árvores. Com

graus prescritos $d_i \geq 1$, $\sum d_i = 2n-2$, há

$$\frac{(n-2)!}{\prod_{i=1}^{n}(d_i-1)!};$$

fora dessas condições, zero. Cada vértice aparece $d_i - 1$ vezes no código de Prüfer, de comprimento $n-2$.

Ordens válidas em floresta enraizada. Em uma floresta fixa com n vértices distintos, o número de ordens em que cada pai aparece antes dos filhos é

$$\frac{n!}{\prod_v s_v}, \qquad s_v = \text{tamanho da subárvore incluindo } v.$$

Não identifique ordens por simetrias. Para usar inversos módulo primo p, exija $n < p$.

Completar uma floresta fixa. Uma floresta sobre os n vértices rotulados, com $c \geq 2$ componentes de tamanhos $s_1, \ldots, s_c$, está contida em

$$n^{c-2}\prod_{i=1}^{c} s_i$$

árvores no grafo completo. Para $c = 1$, há uma única conclusão: a própria árvore.

Somas e sequências

Progressão aritmética.

$$a + (a+d) + \cdots + (a+(n-1)d) = \frac{n(2a+(n-1)d)}{2}$$

Progressão geométrica.

$$a + ar + \cdots + ar^{n-1} = a\frac{r^n - 1}{r-1} \qquad (r \neq 1)$$

Somas de potências comuns.

$$\sum_{i=1}^{n} i^2 = \frac{n(n+1)(2n+1)}{6}$$

$$\sum_{i=1}^{n} i^3 = \left(\frac{n(n+1)}{2}\right)^2$$

Diferenças finitas e somas polinomiais. Para P de grau d, defina $\Delta P(x) = P(x+1) - P(x)$. Para inteiro $n \geq 0$,

$$P(n) = \sum_{j=0}^{d} \binom{n}{j} \Delta^j P(0),$$

$$\sum_{i=0}^{n-1} P(i) = \sum_{j=0}^{d} \binom{n}{j+1} \Delta^j P(0).$$

$\Delta^j P(0)$: primeiro elemento da j-ésima linha de diferenças de $P(0), \ldots, P(d)$. Com fatoriais inversos módulo primo p, requer $d + 1 < p$.

Convolução binomial (Vandermonde).

$$\sum_{k} \binom{a}{k}\binom{b}{n-k} = \binom{a+b}{n}.$$

Inversão binomial.

$$b_n = \sum_{k=0}^{n} \binom{n}{k} a_k \quad \Longleftrightarrow \quad a_n = \sum_{k=0}^{n} (-1)^{n-k} \binom{n}{k} b_k.$$

Valor esperado por indicadores. Se

$$X = \sum_i I_i,$$

então

$$\mathbb{E}[X] = \sum_i \Pr(I_i = 1).$$

Contagem de iterações em submasks. Número total de pares (m, s) com $s \subseteq m \subseteq \{0, \ldots, n-1\}$:

$$3^n.$$

Fórmula de contribuição na soma de todos os subarrays.

$$\sum_{0\le l\le r<n}\sum_{i=l}^{r} a_i = \sum_{i=0}^{n-1} a_i(i+1)(n-i).$$

Contribuição de uma aresta para a soma das distâncias na árvore. Se remover uma aresta de peso w separa a árvore em componentes de tamanhos s e $n-s$, então sua contribuição para

$$\sum_{u<v}\operatorname{dist}(u,v)$$

é

$$w\cdot s(n-s).$$

Matriz de Fibonacci. Para $F_0=F_1=1$, $F_n=F_{n-1}+F_{n-2}$ e $n\ge 2$,

$$\begin{bmatrix}1&1\\1&0\end{bmatrix}^n=\begin{bmatrix}F_n&F_{n-1}\\F_{n-1}&F_{n-2}\end{bmatrix}$$

Fórmulas para números de Fibonacci.

$$F_{n+m}=F_nF_m+F_{n-1}F_{m-1}$$

$$\sum_{i=0}^{n}F_i=F_{n+2}-1$$

$$\sum_{i=0}^{n}F_i^2=F_nF_{n+1}$$

Somas de Newton. Para $a_nx^n+\cdots+a_0$, $a_n\neq 0$, e $p_k=\sum_{j=1}^{n}x_j^k$ (raízes com multiplicidade):

$$a_np_k+\sum_{j=1}^{k-1}a_{n-j}p_{k-j}+ka_{n-k}=0\quad(1\le k\le n).$$

Para $k>n$: $\sum_{j=0}^{n}a_{n-j}p_{k-j}=0$; $p_0=n$.

Tabela de funções geradoras

$S(n,k)$: Stirling do segundo tipo, $k\ge 0$ fixo.

$[x^n]F(x)$	$F(x)$
1	$\dfrac{1}{1-x}$
a^n	$\dfrac{1}{1-ax}$
n	$\dfrac{x}{(1-x)^2}$
$\binom{n+k}{k}$	$\dfrac{1}{(1-x)^{k+1}}$
$\binom{n-a+k-1}{k-1}$	$\dfrac{x^a}{(1-x)^k}$
$\binom{k}{n}$	$(1+x)^k$
$S(n,k)$	$\dfrac{x^k}{\prod_{j=1}^{k}(1-jx)}$
$\dfrac{S(n,k)}{n!}$	$\dfrac{(e^x-1)^k}{k!}$
C_n	$\dfrac{1-\sqrt{1-4x}}{2x}$
F_n $(F_0=F_1=1)$	$\dfrac{1}{1-x-x^2}$
$\dfrac{D_n}{n!}$	$\dfrac{e^{-x}}{1-x}$
$\dfrac{1}{n}$	$\log\dfrac{1}{1-x}$

Produtos geradores via logaritmo. Se há $c_i\ge 0$ tipos de tamanho $i\ge 1$, cada um com multiplicidade ilimitada, a contagem f_n de multiconjuntos de tamanho total

n satisfaz

$$F(x) = \prod_{i\geq 1}(1-x^i)^{-c_i} = \sum_{n\geq 0} f_n x^n.$$

Defina $b_k = \sum_{d|k} d\, c_d$. Então

$$F(x) = \exp\left(\sum_{k\geq 1} \frac{b_k}{k} x^k\right), \qquad n f_n = \sum_{k=1}^{n} b_k f_{n-k}, \quad f_0 = 1.$$

Até grau N: somas de divisores para b_k em $O(N \log N)$, depois exponencial de séries formais; recorrência direta $O(N^2)$. Módulo primo p, requer $N < p$ para essas divisões.

Identidades para convolução

Convolução linear: $(A * B)_k = \sum_i A_i B_{k-i}$, com coeficientes fora do vetor iguais a zero.

Correlação (deslocamentos). Para a de tamanho $n > 0$ e b de tamanho $m > 0$, inverta a: $A_i = a_{n-1-i}$. Se $C = A * b$, então

$$\sum_i a_i b_{i+d} = C_{n-1+d}, \qquad -(n-1) \leq d \leq m-1.$$

Convolução binomial. Para obter $c_k = \sum_i \binom{k}{i} a_i b_{k-i}$, convolua $A_i = a_i/i!$, $B_i = b_i/i!$ e recupere $c_k = k!(A * B)_k$.

Translação de polinômio (Taylor shift). Para $P(x) = \sum_{i=0}^{n} a_i x^i$, forme $A_{n-i} = i! a_i$ e $B_j = c^j/j!$ $(0 \leq i, j \leq n)$. Se $C = A * B$, o coeficiente de x^k em $P(x+c)$ é $C_{n-k}/k!$, para $0 \leq k \leq n$.

Nas duas últimas receitas, os fatoriais usados devem ser invertíveis. Módulo primo p, exija grau de saída $< p$ na convolução binomial e $n < p$ no Taylor shift.

Atualização de matriz por posto 1

Lema do determinante e Sherman–Morrison: para A invertível, atualize determinante e inversa após $A \leftarrow A + uv^T$ em $O(n^2)$ se A^{-1} já é conhecida. Defina $z = A^{-1}u$, $w^T = v^T A^{-1}$ e $d = 1 + v^T z$:

$$\det(A + uv^T) = \det(A)\, d, \qquad (A + uv^T)^{-1} = A^{-1} - \frac{zw^T}{d}.$$

A fórmula da inversa requer $d \neq 0$. Válido sobre um corpo, inclusive módulo primo. Uma mudança de peso de aresta em Kirchhoff é uma atualização de posto 1 do menor laplaciano. A inversa pode não existir módulo p mesmo com contagem inteira não nula.

Combinatória e otimização

Teorema de Dilworth. Em qualquer poset finito, o tamanho máximo de uma anticadeia é igual ao número mínimo de cadeias necessárias para cobrir todos os elementos.

Erdős–Szekeres. Toda sequência de $(r-1)(s-1)+1$ números distintos contém ou uma subsequência crescente de tamanho r, ou uma subsequência decrescente de tamanho s.

Teorema de Mirsky. Em qualquer poset finito, o tamanho máximo de uma cadeia é igual ao número mínimo de anticadeias necessárias para cobrir todos os elementos.

Mediana minimiza soma de desvios absolutos.

$$\arg\min_x \sum_i |x - a_i|$$

é qualquer mediana dos a_i. Mais geralmente,

$$\arg\min_x \sum_i w_i |x - a_i|$$

é qualquer mediana ponderada.

Média minimiza soma de quadrados.

$$\arg\min_x \sum_i (x - a_i)^2$$

é

$$x = \frac{1}{n}\sum_i a_i.$$

Se x precisa ser inteiro, teste $\lfloor \bar{a} \rfloor$ e $\lceil \bar{a} \rceil$.

Intervalos disjuntos e pontos de cobertura. Para intervalos fechados, o máximo de intervalos dois a dois disjuntos é o mínimo de pontos que intersectam todos. Ordene pelo extremo direito: escolha o primeiro intervalo ainda não coberto, marque seu extremo direito e descarte os que contêm esse ponto. Para selecionar intervalos semiabertos $[l, r)$ disjuntos, aceite $l \geq r_{\text{último}}$.

Maior soma com descarte. Cada escolhido deve ser pareado com um índice anterior distinto não escolhido, sem reutilização. Percorra 0-based, insira a_i em um

min-heap e retire o mínimo em cada índice par. Ao final, some os valores mantidos; se escolher é opcional, some só os positivos.

DP

Desigualdade de Monge. Um array A é Monge se

$$A[i][j] + A[i'][j'] \le A[i][j'] + A[i'][j]$$

para $i \le i'$ e $j \le j'$.

Condição da otimização de Knuth. Para aplicar Knuth à DP de intervalos, estabeleça a monotonicidade dos índices ótimos:

$$opt[l][r-1] \le opt[l][r] \le opt[l+1][r]$$

DP contando subsequências. $\texttt{DP}[i]$ conta subsequências distintas do prefixo de tamanho i, incluindo a vazia; $\texttt{DP}[0] = 1$. Se j é a última ocorrência anterior de s_i (índices a partir de 1),

$$\texttt{DP}[i] = 2\texttt{DP}[i-1] - \begin{cases} \texttt{DP}[j-1], & j \text{ existe}, \\ 0, & \text{caso contrário.} \end{cases}$$

Convolução min-plus. Para sequências convexas, mescle as diferenças em ordem crescente; o resultado também é convexo.

Truque dos intervalos circulares. Duplique o array/lista de ângulos para tratar intervalos circulares em uma estrutura linear de tamanho $2n$.

Grafos e jogos

Havel–Hakimi. Para realizar uma sequência de graus de grafo simples, retire um vértice de maior grau d, ligue-o aos d vértices restantes de maior grau e decremente esses graus. Reinsira-os só após escolher todos os d vizinhos. Se faltarem vizinhos de grau positivo, a sequência é impossível; terminar com todos os graus zero produz uma realização. Com heap, $O((n+m)\log n)$.

Teorema do casamento de Hall. Um grafo bipartido (L, R) tem um matching saturando L sse

$$\forall S \subseteq L, \quad |N(S)| \ge |S|.$$

Fórmula de Tutte–Berge. Em um grafo não direcionado $G = (V, E)$, o mínimo número de vértices que ficam sem par em um matching é

$$\operatorname{def}(G) = \max_{S \subseteq V} \big(\operatorname{odd}(G - S) - |S|\big),$$

onde $\operatorname{odd}(G - S)$ conta as componentes conexas com número ímpar de vértices após remover S. Logo, o matching máximo tem $(|V| - \operatorname{def}(G))/2$ arestas. Em particular, existe matching perfeito sse $\operatorname{odd}(G - S) \le |S|$ para todo $S \subseteq V$ (teorema de Tutte).

Arestas possíveis e obrigatórias em matching perfeito. Fixe um matching *perfeito* M de um grafo bipartido simples (L, R). Oriente arestas fora de M de L para R, e as de M de R para L. Uma aresta fora de M pertence a algum matching perfeito sse seus extremos estão na mesma SCC. Uma aresta de M pertence a todos os matchings perfeitos sse seus extremos estão em SCCs distintas. Classifica todas as arestas em $O(V + E)$ após obter M.

Vértices que podem ficar sem par. Fixe um matching *máximo* de um grafo bipartido (L, R). Oriente arestas fora dele de L para R, e as dele de R para L. Um vértice de L fica livre em algum matching máximo sse é alcançável a partir dos livres de L. Para R, parta dos livres de R no grafo reverso. Inclua as origens. Duas buscas classificam todos em $O(V+E)$ após obter o matching; ser livre individualmente não implica poder liberar vários simultaneamente.

Teorema de König. Em um grafo bipartido, o máximo número de arestas sem extremos em comum (matching) é igual ao mínimo número de vértices que tocam todas as arestas (cobertura de vértices).

Para construir uma cobertura mínima C, sejam L, R as duas partes e fixe um matching máximo. Parta dos vértices não pareados de L; percorra arestas fora do matching de L para R e do matching de R para L. Se Z é o conjunto de vértices alcançados, $C = (L \setminus Z) \cup (R \cap Z)$. Os vértices fora de C formam um conjunto independente máximo.

Condições para trilha/circuito euleriano (direcionado). Desconsidere isolados; exija conectividade fraca:

- existe circuito euleriano sse $\deg^+(v) = \deg^-(v)$ para todo vértice;
- existe trilha aberta sse exatamente um vértice tem $\deg^+(v) = \deg^-(v) + 1$, exatamente um tem $\deg^-(v) = \deg^+(v) + 1$, e todos os outros são balanceados.

Condições para trilha/circuito euleriano (não direcionado). Desconsidere isolados; exija conectividade:

- existe circuito euleriano sse todo vértice tem grau par;
- existe trilha euleriana sse exatamente 0 ou 2 vértices têm grau ímpar.

Critério de 2-SAT. Uma instância de 2-SAT é satisfatível sse, para toda variável x, os literais x e $\neg x$ estão em SCCs diferentes do grafo de implicação.

Restrições de diferença $\leftrightarrow$ menores caminhos. Um sistema de desigualdades da forma

$$x_v - x_u \leq w$$

pode ser modelado por uma aresta $u \to v$ de peso w. Então:

- o sistema é viável sse não existe ciclo negativo;
- rode Bellman–Ford com distâncias iniciais zero em todos os V vértices; as distâncias finais dão x_v. Relaxamento na V-ésima rodada indica ciclo negativo.

Cobertura por caminhos em DAG via matching. O número mínimo de caminhos disjuntos em vértices que cobrem todos os vértices de um DAG é

$$n - |M_{\max}|.$$

Crie cópias L, R dos vértices e aresta $u_L v_R$ para cada $u \to v$ original. Cada par do matching define o sucessor de u. Reconstrua começando nos vértices não pareados em R; inclua caminhos de um único vértice. Não use fecho transitivo para exigir caminhos disjuntos no grafo original.

Todos os cortes mínimos. Após um fluxo máximo s–t, considere apenas arcos de capacidade residual positiva. $(S, V \setminus S)$ é corte mínimo sse $s \in S$, $t \notin S$ e nenhum arco residual sai de S. Logo, cada SCC fica inteira em um lado; na condensação, incluir uma componente em S obriga incluir todas as alcançáveis. Vértices alcançáveis de s ficam sempre em S; os que alcançam t ficam sempre fora. O corte é único sse esses dois conjuntos cobrem todos os vértices.

Seleção com dependências (fecho de peso máximo). Cada item v tem ganho w_v (negativo para custo); escolher u pode obrigar escolher v. Maximize a soma dos ganhos respeitando todas as dependências. Crie $s \to v$ de capacidade w_v se positivo, $v \to t$ de capacidade $-w_v$ se negativo e $u \to v$ de capacidade INF para cada dependência. Use INF $> \sum_{w_v>0} w_v$. O ganho ótimo é essa soma menos o corte mínimo. Após fluxo máximo, escolha os itens alcançáveis de s no residual; escolher nenhum é permitido.

Custos binários por corte mínimo. Minimize $\sum D_v(x_v) + \sum E_{uv}(x_u, x_v)$, $x_v \in \{0, 1\}$; lado da fonte representa 0. Inicie constante $K = \sum D_v(0)$ e coeficientes $a_v = D_v(1) - D_v(0)$. Para cada par, exija $w = E_{01} + E_{10} - E_{00} - E_{11} \geq 0$; adicione $u \to v$ com capacidade w, some E_{00} a K, $E_{10} - E_{00}$ a a_u e $E_{11} - E_{10}$ a a_v. Depois: se $a_v \geq 0$, adicione $s \to v$ com capacidade a_v; senão, $v \to t$ com capacidade $-a_v$ e some a_v a K. Ótimo: $K+$ corte mínimo; recupere os rótulos pelos lados do corte.

Capacidade em vértices. Troque v por $v_{in} \to v_{out}$ com capacidade c_v; cada aresta $u \to v$ vira $u_{out} \to v_{in}$. Para caminhos internamente disjuntos, use $c_v = 1$ nos internos e capacidades 1 nas arestas originais; fonte s_{out}, destino t_{in}.

Circulação com limites inferiores. Para $0 \leq \ell_e \leq f_e \leq u_e$, use capacidade $u_e - \ell_e$ e $b(v) = \sum_{e \to v} \ell_e - \sum_{v \to e} \ell_e$. Adicione $S \to v$ com $b(v)$ se positivo, ou $v \to T$ com $-b(v)$ se negativo. Viável sse o fluxo $S \to T$ satura todas as arestas de S; recupere $f_e = \ell_e + f'_e$. Para fluxo $s \to t$ não negativo, adicione antes $t \to s$ com capacidade suficiente (por exemplo, $\sum u_e$). Para maximizar: após obter circulação viável, guarde o fluxo F em $t \to s$, remova essa aresta e as de S, T em *ambos* os sentidos residuais; aumente $s \to t$ na rede residual restante. Resposta: $F+$ aumento. Para fluxo exatamente K, use $t \to s$ com limites $[K, K]$ na redução.

Teorema Matrix-Tree de Kirchhoff. Em um grafo não direcionado, construa L inicialmente zero. Para cada aresta $u \neq v$, some 1 a L_{uu}, L_{vv} e subtraia 1 de L_{uv}, L_{vu}; arestas paralelas contam separadamente, self-loops são ignorados. Remova a mesma linha e coluna: o determinante da matriz restante é o número de árvores geradoras. Um único vértice tem uma árvore.

Kirchhoff direcionado. Para contar árvores em que todo vértice alcança a raiz r, inicie $L = 0$; cada arco $u \to v$, $u \neq v$, soma 1 a L_{uu} e subtrai 1 de L_{uv}. Remova linha e coluna r: $\det L_{\widehat{r},\widehat{r}}$ é a contagem. Para árvores alcançáveis a partir da raiz, inverta os arcos. Paralelos contam separadamente; pesos no lugar de 1 dão a soma dos produtos dos pesos das árvores.

BEST: contar circuitos eulerianos direcionados. Em um digrafo com arestas, graus de entrada e saída iguais e vértices ativos conexos no grafo subjacente, fixe a primeira aresta saindo de r. O número de circuitos é

$$t_r \prod_{v:\, d^+(v)>0} (d^+(v) - 1)!,$$

onde t_r conta árvores direcionadas para r no subgrafo ativo. Arestas paralelas são distintas; loops entram nos graus. Para fixar apenas o vértice inicial r, multiplique por $d^+(r)$. A fórmula com primeira aresta fixa também conta circuitos identificados por rotação.

Lindström–Gessel–Viennot (LGV). Em um DAG, sejam $a_1, \ldots, a_k$ origens dis-

tintas e $b_1, \ldots, b_k$ destinos distintos; M_{ij} conta caminhos de a_i a b_j. Então

$$\det M = \sum_{\pi \in S_k} \operatorname{sgn}(\pi) N_\pi,$$

onde N_π conta famílias de caminhos $a_i \to b_{\pi(i)}$ sem compartilhar vértices. Se apenas o pareamento $a_i \to b_i$ admite famílias disjuntas, $\det M$ é sua contagem; sem essa condição, é uma soma com sinais. Versão ponderada: M_{ij} soma produtos dos pesos das arestas dos caminhos; famílias recebem o produto dos pesos de seus caminhos.

Contar árvores geradoras mínimas. Processe pesos em ordem crescente, como Kruskal. Para cada peso, contraia as componentes formadas por pesos menores; descarte self-loops e preserve arestas paralelas. Em cada componente conexa do multigrafo desse peso, conte árvores geradoras por Kirchhoff e multiplique as contagens. Só depois una no DSU todas as arestas do grupo. Se o grafo original não ficar conexo, a resposta é zero.

Jogos com ciclos. Em um grafo finito de estados com turnos alternados, sem jogadas perde e partida infinita empata. Marque terminais como derrotas e propague pelos arcos reversos: um estado vence se alcança uma derrota; perde se todos os sucessores vencem. Após esgotar a propagação, estados não classificados empatam. Fila e contador de sucessores ainda não classificados dão $O(V + E)$; inclua a vez no estado quando necessário.

Teorema de Sprague–Grundy. Todo jogo imparcial finito em normal play é equivalente a uma pilha de Nim cujo tamanho é seu número de Grundy; o número de Grundy é o mex dos números de Grundy das jogadas possíveis. O número de Grundy de uma soma disjunta é o xor dos números de Grundy das componentes.

Jogo UGV. Considere o jogo de mover ao longo de arestas de um grafo sem poder repetir vértices, e o jogador que não puder mover perde. O segundo jogador tem uma estratégia vencedora sse existe um matching máximo no grafo que deixa o vértice inicial insaturado.

Strings

Matching com wildcards por convolução. Padrão P de tamanho $m \geq 1$, texto T de tamanho $n \geq m$; codifique letras distintas por $1, \ldots, \sigma$ e wildcard por 0, em ambos. Cada wildcard substitui *um* caractere. Para $0 \leq s \leq n - m$,

$$D_s = \sum_{i=0}^{m-1} P_i T_{s+i} (P_i - T_{s+i})^2;$$

há match sse $D_s = 0$ nos inteiros. Para $r = 1, 2, 3$, tome $A_r = \operatorname{rev}(P^r)$ e $B_r = T^r$, com potências por elemento. Três convoluções lineares dão

$$C = A_3 * B_1 - 2(A_2 * B_2) + A_1 * B_3, \quad D_s = C_{m-1+s}.$$

Custo $O((n + m) \log(n + m))$ por módulo. Para NTT determinística, use primos distintos cujo produto seja $> m\sigma^2(\sigma - 1)^2$ e exija zero em todos: esse é um limite superior de $D_s \geq 0$. Um módulo insuficiente pode aceitar falsos matches.

Strings que comutam. Para strings não vazias x, y, $xy = yx$ sse $x = z^a$ e $y = z^b$ para alguma string não vazia z e inteiros $a, b \geq 1$: ambas repetem o mesmo bloco primitivo. A raiz primitiva (bloco mais curto cuja repetição forma a string inteira) é única.

Teorema de Fine–Wilf. Se uma string tem períodos p e q, e seu tamanho é pelo menos

$$p + q - \gcd(p, q),$$

então ela também tem período $\gcd(p, q)$.

Hash polinomial de substring. Se

$$H[i + 1] = H[i] \cdot B + s[i],$$

então

$$hash(l, r) = H[r] - H[l] \cdot B^{r-l}$$

para o intervalo semiaberto $[l, r)$.

Substrings distintas. Usando array de sufixos:

$$\frac{n(n+1)}{2} - \sum lcp[i]$$

Usando autômato de sufixos:

$$\sum_{i \neq \text{raiz}} \big(\texttt{len}(i) - \texttt{len}(\texttt{link}(i))\big)$$

Notas de autômato de sufixos. O automato de sufixos é um DAG. Cada caminho a partir da raiz representa uma substring. Cada nó representa um conjunto de `endpos`. `link(i)` é o maior sufixo com um conjunto `endpos` diferente.

Geometria

Transformações da distância Manhattan. Em 2D,

$$|x_1 - x_2| + |y_1 - y_2| = \max\Big(|(x+y)' - (x+y)|,\, |(x-y)' - (x-y)|\Big)$$

Na prática, para a maior distância entre pares:

$$\max_{i,j} |x_i - x_j| + |y_i - y_j| = \max(D_+, D_-)$$

onde

$$D_+ = \max(x+y) - \min(x+y), \qquad D_- = \max(x-y) - \min(x-y)$$

Teorema de Euler para grafos planares. Num grafo planar com v vértices, e arestas, f faces e k componentes conexas,

$$v - e + f = 1 + k$$

Teorema de Pick. Para um polígono simples na grid,

$$A = I + \frac{B}{2} - 1,$$

onde I é o número de pontos de grid internos e B é o número de pontos de grid na fronteira.

Pontos de grid em um segmento. O número de pontos de grid no segmento de (x_1, y_1) até (x_2, y_2) é

$$\gcd(|x_1 - x_2|, |y_1 - y_2|) + 1.$$

Distância de ponto a reta. Distância do ponto P à reta passando por A, B:

$$\frac{|\operatorname{cross}(B - A, P - A)|}{|B - A|}$$

Fórmula de Heron. Para lados a, b, c e semiperímetro $s = (a+b+c)/2$, a área é

$$\sqrt{s(s-a)(s-b)(s-c)}.$$

Lei dos cossenos.

$$c^2 = a^2 + b^2 - 2ab\cos C$$

Ângulo pelo produto escalar.

$$\cos\theta = \frac{a \cdot b}{|a||b|}$$

Raio da circunferência circunscrita e inscrita. Para área do triângulo Δ:

$$R = \frac{abc}{4\Delta}, \qquad r = \frac{\Delta}{s}$$

onde $s = \frac{a+b+c}{2}$.

Comprimento da corda. Para raio r e ângulo central θ:

$$2r\sin\frac{\theta}{2}$$

Teorema de Helly em $\mathbb{R}^d$. Para uma família finita de convexos em $\mathbb{R}^d$, se toda subfamília de até $d+1$ conjuntos tem interseção não vazia, a família inteira também tem.

Teorema de Carathéodory. Se um ponto pertence ao fecho convexo de um conjunto em $\mathbb{R}^d$, então ele pertence ao fecho convexo de no máximo $d+1$ pontos desse conjunto.

Ehrhart: contar pontos inteiros em dilatações. Se $P \subset \mathbb{R}^D$ é um politopo convexo limitado de dimensão d com vértices inteiros, então $L_P(t) = |tP \cap \mathbb{Z}^D|$, para inteiro $t \geq 0$, é um polinômio de grau d. Se os vértices são racionais e $q > 0$ é inteiro com qP integral (use o MMC dos denominadores das coordenadas), a contagem é um quase-polinômio de período divisor de q: para cada resíduo r, $L_P(r + qk)$ é polinomial em k de grau no máximo d. Em módulo primo, as abscissas usadas precisam ser distintas módulo p. O resultado vale para dilatações de um politopo fixo; parâmetros arbitrários em restrições não garantem essa forma.

Debugging

Confira cada grupo mesmo quando parecer correto. Reproduza a falha e reduza o caso; compare estados intermediários com uma simulação manual ou força bruta.

Enunciado e modelo

- ☐ Releia o que deve ser contado/otimizado: ordem importa? Repetições são distintas? A escolha vazia é permitida?
- ☐ Confira as hipóteses do algoritmo: pesos negativos, monotonicidade, conectividade, empates. Elas vêm do enunciado ou foram presumidas?

☐ Siga à mão um caso pequeno; confira o significado de cada estado, transição, base e resposta extraída.

Limites e índices

☐ Force menor tamanho permitido, um elemento, todos iguais, valores extremos e resposta inexistente.

☐ Confira cada acesso: 0/1-based, n/m trocados, $i-1$, $i+1$, sentinelas $-1/n$, tamanho $n+1/2n$.

☐ Refaça os extremos dos intervalos: $[l, r)$ ou $[l, r]$? Teste vazio, unitário e intervalo inteiro; confira $<$ versus $\leq$.

Aritmética

☐ Verifique o tipo de cada produto, soma e shift *antes* da atribuição; cabe no tipo? Alguma operação usa `INF`?

☐ Confira módulo, resíduos negativos e divisão modular (o inverso existe?). Na divisão inteira, o arredondamento é o desejado?

☐ Teste denominador zero, raiz de negativo, `NaN`/infinito e cancelamento numérico; confira tolerância e precisão da saída.

Estado e atualizações

☐ Rode o mesmo caso sozinho e após outro: globais, containers, flags e lazy tags foram reinicializados?

☐ Cada valor foi inicializado antes de ser lido? Algum estado impossível foi usado como válido?

☐ A atualização usa o estado anterior ou já alterado? Confira direção do loop, ordem de composição e rollback; referências/iteradores continuam válidos?

Representação e ordem

☐ Teste componentes desconexas, isolados, self-loops e arestas paralelas; direção, aresta reversa e identificação pelo pai estão corretas?

☐ Force empates: a ordenação tem desempate correto e comparador estrito? Eventos na mesma coordenada entram/saem na ordem certa?

☐ Após ordenar/comprimir, os IDs e pesos ainda correspondem? A compressão perdeu distâncias necessárias? A reconstrução produz uma solução válida?

Tempo e memória

☐ Cada loop progride? Recursão revisita estados ou excede a pilha? Teste cadeia, estrela e entrada adversária para a estrutura.

☐ Conte cópias, alocações, limpeza por teste e fatores extras de $\log n$; confira o custo total de todos os casos e o argumento amortizado.

☐ Calcule o tamanho real dos buffers/estados; confira se containers crescem sem limite ou se estruturas locais grandes ocupam a pilha.

Execução e saída

☐ Recompile o arquivo correto e rode com sanitizers. Leia todos os campos e casos? Removeu prints de debug? Formato e precisão estão corretos?

☐ Em stress testing, o gerador respeita as restrições e a força bruta é independente? Respostas múltiplas exigem validar a solução, não igualdade textual.